

Identify and localize COVID-19 abnormalities on chest radiographs

Puneesh Khanna

Final Report

Liverpool John Moores University – M.Sc. in Machine Learning & AI

December 2021

DEDICATION

I dedicate my dissertation work to my wife Jyoti Khanna and to my kids Viraj Khanna (son) and Mishka Khanna (daughter). I would like to thank my managers Ashish Sharma ([linkedin.com/in/ashish-sharma-315689](https://www.linkedin.com/in/ashish-sharma-315689)) and Chawan Mehra ([linkedin.com/in/chawan-mehra-71a3b9](https://www.linkedin.com/in/chawan-mehra-71a3b9)) at Intel Technologies for their encouragement and support in this quest. I would also like to express my gratitude to Nescafe Coffee which helped me burn midnight oil and write this report.

ACKNOWLEDGEMENTS

I would like to thank my thesis supervisor Sidhant Chatterjee ([linkedin.com/in/sidhant-chatterjee](https://www.linkedin.com/in/sidhant-chatterjee)) for his guidance and support throughout the project and going through multiple revisions of the report. I would like to acknowledge Dr. Ahmed Kaky ([linkedin.com/in/ahmed-kaky-aljaaf-5069191ba](https://www.linkedin.com/in/ahmed-kaky-aljaaf-5069191ba)) and Dr. Rupal Bhargava ([linkedin.com/in/rupee-bhargava-051021105](https://www.linkedin.com/in/rupee-bhargava-051021105)) for their amazing lectures and supervision during the course. I really appreciate their patience and understanding to numerous questions and doubts. I am grateful to Liverpool John Moore's University and Upgrad for providing such an opportunity to working professionals to pursue M.Sc in Machine Learning and AI. Last but not the least, I would like to thank my colleague Pooja Agarwal ([linkedin.com/in/pooja-agarwal-5883972](https://www.linkedin.com/in/pooja-agarwal-5883972)) who also pursued this course and helping and encouraging each other to overcome any obstacles.

ABSTRACT

Five times more deadly than the flu, COVID-19 causes significant morbidity and mortality. Like other pneumonias, pulmonary infection with COVID-19 results in inflammation and fluid in the lungs. COVID-19 looks very similar to other bacterial and viral pneumonias on chest radiographs and hence difficult to diagnose. A computer vision deep learning model to predict and categorize COVID-19 on chest radiographs can assist doctors provide a quick and confident diagnosis. In addition, object detector models can further support doctors and radiologists with localization of the disease, such as with a visual bounding box. As a result, patients can get the right treatment before the most adverse effects of the COVID-19 virus take hold. Ensembled EfficientNet-v2m and EfficientNet-v1 B5 classification models trained using an auxiliary head, attention layer and class weights achieved highest mean average precision score of 58.5% and AUC-ROC score of 81.7% on validation dataset to comprehend 4-classes of overall opacity appearance on chest X-rays. Ensembled YOLO and EfficientDet detection models achieved highest mean average precision score of 56.4% on validation dataset to localize the opacities in terms of bounding boxes on chest X-rays. Submission of both classification and localization predictions on private test dataset by the trained models attained a mean average precision score of 62.3% which stands in top-15 results in the leader board ranking. The study concludes that deep learning and computer vision models to classify and localize the COVID abnormalities on chest X-rays have promising results and can play a significant role to combat the COVID-19 epidemic.

TABLE OF CONTENTS

DEDICATION	ii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	ix
LIST OF TABLES	xiii
LIST OF ABBREVIATIONS	xv
Chapter 1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Aim and Objectives	3
1.4 Significance of the Study	4
1.5 Scope of the Study	4
1.6 Structure of the Study	4
Chapter 2 LITERATURE REVIEW	6
2.1 Introduction	6
2.2 Computer Vision Networks	9
2.2.1 Understanding CNNs	11
2.2.2 Simple CNN architecture for digit recognition	13
2.3 Review of Image Classification Models	14
2.3.1 ImageNet Dataset and Evaluation	14
2.3.2 ResNet Architectures	14
2.3.3 DenseNet Architectures	16
2.3.4 EfficientNet Architectures	18
2.4 Review of Object Detection Models	21
2.4.1 COCO Dataset and Evaluation	21
2.4.2 YOLO	22
2.4.3 EfficientDet	25
2.4.4 Survey on Object Detection Models	27
2.5 Data Augmentation Techniques	28
2.6 Transfer Learning	30

2.7 Ensembling Techniques	31
2.7.1 k-fold Cross Validation Ensemble	32
2.7.2 Model Averaging Ensemble	33
2.7.3 Weighted Averaging Ensemble	33
2.7.4 Weighted Box Fusion	33
2.8 Computer Vision in Healthcare	34
2.9 Predictive Modelling on Chest Images Datasets	34
2.9.1 CheXNet and ChestNet for Pneumonia Classification on CXRs	34
2.9.2 Deep Learning Ensembles for Tuberculosis Classification on Chest Radiographs	35
2.9.3 Foreign Object Detection in Chest X-rays	35
2.9.4 Object Detection Ensembles for Pneumonia Opacity Detections in CXRs	36
2.10 Predictive Modelling on COVID Datasets	37
2.10.1 Datasets used in existing research	37
2.10.2 Diagnosing COVID-19 pneumonia - AlexNet Transfer Learning	38
2.10.3 COVID19-CT DataSet - Random Initialized Networks vs Transfer Learning	39
2.10.4 Multiple CNN architectures on SARS-CoV-2 CT and COVID19-CT datasets	40
2.10.5 COVID GANs to classify normal and COVID CXRs	41
2.10.6 ResNet50 to classify normal, pneumonia and COVID CXRs	42
2.10.7 Deep Learning Ensembles to classify normal, bacterial and COVID pneumonia CXRs	43
2.10.8 CoroNet and CovidNet to classify normal, viral, bacterial and COVID pneumonia CXRs	44
2.11 Review of top solutions	44
2.12 Discussion	45
2.13 Summary	45
Chapter 3 RESEARCH METHODOLOGY	46
3.1 Introduction	46
3.2 Dataset Description	46
3.3 Data Understanding	47
3.4 Data Preparation and Pre-Processing	50
3.5 Feature Engineering	52
3.6 Flowchart and Training Pipeline	52
3.7 Training Classification Models	54
3.7.1 ResNet	54
3.7.2 DenseNet	54

3.7.3 EfficientNet	55
3.7.4 Parameters for Training Classification Models	55
3.7.5 Ensembling Classification Models	56
3.7.6 Binary Classification Model to predict presence bounding boxes	57
3.8 Training Object Detection Models	57
3.8.1 YOLO	57
3.8.2 EfficientDet	58
3.8.3 Parameters for Training Object Detection Models	58
3.8.4 Weighted Box Fusion	59
3.9 Evaluation Metrics	59
3.9.1 Accuracy	59
3.9.2 AUC-ROC	59
3.9.3 Precision/Recall/F1 score	60
3.9.4 Mean average precision (mAP)	61
3.9.5 Summary of evaluation metrics for various models	62
3.10 Requirements Resources	63
3.11 Summary	64
Chapter 4 ANALYSIS AND IMPLEMENTATION	65
4.1 Introduction	65
4.2 Exploratory Data Analysis	65
4.3 Data Preparation and Cleaning	74
4.3.1 Converting DICOM to png images	74
4.3.2 Removing duplicate images	74
4.3.3 Generation of segmentation masks from bounding boxes annotations	75
4.3.4 Normalization of bounding boxes annotations	75
4.4 Data Partitioning	76
4.5 Data Augmentation Pipeline	77
4.6 Image Classification Models Implementation	79
4.6.1 EfficientNet-v2m	79
4.6.2 EfficientNet-v1 B5	87
4.6.3 ResNet50 and DenseNet	91
4.6.4 EfficientNet-v2m for binary classification	91
4.7 Object Detection Models Implementation	93
4.7.1 YOLO-v5l	93
4.7.2 YOLO-v5x	97

4.7.3 EfficientDet-D3	99
4.7.4 EfficientDet-D5	102
4.8 Summary	104
Chapter 5 RESULTS AND DISCUSSIONS	105
5.1 Introduction	105
5.2 Final Classification Models	105
5.2.1 EfficientNet v2m - Test Dataset Evaluation	105
5.2.2 EfficientNet v1 B5 - Test Dataset Evaluation	108
5.2.3 Weighted Ensemble of EfficientNet v2m and v1 B5 - Test Dataset Evaluation	110
5.2.4 EfficientNet v2m comparison with ResNet50/DenseNet	111
5.3 Final Detection Models	112
5.3.1 EfficientDet-D3 and EfficientDet-D5 - Test Dataset Evaluation	112
5.3.2 YOLO-v5l and YOLO-v5x - Test Dataset Evaluation	113
5.3.3 Weighted box fusion of EfficientDet and YOLO - Test Dataset Evaluation	114
5.4 Final Classification and Detection Models	115
5.5 Error Analysis	117
5.5.1 Classification Models Errors	117
5.5.2 Detection Models Errors	118
5.6 Summary	121
Chapter 6 CONCLUSIONS AND RECOMMENDATIONS	122
6.1 Introduction	122
6.2 Discussion and Conclusion	122
6.3 Contribution to Knowledge	123
6.4 Future Recommendations	124
REFERENCES	127
Appendix A	Error! Bookmark not defined.
RESEARCH PLAN	Error! Bookmark not defined.
Appendix B	Error! Bookmark not defined.
RESEARCH PROPOSAL	Error! Bookmark not defined.

LIST OF FIGURES

Figure 2.1 AI ML Deep Learning.....	6
Figure 2.2 Training a Neural Network	7
Figure 2.3 Artificial Neuron	8
Figure 2.4 Training vs Inference	8
Figure 2.5 Computer Vision - Image Classification.....	9
Figure 2.6 Computer Vision - Image Localization.....	10
Figure 2.7 Computer Vision - Object Detection.....	10
Figure 2.8 Computer Vision - Image Segmentation.....	10
Figure 2.9 CNN - Convolution Operation	12
Figure 2.10 CNN - Convolution (Stride, Padding) + Maxpool (Li et al., n.d.).....	12
Figure 2.11 CNN - Digit Recognition	13
Figure 2.12 ResNet - Skip Connection.....	15
Figure 2.13 ResNet - Residual Blocks	15
Figure 2.14 ResNet50 - Full Architecture	16
Figure 2.15 DenseNet - Concatenation of feature maps.....	16
Figure 2.16 DenseNet - Dense Block with 6 dense layers	17
Figure 2.17 DenseNet - Full Architecture	17
Figure 2.18 DenseNet - Comparison with Resnet	18
Figure 2.19 EfficientNet - Compound Scaling (Tan and Le, 2019).....	18
Figure 2.20 EfficientNet - Depthwise Convolution vs Normal Convolution.....	19
Figure 2.21 EfficientNet-B0 Full Architecture	19
Figure 2.22 EfficientNet - Model size vs Accuracy Comparison (Tan and Le, 2019).....	20
Figure 2.23 Intersection Over Union.....	22
Figure 2.24 YOLO - 7x7 grid cells.....	23
Figure 2.25 YOLO - bounding boxes of a grid cell share class predictions.....	24
Figure 2.26 YOLOv3 - Architecture	24
Figure 2.27 YOLOv5 comparison with EfficientDet	25
Figure 2.28 EfficientDet - Architecture.....	26
Figure 2.29 EfficientDet - Comparison with other Object Detectors.....	26
Figure 2.30 Survey of Object Detection Models on COCO Dataset (Zaidi et al., 2021).....	27
Figure 2.31 Data Augmentation - Random Flip Left Right and Up Down.....	28
Figure 2.32 Data Augmentation - Random Brightness	28

Figure 2.33 Data Augmentation - Random Contrast.....	29
Figure 2.34 Data Augmentation - Random Saturation.....	29
Figure 2.35 Transfer Learning.....	30
Figure 2.36 Transfer Learning using ImageNet weights vs Training from scratch with random initialized weights (Kornblith et al., n.d.).....	31
Figure 2.37 Ensembling multiple models.....	32
Figure 2.38 Ensembling - k fold.....	32
Figure 2.39 Weighted Box Fusion - Blue boxes different models' predictions, red box golden truth (Solovyev et al., 2021)	33
Figure 2.40 Foreign Object Detection on CXRs using Faster R-CNN	36
Figure 2.41 Predictive Modelling COVID CXRs - AlexNet Covid vs Normal.....	38
Figure 2.42 Predictive Modelling COVID CXRs - ResNet50 Confusion Matrix.....	42
Figure 2.43 Predictive Modelling COVID CXRs - Pre-processed Cropped Lung Images (Rajaraman et al., 2020)	43
Figure 3.1 Chest X-ray	46
Figure 3.2 Chest X-ray Normal/Negative Class.....	48
Figure 3.3 Chest X-ray Typical Class	48
Figure 3.4 Chest X-ray Indeterminate Class	48
Figure 3.5 Chest X-ray Atypical Class.....	49
Figure 3.6 Training Dataset Class Distribution.....	49
Figure 3.7 Training Dataset Bounding Box Distribution	50
Figure 3.8 Stratified k-fold.....	51
Figure 3.9 Research Methodology Flowchart	53
Figure 3.10 Training Pipeline.....	54
Figure 3.11 Softmax Activation	55
Figure 3.12 Categorical Cross Entropy	55
Figure 3.13 Ensembling Classification Models.....	56
Figure 3.14 Ensembling Object Detection Models	59
Figure 3.15 AUC ROC	60
Figure 3.16 Precision Recall Curve.....	62
Figure 4.1 Visualizing training CXRs labelled as negative class.....	66
Figure 4.2 Visualizing training CXRs labelled as typical class with bounding boxes.....	66
Figure 4.3 Visualizing training CXRs labelled as indeterminate class with bounding boxes..	67
Figure 4.4 Visualizing training CXRs labelled as atypical class with bounding boxes	67

Figure 4.5 Scatter plot of image size	68
Figure 4.6 Distribution of classification classes in training dataset	68
Figure 4.7 Distribution % of classification classes in training dataset.....	69
Figure 4.8 Count of train images with number of bounding boxes.....	69
Figure 4.9 Distribution of total bounding boxes across the classification classes	70
Figure 4.10 Box plot of bounding boxes sizes	70
Figure 4.11 Images with large bounding boxes annotations	71
Figure 4.12 Images with small bounding boxes annotations	71
Figure 4.13 Bounding boxes heatmap	72
Figure 4.14 Sampling of training images across patient gender	73
Figure 4.15 Sampling of training images across modality	73
Figure 4.16 Sampling of training images across monochrome	73
Figure 4.17 No bounding boxes across classification classes	74
Figure 4.18 Segmentation Masks	75
Figure 4.19 Augmentation - Shift Rotate Scale.....	77
Figure 4.20 Augmentation – Horizontal Flip	77
Figure 4.21 Augmentation – Random Brightness Contrast.....	78
Figure 4.22 Augmentation – Coarse Dropout	78
Figure 4.23 Augmentation – Random Resized Crop.....	78
Figure 4.24 Segmentation Head during training of classification models	80
Figure 4.25 EfficientNetv2m Training/Validation loss and Validation mAP across epochs- fold0.....	85
Figure 4.26 EfficientNetv2m Training/Validation loss and Validation mAP across epochs- fold3.....	85
Figure 4.27 EfficientNet-v2m Learning Rate per epoch	85
Figure 4.28 EfficientNet Full Training Pipeline.....	86
Figure 4.29 EfficientNet-B5 Training/Validation loss and Validation mAP across epochs- fold0.....	90
Figure 4.30 EfficientNet-B5 Training/Validation loss and Validation mAP across epochs- fold4.....	90
Figure 4.31 YOLO-v5l mAP metric across training epochs	94
Figure 4.32 YOLO-v5l Box and Object loss across training epochs	95
Figure 4.33 YOLO-Mosaic Augmentation.....	96
Figure 4.34 YOLO Full Training Pipeline	96

Figure 4.35 YOLO-v5x mAP metric across training epochs	98
Figure 4.36 YOLO-v5x Box and Object loss across training epochs	99
Figure 4.37 EfficientDet-D3 fold-0 Box loss and mAP plot across training epochs	101
Figure 4.38 EfficientDet-D3 fold-3 Box loss and mAP plot across training epochs	101
Figure 4.39 EfficientDet-D3 Learning Rate per epoch	101
Figure 4.40 EfficientDet-D5 fold-2 Box loss and mAP plot across training epochs	103
Figure 4.41 EfficientDet-D5 fold-4 Box loss and mAP plot across training epochs	103
Figure 4.42 EfficientDet-D5 Learning Rate per epoch	104
Figure 5.1 mAP Results of EfficientNet-v2m experiments on OOF Validation Dataset.....	106
Figure 5.2 AUC-ROC Results of EfficientNet-v2m experiments on OOF Validation Dataset	107
Figure 5.3 F1-Score Results of EfficientNet-v2m experiments on OOF Validation Dataset	107
Figure 5.4 Test Dataset mAP using final EfficientNet-v2m 5-fold models	108
Figure 5.5 Test Dataset mAP using final EfficientNet-v1 B5 5-fold models	110
Figure 5.6 Test Dataset mAP: Weighted Ensemble of EfficientNetv2m and EfficientNetv1-B5	110
Figure 5.7 mAP Comparison of EfficientNet with ResNet and DenseNet on Validation Dataset	111
Figure 5.8 AUC-ROC Comparison of EfficientNet with ResNet and DenseNet on Validation Dataset	111
Figure 5.9 Test Dataset mAP using ensembling of EfficientDet-D3 and EfficientDet-D5 ...	113
Figure 5.10 Test Dataset mAP using ensembling of YOLO-v5l and YOLO-v5x	114
Figure 5.11 Test Dataset mAP: Weighted box fusion of EfficientDet and YOLO	115
Figure 5.12 Test Dataset Submission Score	115
Figure 5.13 Test Dataset mAP: Final Classification and Detection Models	116
Figure 5.14 Test Dataset Leaderboard Ranking	116
Figure 5.15 Test Dataset MAP: Final Classification and Pruned Detection Models	117
Figure 5.16 YOLO-v5l fold-0 model - confusion matrix on validation dataset.....	119
Figure 5.17 Visualization of actual opacities on validation dataset	119
Figure 5.18 Visualization of predicted opacities by detection model on validation dataset ..	120
Figure 5.19 Visualization of actual opacities on validation dataset	120
Figure 5.20 Visualization of predicted opacities by detection models on validation dataset.	121

LIST OF TABLES

Table 2.1 EfficientNet - Comparison of different variants.....	21
Table 2.2 Transfer Learning Top-1 Accuracy Results via EfficientNet.....	31
Table 2.3 Predictive Modelling - COVID Datasets.....	37
Table 2.4 Predictive Modelling - CT Dataset Performance Comparison between random initialized networks (Random.) and ImageNet Pretrained networks (Transfer.)	39
Table 2.5 Predictive Modelling COVID CXRs - Multiple CNNs.....	40
Table 2.6 Predictive Modelling COVID CXRs - Covid GAN Test Dataset Results	42
Table 2.7 CoroNet DataSet.....	44
Table 3.1 Training Dataset Annotations.....	51
Table 3.2 Parameters for training classification models.....	56
Table 3.3 Parameters for training object detection models	58
Table 3.4 Classification Report	60
Table 3.5 Precision Recall Report	61
Table 3.6 Evaluation of Classification Models	62
Table 3.7 Evaluation of Object Detection Models	63
Table 4.1 Data Partitioning – Stratified 5-fold	76
Table 4.2 EfficientNetv2m mAP comparison with and without segmentation head	81
Table 4.3 EfficientNetv2m AUC and Accuracy comparison with and without segmentation head.....	81
Table 4.4 EfficientNetv2m F1 scores on validation dataset with and without class weights...	83
Table 4.5 EfficientNetv2m mAP scores on validation dataset with and without class weights	83
Table 4.6 EfficientNetv2m AUC/Accuracy on validation data with and without class weights	84
Table 4.7 EfficientNet B5 mAP scores on validation dataset with class weights	88
Table 4.8 EfficientNet B5 AUC scores on validation dataset with class weights.....	88
Table 4.9 EfficientNet B5 Accuracy scores on validation dataset with class weights.....	89
Table 4.10 EfficientNet B5 F1 scores on validation dataset with class weights	89
Table 4.11 ResNet50 and DenseNet201 results on validation dataset	91
Table 4.12 Metrics of EfficientNet-v2m binary classification model to predict negative class vs rest on validation dataset.....	92

Table 4.13 Confusion Matrix and F1 score of EfficientNet-v2m binary model (fold-0) on validation dataset	92
Table 4.14 YOLO-v5l Highest mAP @IOU=0.5 on Validation Dataset	94
Table 4.15 YOLO-v5x Highest mAP @IOU=0.5 on Validation Dataset	98
Table 4.16 EfficientDet-D3 Highest mAP @IOU=0.5 on Validation Dataset	100
Table 4.17 EfficientDet-D5 Highest mAP @IOU=0.5 on Validation Dataset	102
Table 5.1 Results of EfficientNet-v2m experiments on OOF Validation Dataset	106
Table 5.2 Results of final EfficientNet-v2m models on OOF Validation Dataset	108
Table 5.3 Results of EfficientNet-v1 B5 experiments on OOF Validation Dataset	109
Table 5.4 Results of final EfficientNet-v1 B5 models on OOF Validation Dataset	109
Table 5.5 Results (mAP@IOU=0.5) of EfficientDets on OOF Validation Dataset	113
Table 5.6 Results (mAP@IOU=0.5) of YOLOs on OOF Validation Dataset	114
Table 5.7 EfficientNetv2m fold-0 model - confusion matrix and F1 score on validation dataset	118

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area under the Curve
BCE	Binary Cross Entropy
BiFPN	Bidirectional Feature Pyramid Network
BIMCV	Medical Imaging Databank of the Valencia Region
CCE	Categorical Cross Entropy
COCO	Common Objects in Context
COVID-19	Coronavirus Disease 2019
CNN	Convolutional Neural Networks
CRISP-DM	Cross-Industry Standard Process for Data Mining
CT Scan	Computed Tomography Scan
CXR	Chest X-ray
DETR	Detection Transformer
DICOM	Digital Imaging and Communications in Medicine
EDA	Exploratory Data Analysis
FN	False Negatives
FP	False Positives
GAN	Generative Adversarial Network
IOU	Intersection Over Union
mAP	Mean Average Precision
MS COCO	Microsoft Common Objects in Context
NAS	Network Architecture Search
OOF	Out of fold
ResNet	Residual Neural Network
RCNN	Region Based Convolutional Neural Networks
RICORD	RSNA International COVID-19 Open Radiology Database
ROC	Receiver Operating Characteristics
RSNA	Radiological Society of North America
RTPCR	Reverse transcription-polymerase chain reaction
TP	True Negatives
TP	True Positives
YOLO	You only look once

Chapter 1 INTRODUCTION

1.1 Background

Coronavirus disease 2019 (COVID-19) is a global epidemic and has claimed more than 40 lacs deaths worldwide. Presently, COVID-19 can be diagnosed via polymerase chain reaction to detect genetic material from the virus. However, it can take a few hours and sometimes days before the molecular test results are back. By contrast, chest radiographs can be obtained in minutes. Chest radiographs can play an important role for the detection and quantification of COVID-19 abnormalities (Cai et al., 2020; Pan et al., 2020) to augment and support the current gold standard Reverse transcription-polymerase chain reaction (RT-PCR) testing.

Deep learning and neural networks have demonstrated a lot of value for imaging analysis, classification, and detection in the medical industry. Such models are mainly based upon supervised based learning of imaging datasets to help predict the patient infection of a disease or virus such as tuberculosis, pneumonia, COVID-19 etc. and assist further diagnosis.

There have been many publications related to automated medical imaging analysis on COVID datasets via deep learning and machine learning algorithms; however some have underperformed over unseen data (Suchá et al., 2020; Wynants et al., 2020). Authors of (Waheed et al., 2020) improved the classification accuracy on a smaller dataset by applying GANs to augment the dataset and achieve distinction between COVID chest X-ray and normal chest X-ray images. Authors of (Alshazly et al., 2021) applied various state of architecture CNN models such as ResNet, ShuffleNet, SqueezeNet, DenseNet etc. on a couple of COVID radiograph datasets and compared the accuracy results on the 2 labelled classes of covid vs non-covid images. Another related research (Usama et al., 2020) was carried out via ResNet architecture based transfer learning to differentiate between 3 groups of normal, pneumonia and COVID-19 images and not just between covid vs non-covid classes. Whereas ensembling and pruning techniques were applied on multiple models to learn and generalize on a labeled dataset for 3 classes that are normal, bacterial pneumonia or COVID-19 typical abnormalities (Rajaraman et al., 2020).

Deep learning models are getting rapidly developed and prototyped to support medical decision making for COVID-19 diagnosis. More study is required to understand whether deep learning models can categorize COVID abnormalities on chest X-rays to determine the severity of the

virus on a patient. Also, object detection models have potential scope to learn and localize the opacities on COVID datasets.

1.2 Problem Statement

Prediction modelling for COVID imaging datasets is growing rapidly to support medical decision making and assist in combating COVID epidemic. This research intends to solve image classification and object detection problems on a COVID chest X-rays (CXRs) digital images dataset via deep learning computer vision models. It is a supervised learning problem that is the outputs corresponding to the input images are labelled and known during training of those models. Computer vision models are expected to learn the relationship between the input chest X-ray images and the expected outputs during training and will be evaluated based upon their performance and predictions on unseen chest radiographs. Classification models will be expected to predict the overall class of COVID-19 pneumonia among 4 labelled classes on chest X-ray images whereas object detection models will need to predict the opacities on those images in terms of bounding boxes.

- **Datasets** - One of the datasets which will be used by this research is RICORD Database (Chung et al., 2020; Tsai et al., 2021). RICORD data set is freely available to the data science community as a research resource for model prototyping on COVID-19 chest imaging.

Another dataset which will be used by this research is BIMCV-COVID19 Data (Vayá et al., 2020) which is one of the largest individual dataset of COVID-19 chest radiograph images freely available.

To the best of my knowledge, combination of the above datasets that is annotated with both image level labels as well as with localization of opacities in the images is one of the largest and challenging COVID-19 datasets that has been recently published around May 2021 and requires modeling of both classification and object detection tasks.

- **Classification task** requires building deep learning models based upon CXRs to classify and differentiate them into 4 classes that are 'Negative for Pneumonia', 'Typical Appearance', 'Indeterminate Appearance' or 'Atypical Appearance'.

- **Object detection task** requires building efficient and robust models to predict visual bounding boxes and determine the localization of opacities and thus these models can augment the analysis of classification models.
- To summarize, this research intends to address an object detection and classification problem that can identify and localize COVID-19 and pneumonia abnormalities on CXRs.

1.3 Aim and Objectives

The primary objective of this research is to develop reliable, generalizable, and accurate deep learning models that can perform classification and localization of COVID-19 abnormalities on chest radiographs.

Classification models will categorize the radiographs into four categories which are negative for pneumonia, typical for pneumonia, indeterminate, or atypical for COVID-19. Detection and localization of COVID-19 opacities via object detection models would further assist doctors provide a quick and confident diagnosis.

The research objectives are as follows:

- Dataset understanding and annotations - This step will require understanding the train dataset and the images that will be used to train both classification and object detection models.
- Dataset preparation and pre-processing - This step will mainly require converting the medical DICOM images to png or jpeg formats of different sizes and normalizing the pixels of the images in a specific range such as 0 to 1 or -1 to +1 etc.
- Data augmentation - This step will mainly require building a data augmentation pipeline which will be feeding to the model training pipeline. Data augmentation techniques are strong methods that help in diversifying the training dataset so that the trained models generalize and perform better on unseen data.
- Training classification models based upon ResNet50, DenseNet and EfficientNet architectures on the train dataset and comparing their performance and accuracy on validation and test datasets.

- Training object detection models based upon EfficientDet and YOLO architectures on the train dataset and comparing their performance and accuracy on validation and test datasets.
- Ensembling and combining classification models to predict the chest radiographs overall class and improve the results further.
- Ensembling and combining object detection models to predict the localization of opacities in the form of bounding boxes and improve the results further.
- Evaluate the performance and accuracy of models on the unseen test dataset based upon metrics for classification and object localization.

1.4 Significance of the Study

A successful computer vision model can help radiologists diagnose the millions of COVID-19 patients more confidently and quickly. This can also enable doctors to see the extent of the disease and help them make decisions regarding treatment. Depending upon severity, affected patients may need hospitalization, admission into an intensive care unit, or supportive therapies like mechanical ventilation. As a result of better diagnosis, more patients can quickly receive the best care for their condition, which could mitigate the most severe effects of the virus.

1.5 Scope of the Study

The scope of study mainly involves building an AI model or algorithm that can learn, identify, and localize COVID-19 abnormalities on chest radiographs provided in the form of a training dataset. Models will be evaluated on unseen test dataset. It is at the discretion of hospitals, doctors, and radiologists to leverage the technology and make decisions regarding treatment.

1.6 Structure of the Study

Chapter 1 is the introduction to the problem statement and the corresponding dataset that needs to be tackled in this research. It covers background of the problem along with the importance of the study. Aims and objectives of the research are clearly developed and summarized in this section.

Chapter 2 is the literature review section in depth. It signifies the importance and applications of Neural Networks and Computer Vision Networks in healthcare industry. State of art Image

Classification and Object Detection architectures are reviewed in depth along with their results on standard datasets. This chapter also covers the concepts of data augmentation, ensembling techniques and transfer learning. Publications related to predictive modelling on Chest Imaging and COVID datasets are reviewed in depth along with their limitations.

Chapter 3 covers the aspects of research methodology that is planned for this research. This chapter describes the dataset and highlights the data understanding and preparation steps that need to be undertaken. This section uncovers the planned methodology to train the classification and object detection models along with their evaluation metrics.

Chapter 4 explains about the actual analysis and implementation that has been carried out in this research. It starts with exploratory data analysis followed by data preparation, cleaning, and partitioning steps. Implementation of image classification and object detection models are further explained along with their loss and validation metrics.

Chapter 5 unfolds the results of the final classification and detection models on the test dataset. Post processing steps of ensembling and combining multiple models along with their results are decoded. Error analysis of the trained models is also performed in this chapter.

Chapter 6 discusses and concludes the research that has been undertaken as part of this thesis. Major contributions to new body of knowledge are summarized and important future recommendations are highlighted.

Chapter 2 LITERATURE REVIEW

2.1 Introduction

Deep learning based Neural Networks is one of the hottest topics in the field of Artificial Intelligence. Deep learning is a subset of machine learning where algorithms are created with numerous layers, each providing a different interpretation to the data it feeds on. Inspired by the structure of the human brain, neural networks have established a reputation for successfully learning complex tasks such as object recognition in images, automatic speech recognition (ASR), text translation, sentiment analysis, image captioning, video classification etc.

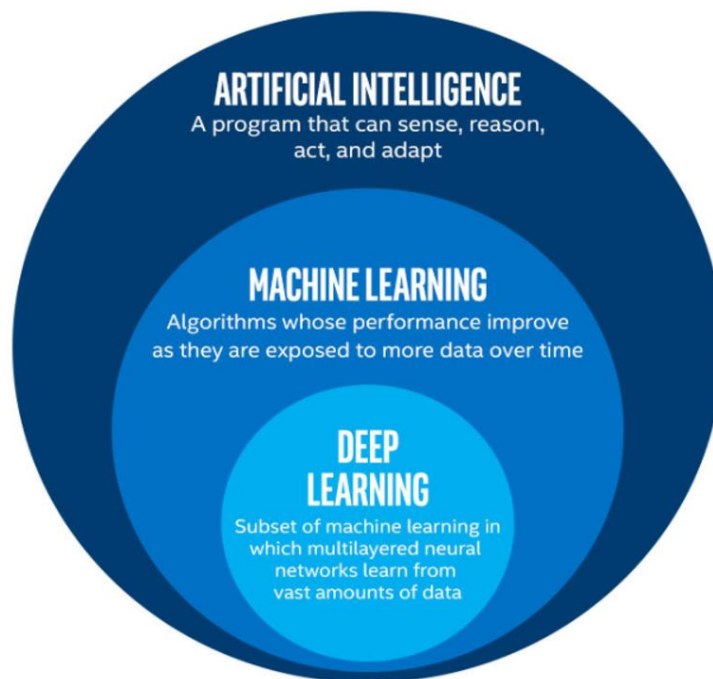


Figure 2.1 AI ML Deep Learning

Figure 2.2 depicts training of a simple feed forward neural network. The annotations indicate how a neural network learns based upon the training inputs which is fed to it. When deployed in real world, the system will just propagate in forward direction and infer or predict on unseen data. Performance of neural networks is judged by how well it generates results on unseen data. A model which performs well on the train data but not on unseen or test data is deemed as overfit and prone to failing for real applications.

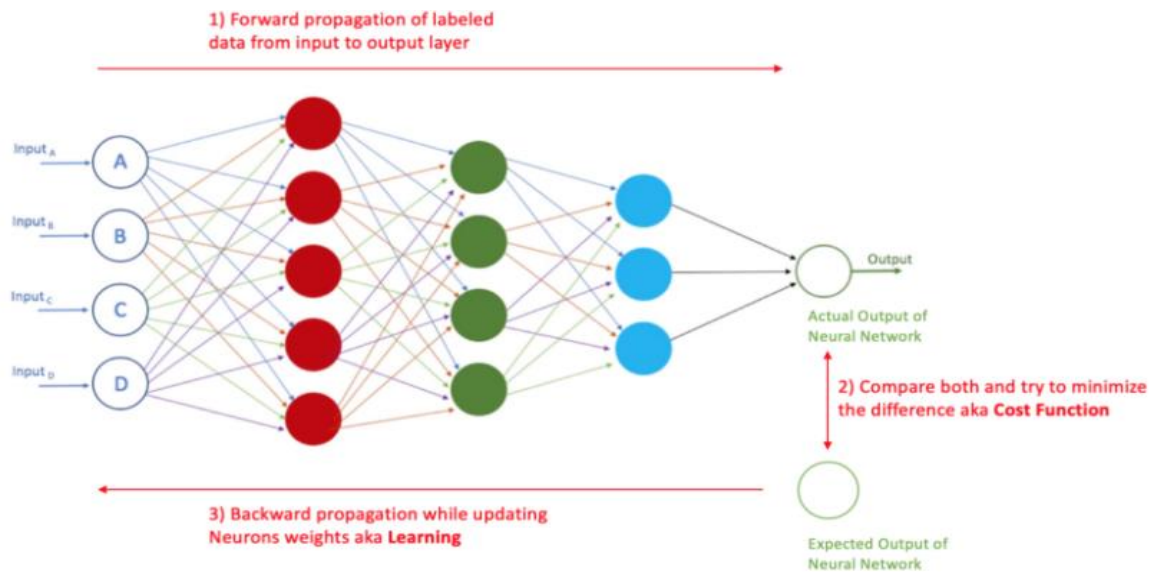


Figure 2.2 Training a Neural Network

Components of a neural network:

- Input and Output layers.
- Hidden layers consisting of multiple neurons and responsible for mapping the learning between input and output layers.
- Network topology with trainable weights and biases. Once trained, the model is frozen for inference.
- Activation functions such as tanh (hyperbolic tangent), Relu (rectified linear unit) etc. Main goal of activation functions is to help learn non-linear representations of real-world data.
- Zooming into the functionality of a single Artificial Neuron calculating a weighted sum of the inputs followed by an activation function to give an output is depicted in Figure 2.3.

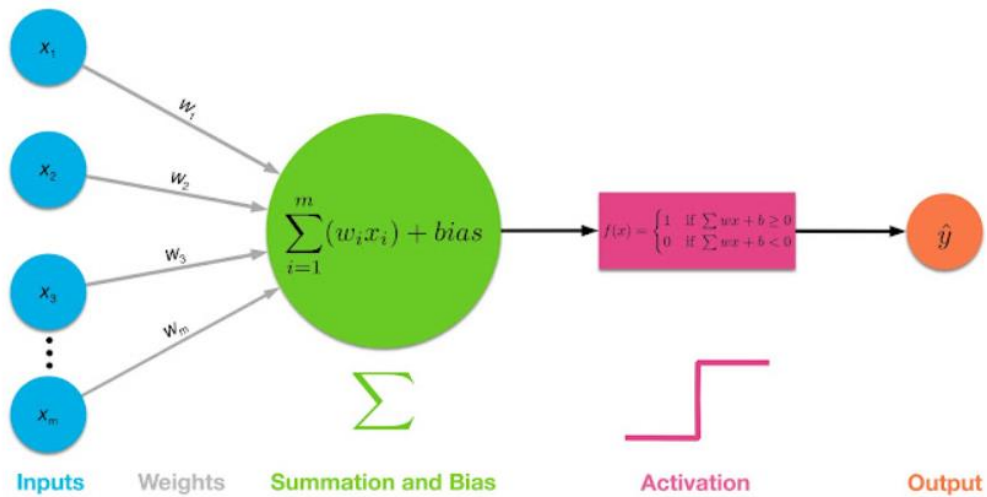


Figure 2.3 Artificial Neuron

Training a neural network is compute intensive and involves seeing the inputs in multiple iterations known as epochs and within each epoch, loss is backpropagated multiple times (usually after a batch of input is fed) from the output layer based upon the framed loss function that the network is trying to minimize. Within all the steps in an epoch, the network tries to learn better and better by adjusting its weights and biases. Finally, after training for multiple epochs, a deep learning model is ready for evaluation on unseen and new data of the problem or task that these AI networks are trying to solve.

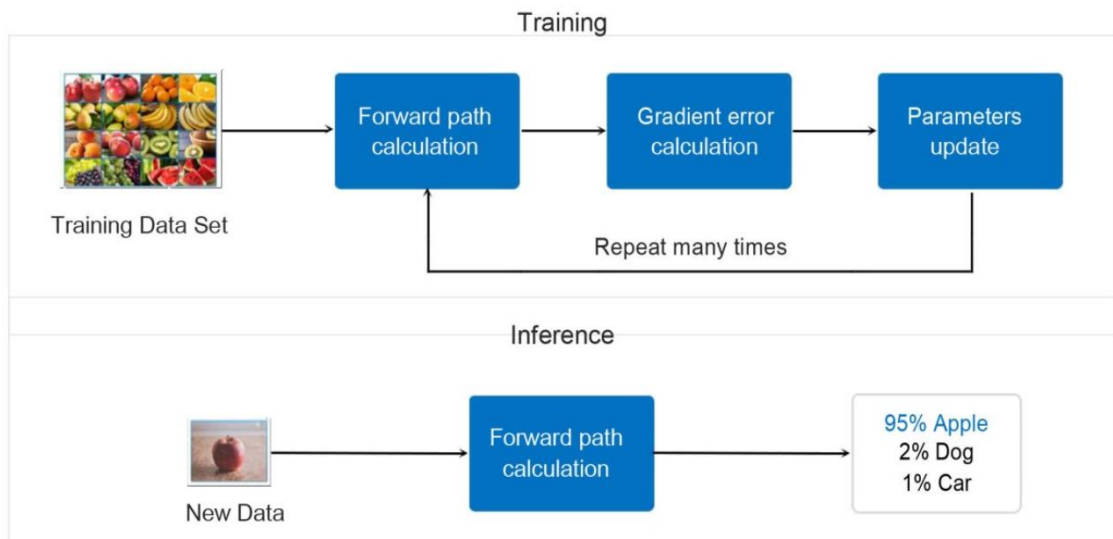


Figure 2.4 Training vs Inference

2.2 Computer Vision Networks

Computer vision models based upon Convolution Neural Networks (CNNs) have been one of the most representative and exciting fields in the field of AI and deep learning. Some of the important computer vision networks are as follows:

- Image classification - Such tasks require an AI model or network to assign a label or a class to an input image.

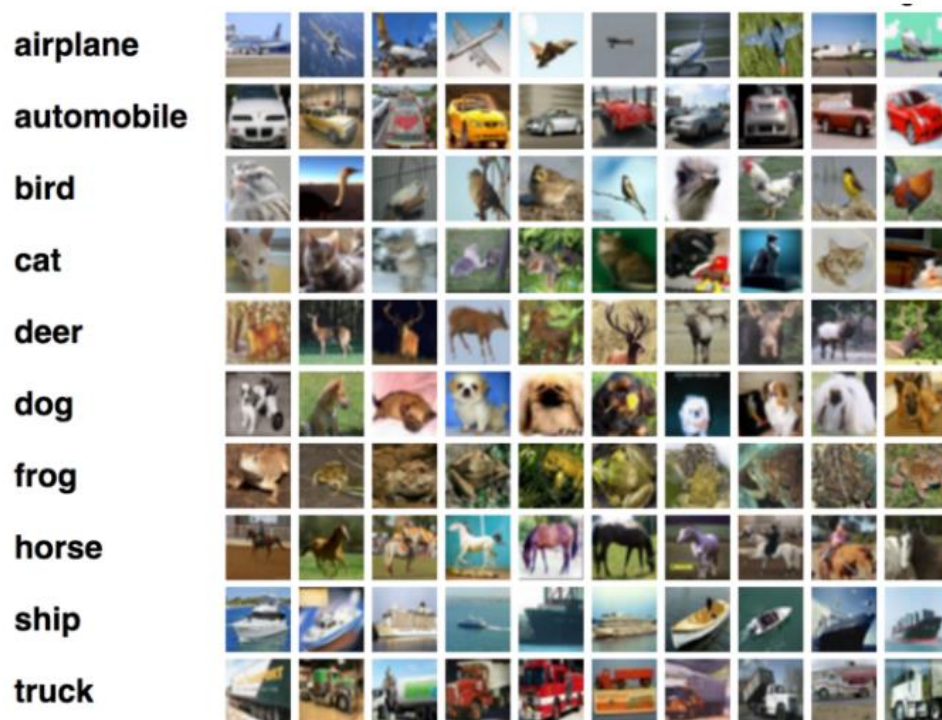


Figure 2.5 Computer Vision - Image Classification

- Image Localization - It helps us to identify the location of a single object in the given image (Figure 2.6).
- Object Detection - In case we have multiple objects present, we then rely on the concept of Object Detection (Figure 2.7). We can predict the location in the form of bounding boxes along with the class for each object using Object Detectors.
- Image Segmentation - Such networks identify which pixels belong to which objects (Figure 2.8).



Figure 2.6 Computer Vision - Image Localization



Figure 2.7 Computer Vision - Object Detection



Figure 2.8 Computer Vision - Image Segmentation

CNNs have been making amazing achievements in computer vision tasks. Some of the many applications of CNNs:

- Facial Recognition such as identifying every entity in a picture.
- Automated inspection of protective equipment such as masks or helmets.
- Defect detection in manufacturing parts.
- Transportation industry such as vehicle classification (Deshpande et al., 2017), automated number plate detection of cars based upon optical character recognition networks.
- Road condition assessments such as potholes, pavements conditions (Camilleri and Gatt, 2020) etc.
- Agriculture industry such as determining the wheat heading period (Sadeghi-Tehran et al., 2017), monitoring the crop growth, animal monitoring etc.
- Detection of medical problems such as brain cancer detection, identifying anomalies in X-ray and MRI scans.

AlexNet (Krizhevsky et al., 2012), a CNN based algorithm won the 2012 ImageNet (Jia Deng et al., 2009) competition which uses images dataset for benchmarking computer vision models. Since then to date, researchers have been experimenting and coming up with better and better architectures to improve model performance, accuracy, and results on various computer vision tasks.

2.2.1 Understanding CNNs

Convolutional Neural Networks or CNNs are specialized deep learning architectures which work particularly well with visual data, that are images and videos. The innovation of convolutional neural networks is the ability to automatically learn many filters (weights of a convolution layer) in parallel specific to a training dataset under the constraints of a predictive modelling problem, such as image classification. The result is highly specific features that can be detected anywhere on input images.

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Multiple such filters are applied to an input that results in a map of activations called feature maps, indicating the locations and strength of a detected feature in an input, such as an image. Mathematically, CNN - Convolution Operation is the summation of the elementwise product of two matrices. Let's take two matrices, X and Y. If you 'convolve the image X using the filter Y', this operation will produce the matrix Z as shown in Figure 2.9

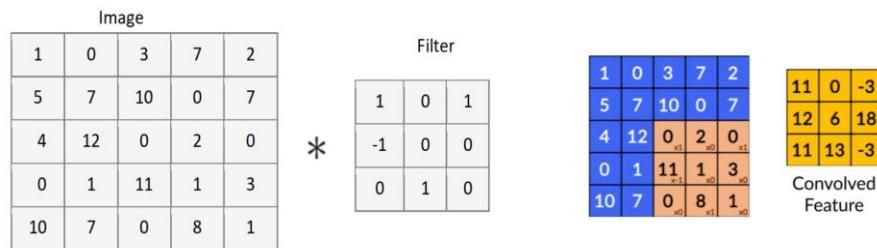


Figure 2.9 CNN - Convolution Operation

Padding operation can be applied to convolution layers to maintain the height and width of the output (same as input) that we get after the convolution and not lose border information. Strides can be configured for convolution operation to control the density of convolving. If we want to make a very deep convolutional neural network with lots of convolutional layers, the spatial dimension will reduce after each convolutional layer, and we may not be able to make the layers deep enough. But we also want a compact representation of the image for classification. To overcome this problem, schema is designed in which the spatial dimension of the input can be preserved with each convolutional operation through the choice of proper stride and padding while reducing the representation periodically through pooling as depicted in Figure 2.10. Convolution operations are usually followed by pooling operations such as max pooling or average pooling. Pooling also helps in controlling overfitting on the training data set.

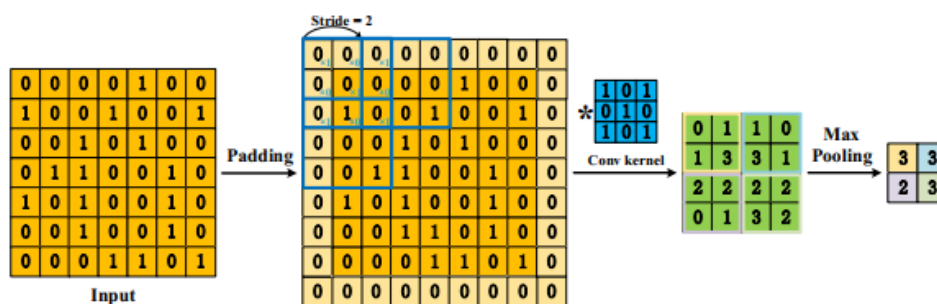


Figure 2.10 CNN - Convolution (Stride, Padding) + Maxpool (Li et al., n.d.)

2.2.2 Simple CNN architecture for digit recognition

A simple CNN based digit recognition architecture that can classify digits and recognize a number in an image between 0 and 9 is depicted in Figure 2.11.

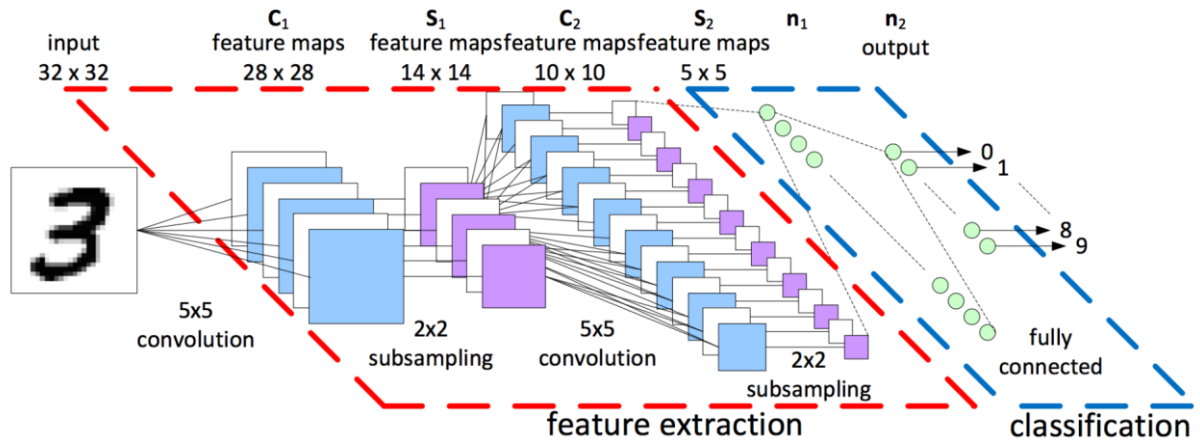


Figure 2.11 CNN - Digit Recognition

Explanation of the architecture:

- Input layer - 32 x 32 pixels of an image with a digit between 0 and 9.
- Convolutional Layer 1 - Applies C₁ 5x5 convolution filters to the input layer pixels and outputs C₁ feature maps of size 28 x 28.
- Max pooling Layer 1 - Selects the maximum element by sliding through the Convolution layer 1 feature maps and reduces the feature maps dimension to 14 x 14.
- Convolutional Layer 2 - Applies C₂ 5x5 convolution filters to the max pooling output and outputs C₂ feature maps of size 10 x 10.
- Max pooling Layer 2 - Selects the maximum element by sliding through the Convolution layer 2 feature maps and reduces the feature maps dimension to 5 x 5.
- Dense layers - Final dense layer consisting of 10 neurons with Softmax activation to predict the probabilities for digits 0 to 9.

2.3 Review of Image Classification Models

Goal of image classification models is to predict the overall main class or category of an image. It generally falls under the category of supervised learning.

2.3.1 ImageNet Dataset and Evaluation

ImageNet (Jia Deng et al., 2009) dataset is a standard image database split into train, validation, and tests data sets across 1000 categories. Train data set consists of 1.2 million images whereas validation and test datasets consist of 50K and 100K images respectively. ImageNet Dataset is primarily used to benchmark image classification networks by the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) organization. Classification task requires the model to assign a class label based upon the main object in the image. Models are trained on labelled train data set and fine-tuned using labelled validation data set and are finally evaluated for their overall top-1 and top-5 accuracy results on the unseen private test data set across the 1000 categories. A classification model is trained to predict a probability value (between 0 and 1) for all the 1000 classes. Top-1 prediction basically means that the golden truth label is predicted by the model with highest probability among the 1000 classes whereas a top-5 prediction indicates that the golden truth label is among the top-5 probable predictions by the model. Test dataset accuracy results are published upon submission to ImageNet organization. Developers of state of art CNN models publish their achieved top-1 and top-5 accuracy results on this dataset and compare their results against other state of the art models.

2.3.2 ResNet Architectures

Can we grow the network deeper and deeper to learn better models? An obstacle to answering this question is the problem of vanishing/exploding gradients (Zagoruyko and Komodakis, 2016), which hamper convergence from the beginning. Backpropagation and gradient descent algorithms form the heart of training a neural network. As the backpropagation algorithm advances in backward direction from the output layer towards the input layer, the gradients often get smaller and approach zero which eventually leaves the weights of the initial or lower layers nearly unchanged, known as vanishing gradients problem. On the contrary, in some cases, the gradients keep on getting larger and larger as the backpropagation algorithm progresses, known as the exploding gradients problem. Skip connection mechanism is the key feature of the ResNet architectures (He et al., 2016) which enabled the training of very deep networks and helped solve the problem of vanishing/exploding gradients. We can propagate

larger gradients to initial layers via skip connections and these layers also could learn as fast as the final layers, giving us the ability to train deeper networks.

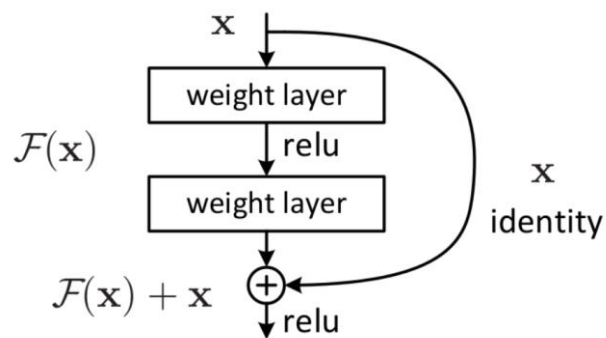


Figure 2.12 ResNet - Skip Connection

Key features of ResNet:

- Batch normalization after every convolution layer. Batch normalization standardizes the output after every layer and helps in stabilizing the model learning process.
- Figure 2.13 illustrates connections of residual blocks with 2 layers and 3 layers in the network.

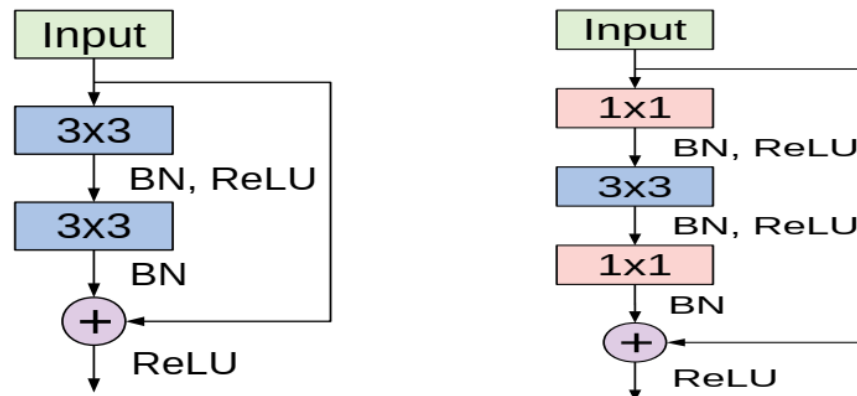


Figure 2.13 ResNet - Residual Blocks

- No fully connected layer except one last fully connected Softmax layer to get the probabilities of each class for classification.
- ResNet50 is a variant of ResNet and is widely used. Full architecture of ResNet50 is depicted in Figure 2.14.

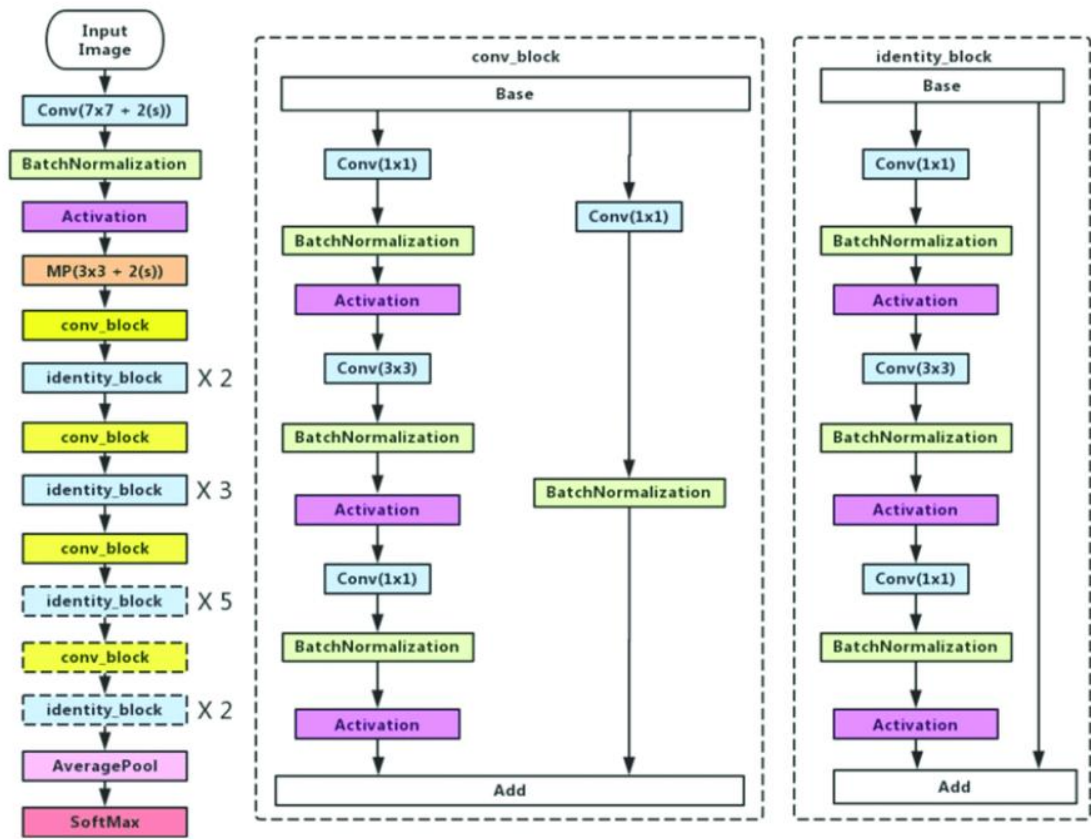


Figure 2.14 ResNet50 - Full Architecture

2.3.3 DenseNet Architectures

Problem of vanishing/exploding gradients is solved in a slightly different manner by concatenating the feature maps from preceding layers (Huang et al., 2017) rather than through summation as done in ResNet.

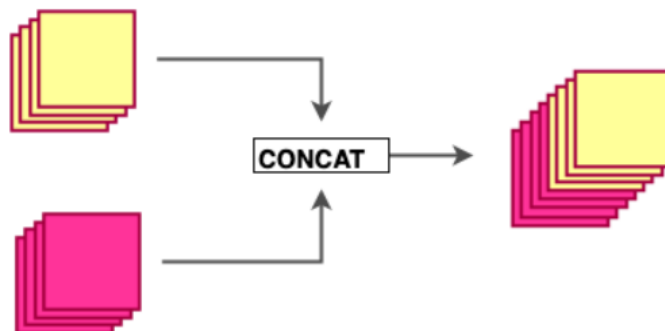


Figure 2.15 DenseNet - Concatenation of feature maps

Key features of DenseNet:

- Dense Blocks comprising of dense layers denoted by DL1 to DL6 as shown in Figure 2.16. Dense layers consist of 2 convolutional layers that are 1 X 1 CONV (conventional operation for extracting features) and 3 X 3 CONV (bringing down the feature depth/channel count).

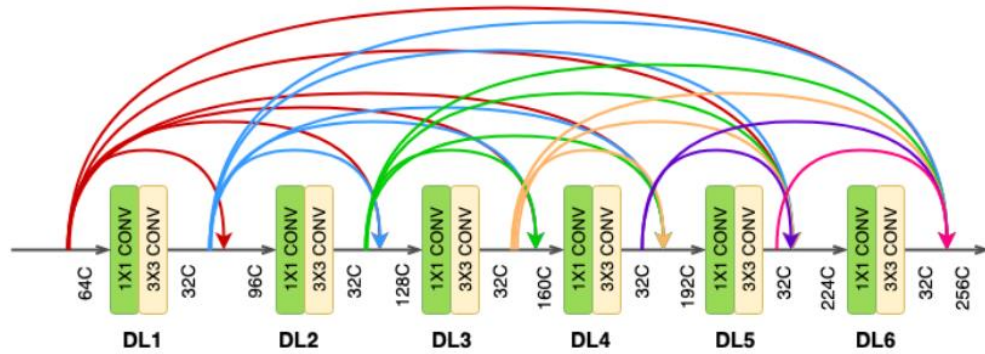


Figure 2.16 DenseNet - Dense Block with 6 dense layers

- Full architecture of DensNet is depicted in Figure 2.17.

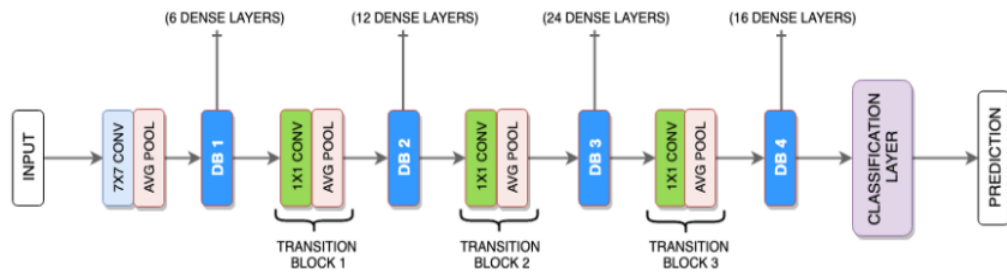


Figure 2.17 DenseNet - Full Architecture

- Reduced parameter count.
- Less redundant feature maps.
- Comparison of DenseNet with ResNet (Figure 2.18) reveals that DenseNet models with much lesser number of parameters than ResNet have same validation errors. DenseNets may perform better with better hyper parameter search.

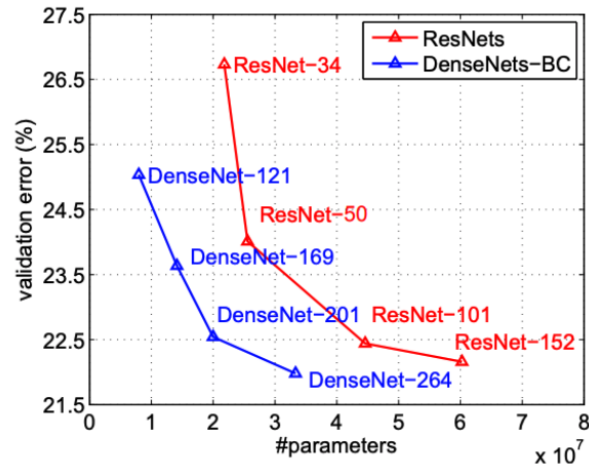


Figure 2.18 DenseNet - Comparison with Resnet

2.3.4 EfficientNet Architectures

Google Research team came up with a baseline architecture using network architecture search (NAS) and systematically scaled it up in depth, width, and resolution (compound scaling) to come up with a new family of architectures known as EfficientNets which have super passed state of art accuracy results (Tan and Le, 2019, 2021) and improved both training time and parameters efficiency. It was observed that scaling networks with larger width, depth or resolution tend to achieve higher accuracy, but accuracy gain saturates at a point, hence the approach of compound scaling (Figure 2.19).

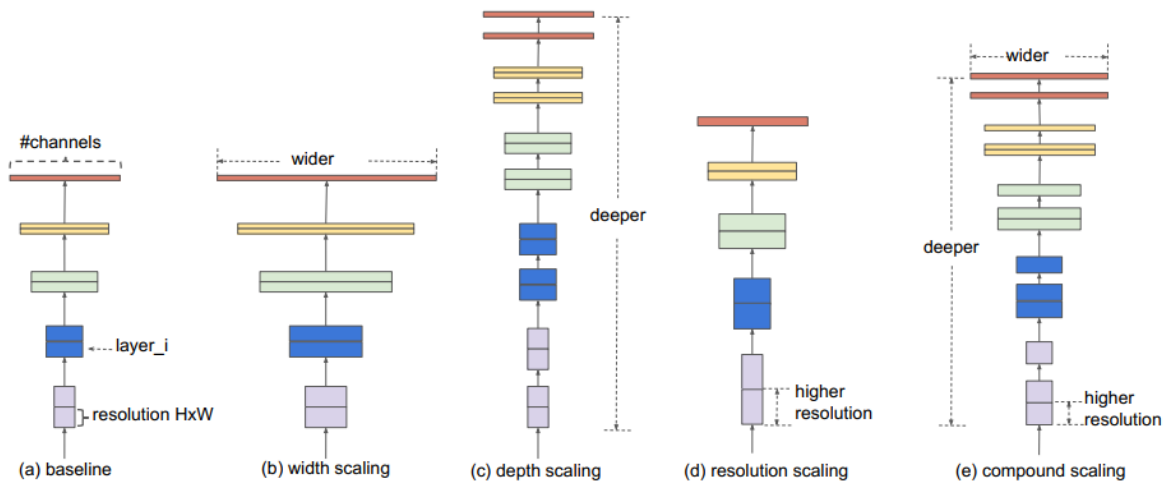


Figure 2.19 EfficientNet - Compound Scaling (Tan and Le, 2019)

Key features of EfficientNet:

- MBConv blocks – MobileNet Conv blocks are the main building blocks of EfficientNet architecture and are basically inverted residual blocks. In original ResNet blocks, pointwise convolution operation first squeezes the channels and then extends the channels whereas in MB Conv blocks, channels are first extended and then squeezed later. Normal convolution is also replaced by Depthwise convolution which makes the architecture further efficient with minimal loss in accuracy. With same number of parameters, depth wise convolution (left in Figure 2.20) gives three output channels with only one 3-channel filter while three 3-channel filters will be required normal convolution (right in Figure 2.20) to achieve the same.

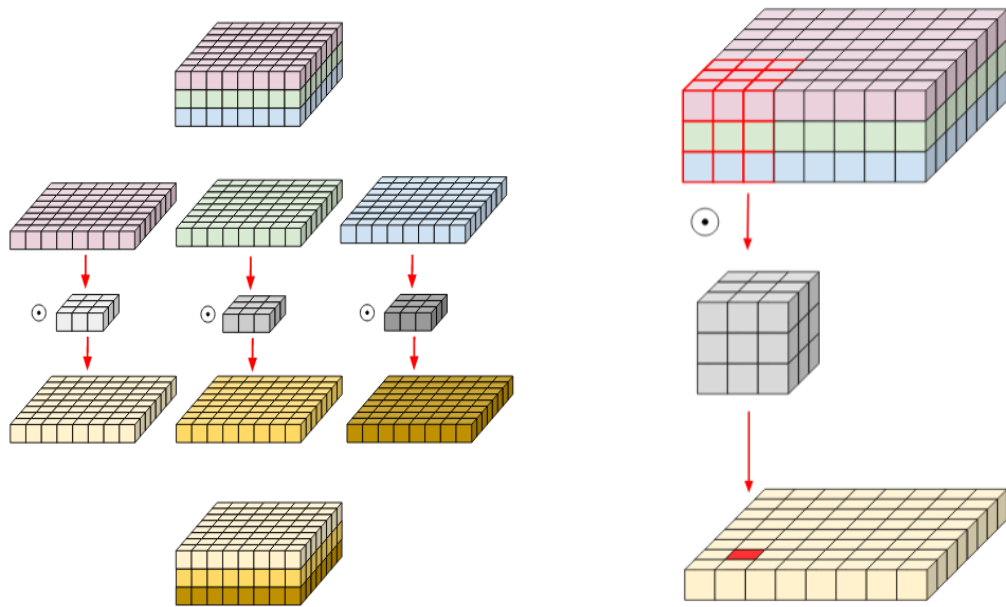


Figure 2.20 EfficientNet - Depthwise Convolution vs Normal Convolution

- EfficientNet architecture (Figure 2.21) consists of multiple MBConv blocks.

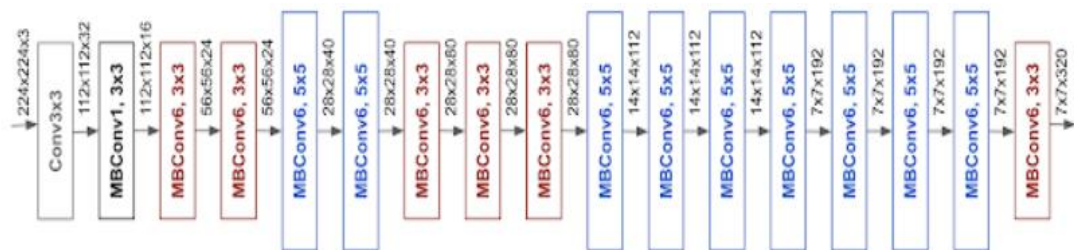


Figure 2.21 EfficientNet-B0 Full Architecture

- Faster training time.
- EfficientNetv2 training on ImageNet dataset used different image sizes between epochs and adaptive regularization based upon image sizes.

Comparison of EfficientNets with existing ConvNets:

- EfficientNet models have both higher accuracy and higher efficiency over existing CNN models as depicted in Figure 2.22.
- EfficientNet-B4 achieves 6% higher top-1 accuracy (82.6%) than Resnet50 (76.3%) and using same number of FLOPs.
- EfficientNet-B7 achieves state of the art 84.4% top-1 / 97.1% top-5 accuracy on ImageNet dataset.

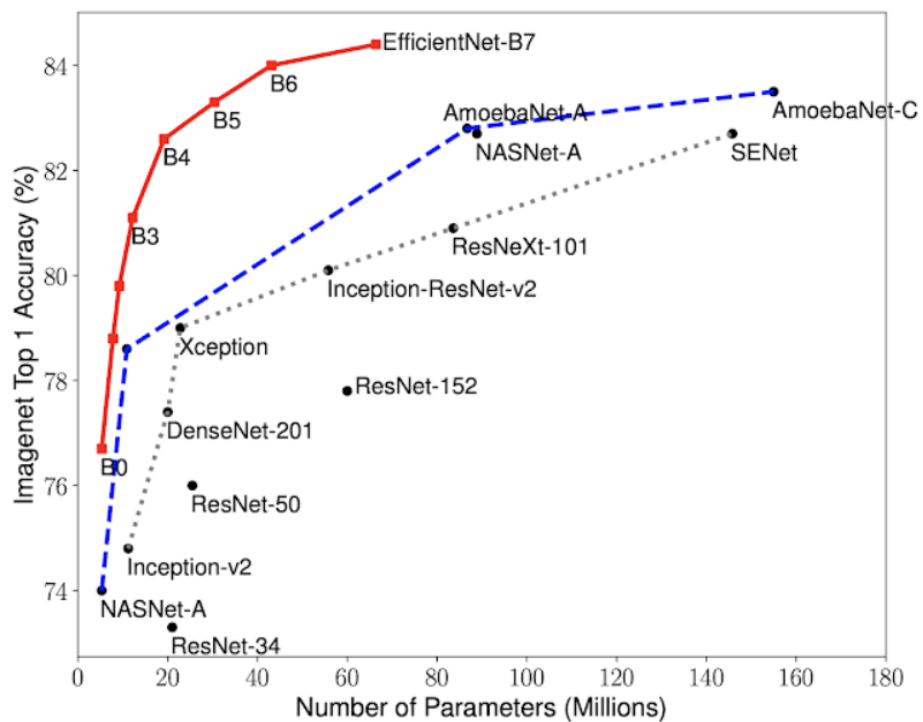


Figure 2.22 EfficientNet - Model size vs Accuracy Comparison (Tan and Le, 2019)

- Table 2.1 highlights the accuracy, parameters, and performance of the family of EfficientNet architectures (Tan and Le, 2021). FLOPs indicate the computational capacity of a model by measuring the number of operations of a frozen model.

Table 2.1 EfficientNet - Comparison of different variants

Model	Top-1 Accuracy Results on ImageNet	Model parameters (in millions)	FLOPs (in billions)
EfficientNet-B3	81.5%	12M	1.9B
EfficientNet-B4	82.9%	19M	4.2B
EfficientNet-B5	83.7%	30M	10B
EfficientNet-B6	84.3%	43M	19B
EfficientNet-B7	84.7%	66M	38B
EfficientNet-X	84.7%	73M	91B
EfficientNetv2-S	83.9%	22M	8.8B
EfficientNetv2-M	85.1%	54M	24B
EfficientNetv2-L	85.7%	120M	53B

2.4 Review of Object Detection Models

Goal of object detection models is to predict and identify all the object classes inside an image and draw a bounding box around it. It generally falls under the category of supervised learning.

2.4.1 COCO Dataset and Evaluation

COCO (Lin et al., 2014) dataset is a large images dataset published by Microsoft for object detection and segmentation tasks. It is used by researchers to benchmark their algorithms for object detection and image segmentation tasks. Dataset consists of 328K images and object detection and segmentation task requires predictions across 80 object categories.

Evaluation of object detection models is done by calculating the mAP (mean average precision) across all object classes and predictions.

- IoU – Intersection over Union (Figure 2.23) is used when calculating mAP. Value of IoU is between 0 and 1 and indicates the percentage of overlap between predicted and true bounding boxes.
- Based upon IoU thresholds (say 0.5) and object labels, predictions are categorized into true positives, false positives, or false negatives.

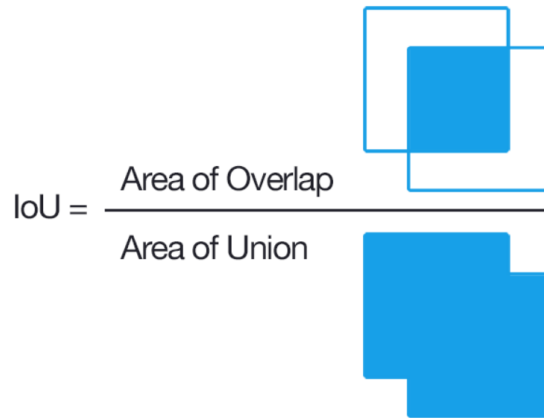


Figure 2.23 Intersection Over Union

- A precision recall curve is generated for each object class and an average precision of that object class is calculated.

$$\begin{aligned} \text{Precision} &= \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \\ &= \frac{\text{True Positive}}{\text{All Observations}} \end{aligned}$$

$$\begin{aligned} \text{Recall} &= \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \\ &= \frac{\text{True Positive}}{\text{All Ground Truth}} \end{aligned}$$

- Mean of average precision across all object classes gives the final mAP score which researchers publish for their state of art object detection algorithms.

2.4.2 YOLO

Prior to YOLO (You Only Look Once) network, most object detection models such as Region Convolution Neural Network (R-CNN) operated in two-stages that is first produce potential bounding boxes on the input image first and second run a classifier model on the bounding boxes to predict the object classes. Such models were difficult to optimize and compute intensive. YOLO (Redmon et al., 2016) algorithm instead reframed the object detection as a single regression problem and came up with an unified architecture to simultaneously predict multiple bounding boxes and object classes in single shot. YOLO system computes all the features of the image and makes predictions at the same time instead of iterating over the

process of classifying different regions in an image. In terms of speed as well as accuracy, YOLO family of models perform very well.

YOLO divides an input into S by S (7 by 7 is default) grid cells as depicted in Figure 2.24. A grid cell is responsible for detecting an object if the center of an object is part of that cell. Each grid cell predicts B bounding boxes along with its parameters which are center coordinates and width, height represented in the form of (x, y, h, w) . A confidence score is also calculated for each bounding box defined as:

Confidence score = $p(\text{Object}) * \text{IOU of true boxes vs predicted boxes}$ where

$p(\text{Object})$ = probability that an object exists inside a grid cell

IOU = Intersection over union of true boxes and predicted boxes.

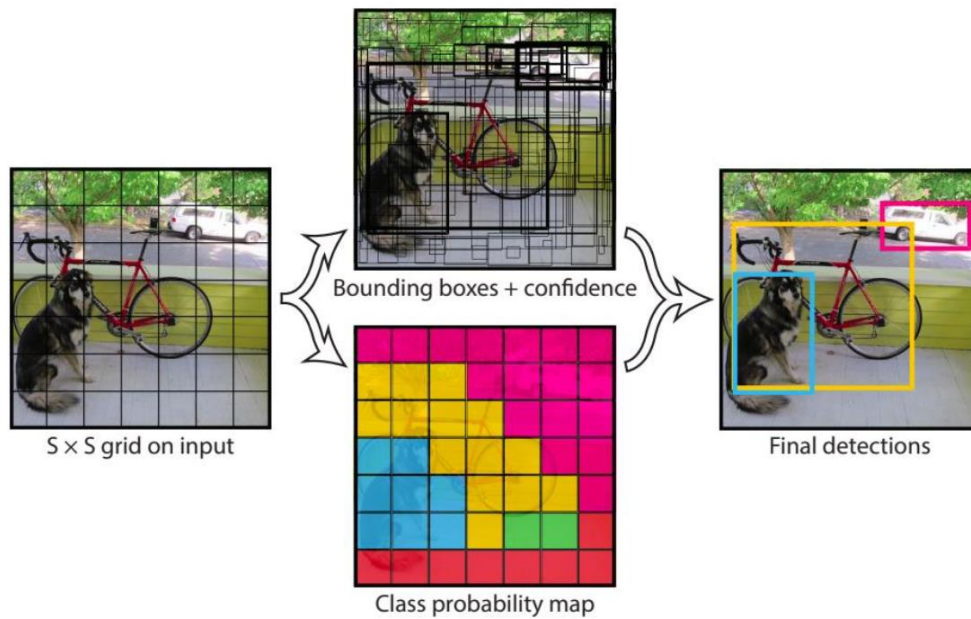


Figure 2.24 YOLO - 7x7 grid cells

Classification probabilities of C different object classes shared between all the bounding boxes predictions is also produced for each grid cell. Figure 2.25 shows 2 (B) bounding boxes predictions for the center cell along with 3 (C) classes probability predictions as an example. Finally, YOLO applies non max suppression algorithm in post processing step to remove overlapping bounding boxes predictions which have intersection over union value greater than a threshold value such as 0.5. It helps in selecting the best bounding boxes.

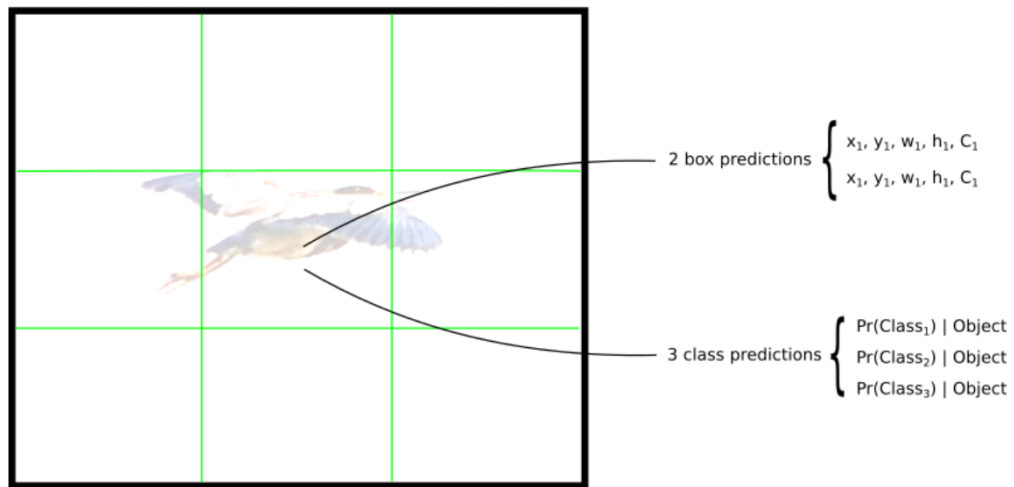


Figure 2.25 YOLO - bounding boxes of a grid cell share class predictions

YOLOv3 (Redmon and Farhadi, 2018) architecture with a notable feature of 3 scale detectors used at different stages is shown in Figure 2.26.

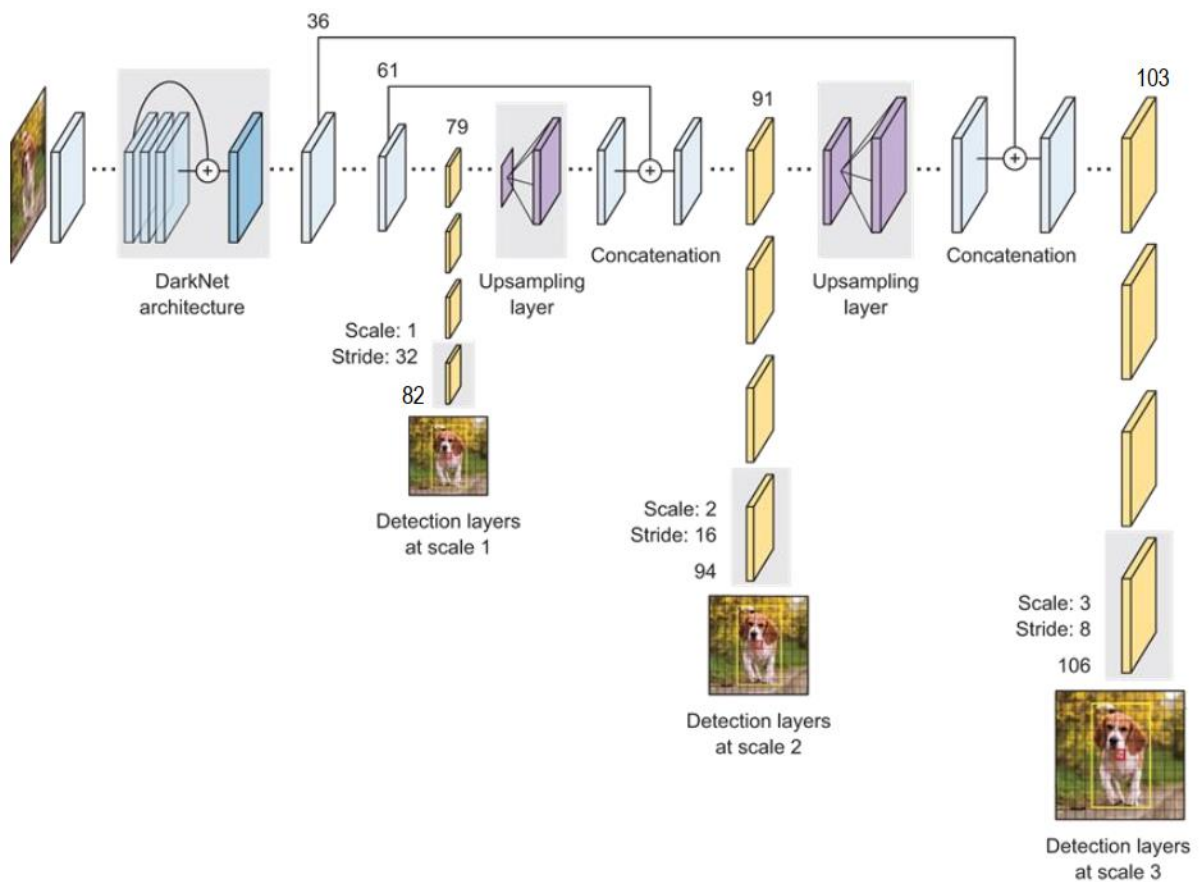


Figure 2.26 YOLOv3 - Architecture

Comparison of latest YOLOv5 architecture variants with EfficientDets claiming higher accuracy and faster inference speed is shown in Figure 2.27.

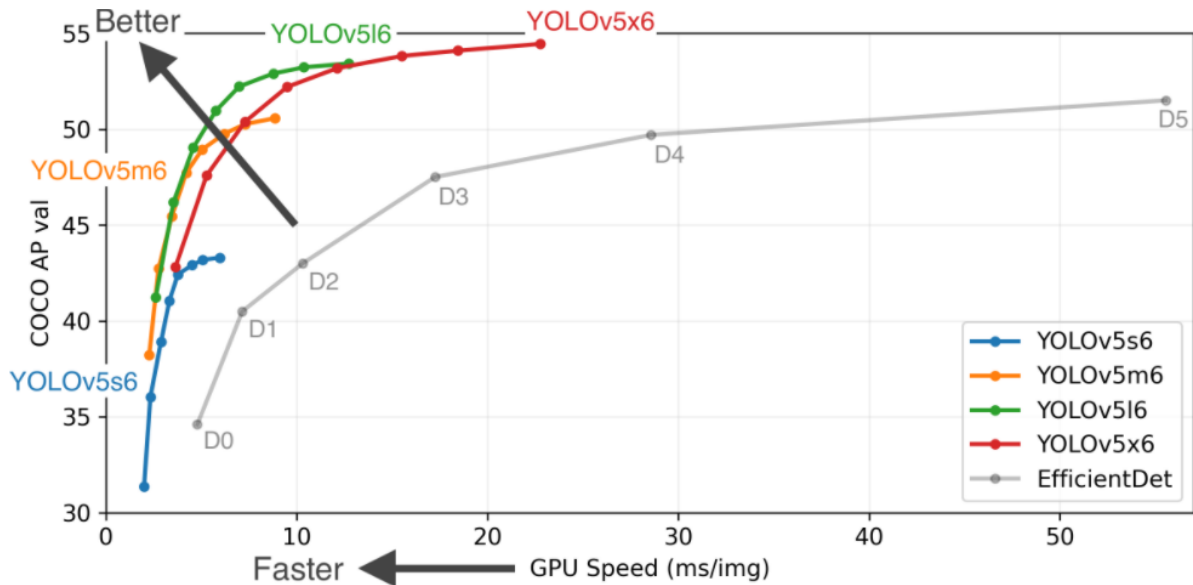


Figure 2.27 YOLOv5 comparison with EfficientDet

2.4.3 EfficientDet

Higher computation costs and larger model sizes deter the deployment of object detection models in real world. To tackle the problem of achieving higher accuracy as well as better efficiency, Google Research team came up with a new family of object detectors named EfficientDets (Tan et al., 2020) that are quite efficient with smaller number of parameters (model size) and achieved state of art accuracy results on the standard COCO dataset.

EfficientDets consists of 3 sub networks. Compound scaling approach was applied to scale resolution, depth, and width for all the three subnetworks at the same time.

- Backbone Network is the EfficientNet ConvNet which learns features from an image and feeds those features to the feature sub network.
- Feature Network is known as Bidirectional feature pyramid network (BiFPN). This network works in bottom up as well as top-down approach hence bidirectional and performs weighted fusion of the features extracted from last few layers of the backbone network. Weights attached to the features are learnable parameters during training. BiFPN layers are further stacked with skip connections and the number of BiFPN blocks differ in the various versions of EfficientDet.

- Prediction Networks – Final networks which are responsible for bounding box and object class predictions and backpropagate the prediction loss to the BiFPN and backbone networks during training. Weights are shared between bounding boxes and object class prediction networks and hence contribute to reduced parameters for the overall network.

EfficientDets being single-shot object detectors train faster. They also transfer well to other domains of computer vision. Figure 2.28 depicts all the 3 sub networks of EfficientDet.

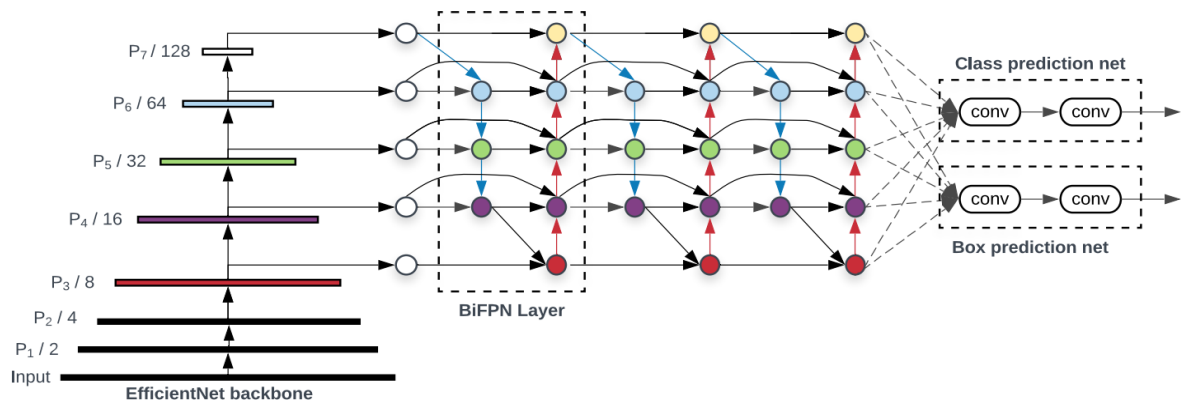


Figure 2.28 EfficientDet - Architecture

Comparison of EfficientDet with existing Object Detectors (Figure 2.29):

- EfficientDet-D7 achieved state of art 52.2 AP (average precision) on COCO dataset with 325 billion FLOPs and 52 million parameters only.

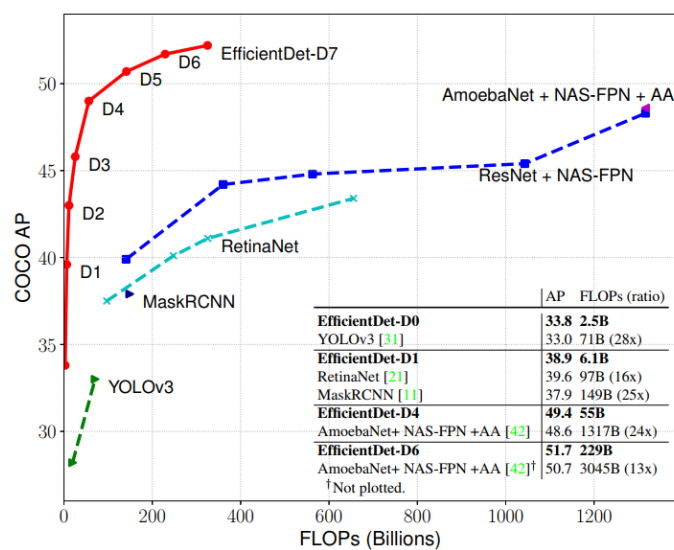


Figure 2.29 EfficientDet - Comparison with other Object Detectors

- EfficientDet-D0 with 28X lesser FLOPs achieves comparable and in fact slightly better results than YOLOv3 model.
- EfficientDet-D1 with 16X to 25X lesser FLOPs have comparable results to RetinaNet and MaskRCNN object detection models.

2.4.4 Survey on Object Detection Models

A recent survey (Zaidi et al., 2021) on various object detection models published their performance results on the MS COCO dataset as shown in Figure 2.30.

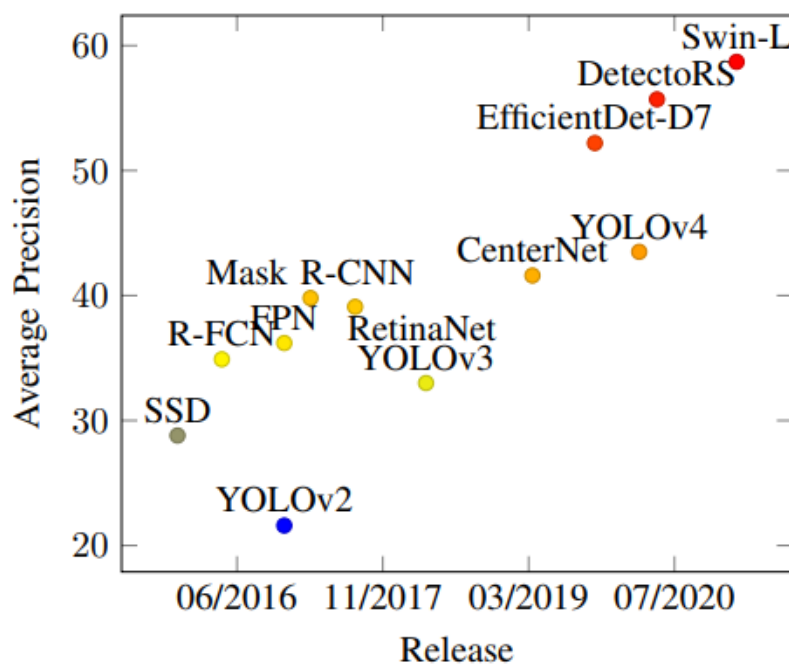


Figure 2.30 Survey of Object Detection Models on COCO Dataset (Zaidi et al., 2021)

YOLOs and EfficientDets are preferred architectures for object detection considering their speed as well as accuracy. DetectoRS (Qiao et al., n.d.) is a 2-stage detector and not suitable for real time applications. Swin-L (Liu et al., 2021) is based upon transformer architecture which have had a big impact in natural language processing tasks. Though it achieved the highest accuracy on COCO dataset, it requires higher number of parameters than CNN based object detectors and evaluation of transformer-based object detection models on other computer vision tasks and datasets is still at an early stage.

2.5 Data Augmentation Techniques

Data augmentation is a technique to apply random but realistic transformations such as image rotations to increase the diversity of the training dataset. Such a technique can further help in generalizing and expanding the training dataset and thus the model can perform better on unseen real data. Some of the common augmentation techniques are as follows:

- Random flip - left to right, up to down

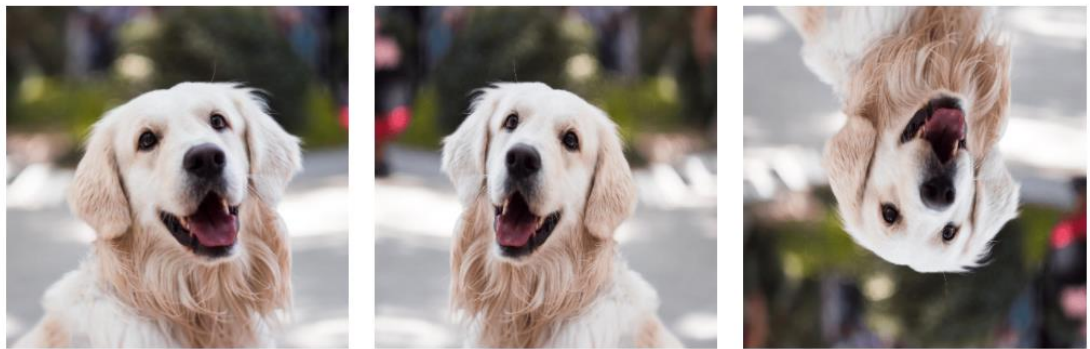


Figure 2.31 Data Augmentation - Random Flip Left Right and Up Down

- Random brightness - New image will be lighter or darker.

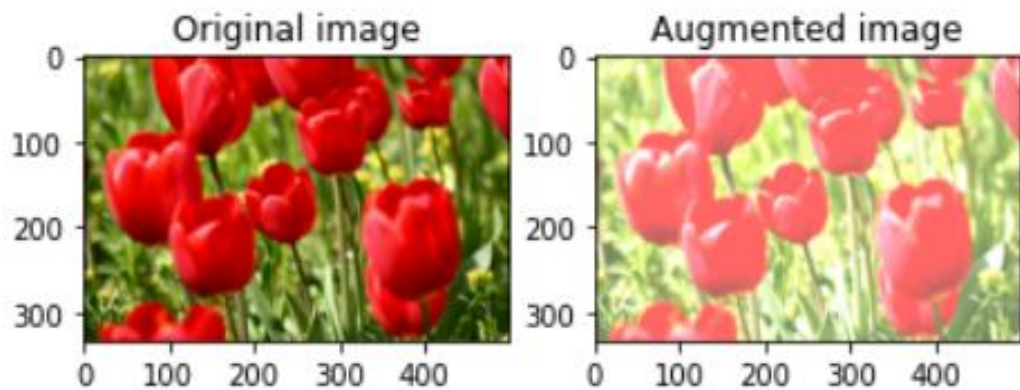


Figure 2.32 Data Augmentation - Random Brightness

- Random crop - Section of an image is cropped and then resized back to original size.
- Random scaling - Image is zoomed inwards or outwards.

- Random contrast - Contrasts of an image are changed.

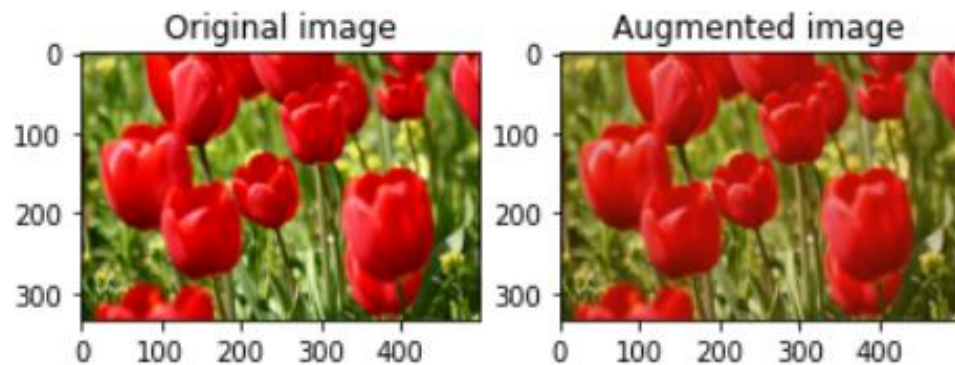


Figure 2.33 Data Augmentation - Random Contrast

- Random saturation - Intensity of a color in an image is saturated by a factor.

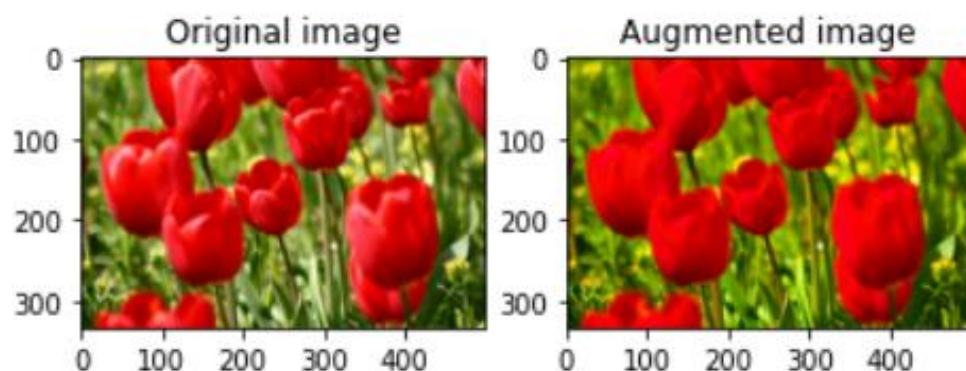


Figure 2.34 Data Augmentation - Random Saturation

Some of the studies related to data augmentation techniques for images are as follows:

- Complex image filtering techniques such as Median and Gaussian filters were applied to improve the quality of MRI lung images and remove unwanted noise (Perumal and Velmurugan, 2018).
- Various data augmentation techniques (Safdar et al., 2020) were applied while training YOLOv3 model and helped achieving higher accuracy on both training as well as test datasets. Image rotations of 180 and 90 degrees and crop and scale augmentation techniques performed significantly better than other techniques.

2.6 Transfer Learning

Main intuition of Transfer Learning (Figure 2.35) is that a model trained on a large dataset can serve as an effective model for learning other visual tasks.

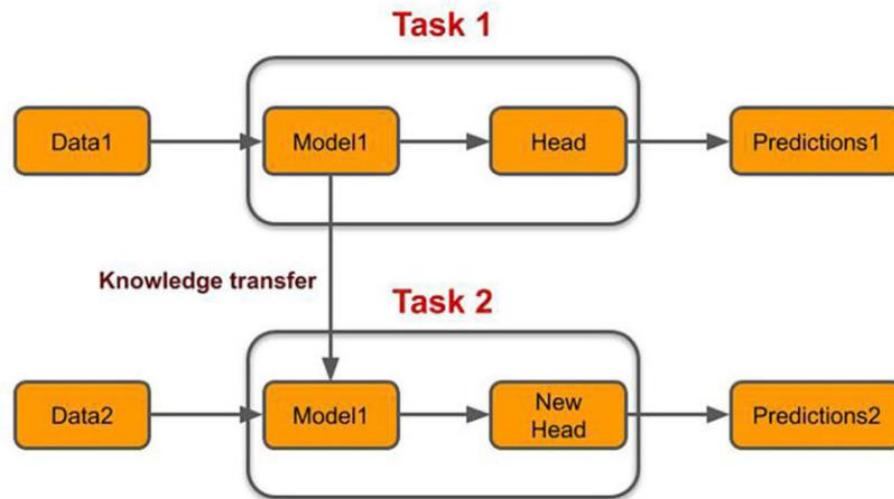


Figure 2.35 Transfer Learning

Transfer learning can have following techniques:

- ‘Freeze’ the initial layers that is use the same weights and biases that the network has learnt from some other tasks, remove the few last layers of the pre-trained model, add your own new layer(s) at the end and train only the newly added layer(s).
- Fine-tune the pre-trained model. Retrain all or few layers starting (initializing) from the weights and biases that the network has already learnt. Retrain using backpropagation. Since you don't want to unlearn a large fraction of what the pre-trained layers have learnt. So, for fine-tuning, we usually choose a low learning rate.

Recent studies have showcased that models trained on ImageNet dataset transfer well on other computer vision tasks. ImageNet architectures converge better and faster and are capable of learning better representations as indicated in Figure 2.36 for some of the standard datasets. It is important to note that more emphasis is laid on the architectures trained on ImageNet and corresponding model weights serve as a good starting point for transfer learning. These models generalize nicely across various vision datasets but the ImageNet features maybe quite sensitive in some cases (Kornblith et al., n.d.).

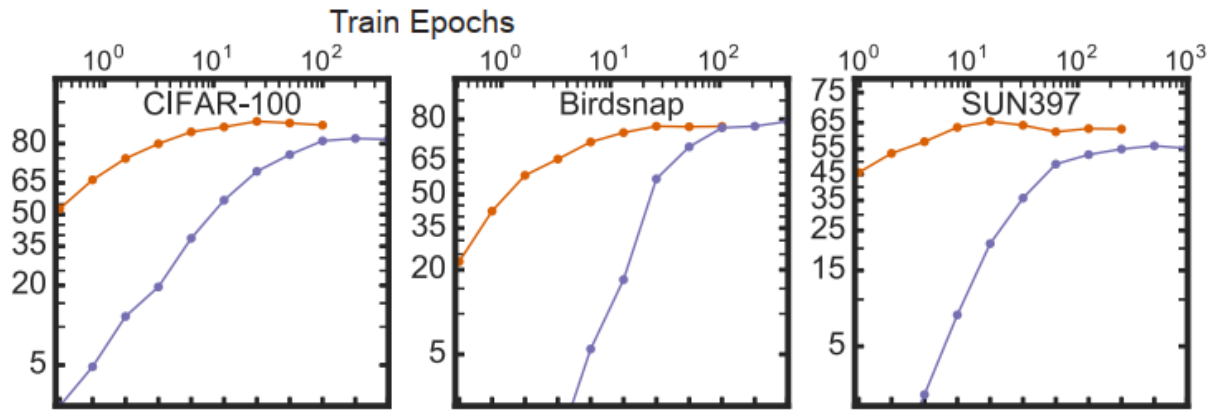


Figure 2.36 Transfer Learning using ImageNet weights vs Training from scratch with random initialized weights (Kornblith et al., n.d.)

Table 2.2 reflects the transfer learning results of EfficientNet (Tan and Le, 2019, 2021) architectures which are pre-trained using ImageNet and fine-tuned on few other standard datasets. Accuracy results are averaged over 5 runs.

Table 2.2 Transfer Learning Top-1 Accuracy Results via EfficientNet

Model	Params	ImageNet Accuracy	CIFAR-10	CIFAR-100	Flowers	Cars
EfficientNet B7	66M	84.7	98.9	91.7	98.8	94.7
EfficientNet v2-S	24M	83.2	98.7	91.5	97.9	93.8
EfficientNet v2-M	55M	85.1	99.0	92.2	98.5	94.6
EfficientNet v2-L	121M	85.7	99.1	92.3	98.8	95.1

2.7 Ensembling Techniques

Ensemble learning (Figure 2.37) combines the predictions of multiple trained models and helps in reducing the variance of neural network models. Combining predictions from multiple models generally produces more stable and better results (Huang et al., 2016; Hirata and Takahashi, 2020) and is one of the steps towards explainable AI results. On other hand, ensembling increases the overall training time and hence require more resources to train

multiple models. It also increases the inference time when the models are deployed commercially and may not be suitable for real time inferencing applications.

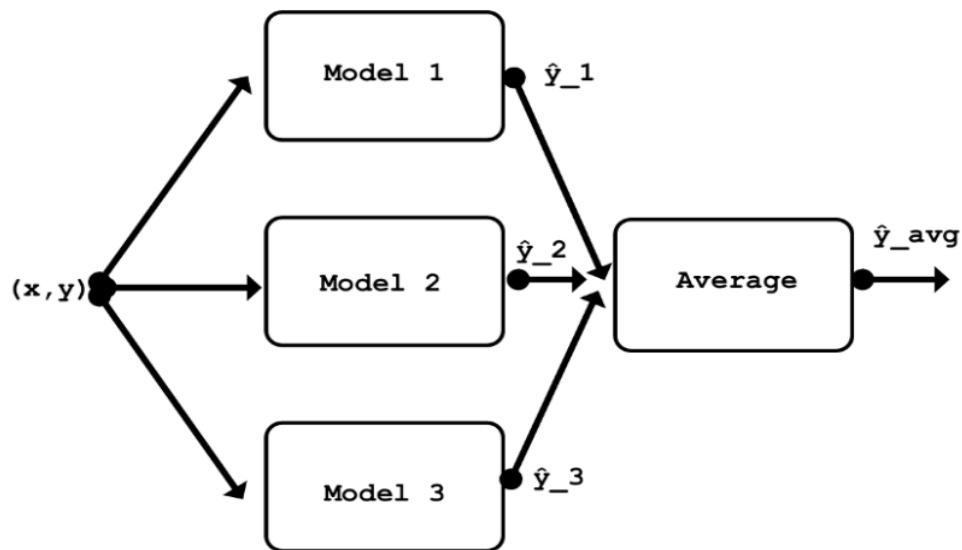


Figure 2.37 Ensembling multiple models

2.7.1 k-fold Cross Validation Ensemble

Training dataset is split into k folds (usually 5) and k models are trained where in each fold data is used as validation dataset and model is trained on the remaining folds as depicted in Figure 2.38. Training results on the out of fold data are compared against the held-out fold data to track model accuracy and convergence. When training on imbalanced data set of c classes, stratified approach is applied to k folds which ensures that each of the folds preserve the proportion of data samples across the c classes. Out of fold accuracy results can be averaged out to evaluate the overall performance of a model architecture which has been trained k times.

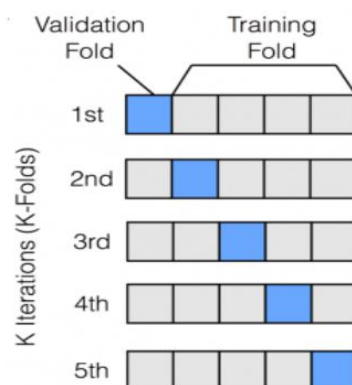


Figure 2.38 Ensembling - k fold

2.7.2 Model Averaging Ensemble

Multiple m models are part of the ensemble for final predictions. Each member of the ensemble has an equal contribution to the final prediction for a data sample. In classification task, prediction probabilities for each class are simply averaged (sum of probabilities divided by m). Models which are combined can be different CNN architectures. A heterogenous collection of varying CNN models can provide better performance and reduce overall variance in contrast to a single architecture.

2.7.3 Weighted Averaging Ensemble

Weighted averaging ensemble is extension to simple model averaging ensemble. Predictions from a model of an ensemble of m models are weighted. There is no direct formula for assigning weightage to different models but can be based upon their results on held-out validation dataset. Such a technique allows better performing models of the ensemble to contribute more than the others in the group.

2.7.4 Weighted Box Fusion

Weighted box fusion (Solovyev et al., 2021) is a novel mechanism to predict the final bounding boxes on an image which has been predicted by N different object detection models as shown in Figure 2.39. It utilizes the confidence scores of all the proposed bounding boxes by different models to construct the final average boxes. Such an approach ensures that boxes with higher confidence scores contribute more to the final fused bounding boxes predictions than boxes with lower scores.

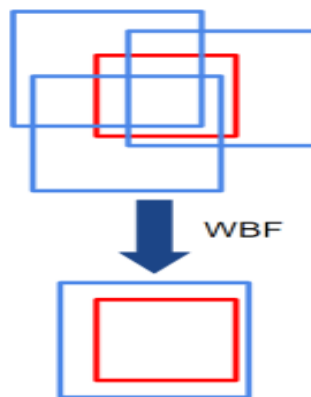


Figure 2.39 Weighted Box Fusion - Blue boxes different models' predictions, red box golden truth (Solovyev et al., 2021)

2.8 Computer Vision in Healthcare

Computer vision based deep learning networks for object detection, image classification and segmentation can greatly benefit the medical and health care industry. Several studies on deep learning for medical imaging have demonstrated profound success in complex medical imaging diagnostics. Such systems could aid medical practitioners and doctors providing second opinions. One of the biggest challenges for these deep learning models to learn and train is the quality, size, and reliability of medical imaging datasets. There are continuous efforts and incremental improvements in that direction in pursuit to improve the performance and generalization of these networks before they can be put into real usage. At the same time, some of the results produced by these models have beaten human experts too. AI in health care is progressing very fast and has the potential to revolutionize and transform the complete industry (Transforming healthcare with AI: The impact on the workforce and organizations | McKinsey, 2021). Applications of computer vision in healthcare:

- Brain Tumor Detection (Badža and Barjaktarović, 2020; Irmak, 2021).
- Pneumonia Detection based upon chest X-rays and CT scans.
- Cancer Detection such as skin cancer, lung cancer (Gulum et al., 2021).
- Recognizing patterns and abnormalities.
- Medical training - Computer vision is also used for medical skill training and not only just medical diagnosis.
- Health Monitoring.

2.9 Predictive Modelling on Chest Images Datasets

2.9.1 CheXNet and ChestNet for Pneumonia Classification on CXRs

A model named CheXNet (Rajpurkar et al., 2017) was developed to classify different types of pneumonia from chest X-ray images. The custom model consisted of 121-layers and was trained on a large dataset that contained over 100,000 X-ray images for 14 different type of diseases. ChexNet claimed outstanding performance at the level of practicing radiologists.

A model named ChestNet (Wang and Xia, 2018) was also developed using the same dataset. The custom model consisted of classification branch based upon ResNet architecture. Classification branch fed the learnt features to an attention branch to exploit the correlation between the learned features and the class labels. Finally, both the branches were combined to learn and predict 14 different type of thorax diseases.

Limitations:

- CheXNet results for multi label prediction of 14 classes were not clearly investigated.
- ChestNet input layer used higher resolution images (1024 x 1024) thus contributing to overall increase of model parameters.
- Both networks are computationally complex. ChestNet took 20 hours to train.

2.9.2 Deep Learning Ensembles for Tuberculosis Classification on Chest Radiographs

Weighted ensemble of InceptionResNet-v2 (0.25 weightage to the predictions), Inception-v3 (0.25 weightage to the predictions) and DenseNet-121 (0.25 weightage to the predictions) models achieved the best results to classify Tuberculosis (TB) infected CXRs vs normal CXRs (Rajaraman and Antani, n.d.). It was also observed that models trained and performed better when they were first pre trained on few publicly available CXR datasets and then fine-tuned on the TB data set.

Limitations:

- Model ensembles are computationally expensive and require more memory.

2.9.3 Foreign Object Detection in Chest X-rays

Faster Region based Convolution Neural Network (Faster R-CNN) achieved 93% F1-score to detect circle-like foreign objects of different sizes in CXRs (Santosh et al., 2020) as depicted in Figure 2.40. Faster R-CNN object detector was trained on a dataset of 400 publicly available CXRs.

Limitations:

- Single-stage object detectors such as EfficientDets and YOLOs were not applied and evaluated.

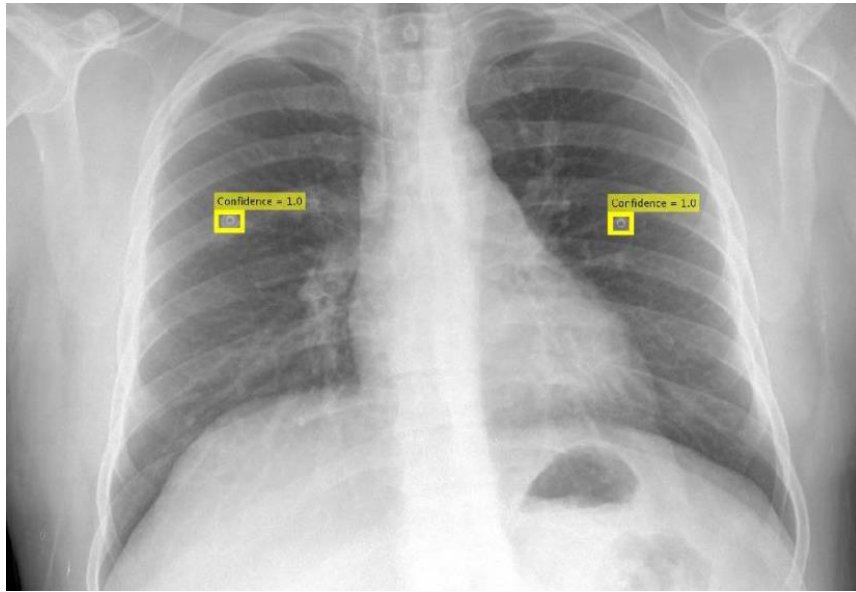


Figure 2.40 Foreign Object Detection on CXRs using Faster R-CNN

2.9.4 Object Detection Ensembles for Pneumonia Opacity Detections in CXRs

Mask-RCNN and RetinaNet (Lin et al., 2020) object detector networks were trained on a competition dataset of 25,684 CXR images annotated with 28,989 bounding boxes. Weighted ensemble of these trained models was used to predict the lung opacity bounding boxes on the test dataset which helped in achieving higher mAP score than the individual model's score (Ko et al., 2019).

Limitations:

- Mask-RCNN is two-stage detector and inference time is quite slow as compared to single-stage detectors.

Another study (The DeepRadiology Team, 2018) implemented a not so famous object detection algorithm called CoupleNet (Zhu et al., 2017) on the same dataset of CXR images labelled with lung opacity bounding boxes and achieved top results on the test dataset consisting of 3000 CXRs.

2.10 Predictive Modelling on COVID Datasets

2.10.1 Datasets used in existing research

Table 2.3 summarizes the COVID chest images datasets that have been used in existing research. As of 2nd October 2021, sizes of these publicly available datasets are mentioned.

Table 2.3 Predictive Modelling - COVID Datasets

Datasets	Total Images	Labels	Additional Information
SARS-CoV-2 CT Scan dataset (Soares et al., 2020)	2482 CT scan images	1252 labelled with COVID 1230 labelled with non-COVID	Scans correspond to 120 patients collected from hospitals in Brazil. Available: www.kaggle.com/plameneduardo/sarscov2-ctscan-dataset
COVID19-CT dataset (He et al., 2020)	746 CT scan images	349 labelled with COVID 397 labelled with non-COVID	349 COVID scans correspond to 143 patients.
Covid Chest X-Ray Dataset (Cohen et al., 2020)	Approx. 660 CXRs	468 labelled with COVID-19	Available: https://github.com/ieee8023/covid-chestxray-dataset
Covid19 Radiography Database (Chowdhury et al., 2020; Rahman et al., 2021)	2905 CXRs	First release- 219 COVID-19 CXRs	Available: https://www.kaggle.com/tawsifurrahman/covid19-radiography-database

		1341 normal CXRs 1345 viral pneumonia CXRs	
COVID-19 Chest X-Ray Dataset Initiative	55 CXRs		Available: https://github.com/agchung/Figure1-COVID-chestxray-dataset
Twitter COVID-19 CXR dataset	134 CXRs	All CXRs correspond to COVID positive patients	Available: https://twitter.com/ChestImaging

2.10.2 Diagnosing COVID-19 pneumonia - AlexNet Transfer Learning

A custom CNN and pretrained AlexNet models were fine-tuned on a very small dataset of 459 images consisting of COVID-19 and normal chest X-ray and CT scan images (Maghdid et al., 2021). This study in early COVID times mainly encouraged research in detecting COVID infection in chest images via CNN models. The solution was not production ready because of limited dataset. Results on the test images are shown in Figure 2.41.

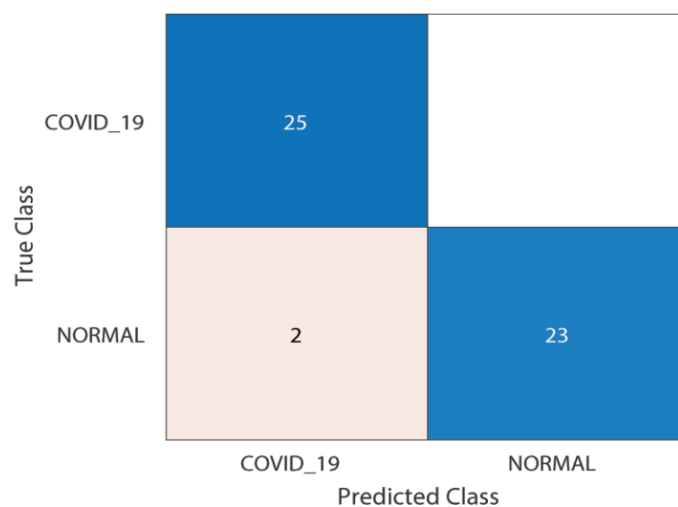


Figure 2.41 Predictive Modelling COVID CXRs - AlexNet Covid vs Normal

Limitations:

- Very small dataset.
- Models may easily be overfitting on such a small dataset.
- Dataset consisting of only normal and COVID CXRs. Bacterial Pneumonia CXRs not part of the dataset. Models won't be able to distinguish between bacterial pneumonia and COVID pneumonia.

2.10.3 COVID19-CT DataSet - Random Initialized Networks vs Transfer Learning

Dataset consisting of 748 scan images is open sourced by this study (He et al., 2020) to facilitate further research. 349 images are labelled as COVID whereas 397 images are labelled as non-COVID. Along with releasing the dataset, the study also highlights the effectiveness of transfer learning via ImageNet pretrained models on this dataset. Performance comparisons between networks with randomly initialized weights and with ImageNet pretrained weights are shown in Table 2.4 when trained on this dataset.

Table 2.4 Predictive Modelling - CT Dataset Performance Comparison between random initialized networks (Random.) and ImageNet Pretrained networks (Transfer.)

	Accuracy		F1 score		AUC	
	Random.	Transfer.	Random.	Transfer.	Random.	Transfer.
VGG-16	0.66	0.76	0.58	0.76	0.74	0.82
ResNet-18	0.67	0.74	0.66	0.73	0.76	0.82
ResNet-50	0.69	0.80	0.72	0.81	0.76	0.88
DenseNet-121	0.76	0.79	0.77	0.79	0.82	0.88
DenseNet-169	0.80	0.83	0.79	0.81	0.86	0.87
EfficientNet-b0	0.72	0.77	0.71	0.78	0.76	0.89
EfficientNet-b1	0.70	0.79	0.62	0.79	0.77	0.84

Limitations:

- Small dataset so models are prone to overfitting.

2.10.4 Multiple CNN architectures on SARS-CoV-2 CT and COVID19-CT datasets

The research was conducted on 2 datasets that are SARS-CoV-2 CT Scan dataset (Soares et al., 2020) consisting of 2482 CT scan images and COVID19-CT dataset (He et al., 2020) consisting of 746 CT scan images. Most advanced ImageNet networks, such as SqueezeNet, Inception, ResNet, ResNeXt, Xception, ShuffleNet, and DenseNet were fine-tuned on these datasets labelled with COVID vs non-COVID images. Custom sized inputs were fed to the networks to maintain the aspect ratio of the original images rather than the recommended input image sizes for these architectures when trained on ImageNet dataset. Table 2.5 highlights some of the models, custom input size and their results achieved in this study. Deeper networks such as Resnet101 and DenseNet201 performed the best (Alshazly et al., 2021).

Table 2.5 Predictive Modelling COVID CXRs - Multiple CNNs

Dataset	Model	Default Input Size	Custom Input Size	Accuracy
SARS-CoV-2 CT	SqueezeNet	227 x 227	335 × 255	95.1 ± 1.3
	ShuffleNet	224 x 224	321 × 225	97.5 ± 0.8
	ResNet50	224 x 224	349 × 253	99.2 ± 0.3
	ResNet101	224 x 224	349 × 253	99.4 ± 0.4
	DenseNet201	224 x 224	349 × 253	99.2 ± 0.2
COVID19-CT	SqueezeNet	224 x 224	335 × 255	87.3 ± 3.2
	ShuffleNet	224 x 224	321 × 225	87.9 ± 2.6
	ResNet50	224 x 224	349 × 253	90.8 ± 1.9
	ResNet101	224 x 224	349 × 253	89.8 ± 2.5
	DenseNet201	224 x 224	349 × 253	92.9 ± 2.2

Limitations:

- Dataset labelled as only normal and COVID CXRs. Bacterial Pneumonia CXRs not labelled as part of the dataset. Models won't be able to distinguish between bacterial pneumonia and COVID pneumonia.
- State of art EfficientNet family of models can provide further gain in accuracy and parameters efficiency.

2.10.5 COVID GANs to classify normal and COVID CXRs

The study (Waheed et al., 2020) was conducted on a dataset of total 1124 CXR images collected from publicly available images (Chowdhury et al., 2020; Cohen et al., 2020). Dataset consisted of 403 images of COVID-CXRs and 721 images of normal CXRs. VGG-16 (Simonyan and Zisserman, 2015) model appended with a couple of custom layers (global average pooling followed by a 64 units dense layer and a softmax layer) was trained on this dataset and achieved an overall accuracy of 85% on the test dataset split out from the total dataset. Further instead of applying standard image augmentation techniques, an auxiliary classifier generative adversarial network (AC-GAN) was trained to generate synthetic images from the original images in the dataset. Generative Adversarial Networks (GANs) consist of 2 sub models, generative and discriminative models competing against each other where finally generative model learns to produce plausible real examples and discriminative sub model is unable to distinguish whether it is a real or fake example. AC-GANs are special type of GANs where discriminator sub model predicts the class label of an image instead of receiving it as an input and allows generation of high-quality images. AC-GAN network was trained to generate approximately 1400 images of normal-CXR and 1670 images of COVID-CXR. VGG-16 architecture was again trained on the original images and these augmented images generated via AC-GANs and achieved an overall accuracy of 95% on the test dataset as depicted in Table 2.6.

Limitations:

- Dataset is limited.
- Bacterial Pneumonia CXRs are not part of the dataset. Models won't be able to distinguish between bacterial pneumonia and COVID pneumonia.
- GAN architecture to generate synthetic images can be improved further.

- Only VGG-16 architecture applied to the dataset which requires a greater number of parameters as compared to ResNet, Shufflenet and various other CNN architectures. Better accuracy and efficiency could have been achieved using other architectures.

Table 2.6 Predictive Modelling COVID CXRs - Covid GAN Test Dataset Results

	Class	Precision	Recall	Accuracy (%)
Actual Data	COVID-CXR	0.89	0.69	85
	NORMAL-CXR	0.84	0.95	
Actual Data + Synthetic Data	COVID-CXR	0.96	0.90	95
	NORMAL-CXR	0.94	0.97	

2.10.6 ResNet50 to classify normal, pneumonia and COVID CXRs

Transfer learning via ResNet50 architecture was applied in this research (Usama et al., 2020) to distinguish between normal, pneumonia and COVID chest X-ray images. It achieved high diagnostic accuracy of 98% which was encouraging that computer vision models can be applied to differentiate the pulmonary changes caused by COVID-19 as compared to other types of pneumonia on digital X-ray images. Confusion Matrix results on both train and test datasets:

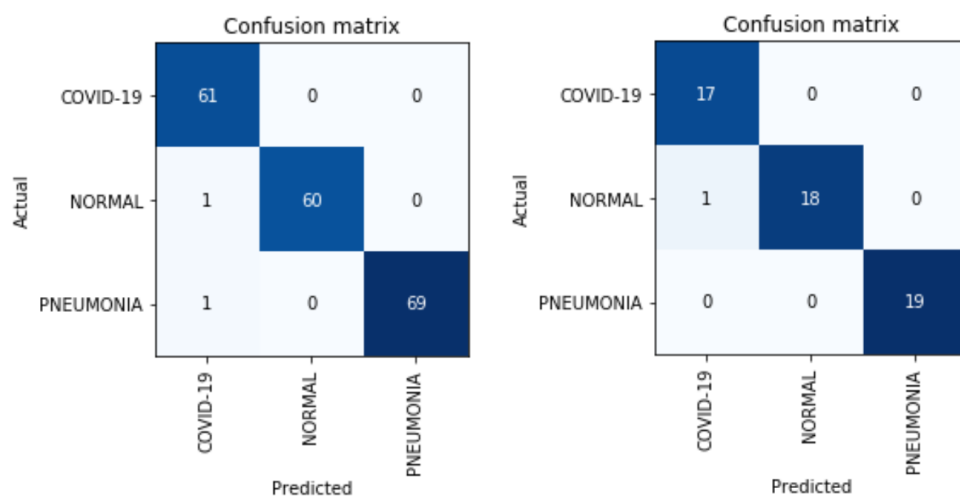


Figure 2.42 Predictive Modelling COVID CXRs - ResNet50 Confusion Matrix

Limitations:

- Main limitation of this study was that it was applied on a very small data set of 278 images and hence has very limited scope for actual usage in real world.
- Models may easily be overfitting on such a small dataset.
- No other architectures are applied to the dataset and compared against the ResNet50 results.

2.10.7 Deep Learning Ensembles to classify normal, bacterial and COVID pneumonia CXRs

Multiple deep learning models were trained to predict a chest X-ray image for normal, bacterial pneumonia or COVID pneumonia (Rajaraman et al., 2020). As part of pre-processing, U-Net model was trained to generate lung masks and these masks were superimposed on the chest X-ray images to generate cropped images of 256 x 256 resolution. These cropped images were finally used to train multiple models as shown in Figure 2.43.



Figure 2.43 Predictive Modelling COVID CXRs - Pre-processed Cropped Lung Images
(Rajaraman et al., 2020)

Pruned VGG-16, VGG-19 (Simonyan and Zisserman, 2015) and Inception-v3 (Szegedy et al., 2016) models were ensembled for the final predictions. Inception-v3, VGG-16 and VGG-19 weightage to the predictions were 0.5, 0.3 and 0.2 respectively. Weighted ensemble of these models gave better results than the individual models on the limited dataset used in this study.

Limitations:

- Limited Dataset.
- VGG architectures used in the ensemble are slower and less efficient as compared to later state of art architectures introduced after them. They are also memory intensive

because of greater number of parameters (5 times as compared to say ResNets). Better accuracy and efficiency could have been achieved using other CNN architectures.

2.10.8 CoroNet and CovidNet to classify normal, viral, bacterial and COVID pneumonia CXRs

CoroNet (Khan et al., 2020) achieved an overall accuracy of 89.6% for 4 class classification that is normal vs viral pneumonia vs bacterial pneumonia vs COVID based upon the CXRs collected from publicly available datasets. CoroNet uses approximately 33 million parameters and is based upon Xception based CNN architecture.

CovidNet (Gunraj et al., 2020) achieved an overall accuracy of 93.3% for 3 class classification between normal, pneumonia and COVID CXRs. Data set size used in training CovidNet was larger than what was used in training CoroNet.

Limitations:

- CoroNet dataset size is small.

Table 2.7 CoroNet DataSet

Class	No of images
Normal	310
Pneumonia bacterial	330
Pneumonia viral	327
COVID-19	284

- CoroNet and CovidNet architectures can further improve.

2.11 Review of top solutions

(COVID-19 AI Detection Challenge (2021), 2021) explores the top-10 classification and detection solutions for the dataset used in this research. Results of the challenge have been published recently in September 2021. EfficientNet, DeepLab, NfNet are some of the architectures that have been used for the classification task. EfficientDet, YOLO, Mask-RCNN, RetinaNet, Detection Transformers are some of the architectures that have been used for the detection task. Most of the winning solutions have ensembled more than 15 trained models on 3 to 4 architectures for both classification and detection predictions. Pre training on external

datasets and test time augmentation are some of the common methods that have been leveraged to improve the performance of the models.

2.12 Discussion

Building a diverse and reliable dataset of chest CT scans and X-rays to detect COVID pneumonia has been a challenge. Most of the study that has been conducted to classify COVID pneumonia is either a binary class (normal vs COVID pneumonia) or a three class (normal, bacterial or COVID pneumonia) classification. What is really needed is to define definite classes and group categories of pneumonias for chest X-rays to determine the severity of COVID impact on a patient and build AI models to classify those categories. Also, very few object-detection models have been developed to localize the opacities on COVID CXR datasets.

2.13 Summary

To summarize, Deep learning and Computer Vision Networks are having a significant impact in healthcare and medical industry. Image classification models can play an important role in combating COVID pandemic by analysing and examining Chest X-rays and classifying them for different categories of COVID pneumonia. Object detection models can further assist in diagnosis of CXRs by determining the location of opacities in the form of bounding boxes on chest images.

Deeper image classification models such as ResNets, DenseNets and EfficientNets perform better for complex vision tasks in terms of accuracy as well as execution speed. YOLOs and EfficientDets are state of art architectures for object detection tasks. These compute vision networks, pretrained on ImageNet or COCO databases transfer well on other tasks. Ensembling or combining multiple trained models generally give better results on the test and validation data sets and hence ensembling techniques can perform better with real world data. Special techniques such as weighted box fusion have been developed to combine results from multiple object detection models.

Predictive modelling on chest images datasets to detect and classify various diseases such as pneumonia or tuberculosis or abnormalities such as foreign objects has been quite successful in last few years and continues to progress and improve further. Similar research and studies have been pursued on chest images datasets to detect and classify COVID-19 pneumonia.

Chapter 3 RESEARCH METHODOLOGY

3.1 Introduction

The study will encompass multiple processes and pipelines to predict COVID-19 abnormalities. Description of the dataset used in this research is detailed in section 3.2. The study will include CRISP-DM processes of data understanding followed by data preparation and feature engineering which are the most important and time-consuming steps. Overall research methodology flowchart and training pipeline are covered in section 3.6. Training of classification and detection models are explored in sections 3.7 and 3.8 respectively. Evaluation metrics for the models are discussed in 3.9.

3.2 Dataset Description

BIMCV-COVID19 Data and MIDRC-RICORD Data will be used in this research and is available online in Kaggle repository. Train dataset comprises 6,334 chest scans in DICOM format. All images are de-identified to protect patient privacy and are labelled by experienced radiologists for the presence of opacities as well as overall appearance. For each image, model is expected to predict both a bounding box and an overall class for all findings.

For each object in an image, model must predict a class ID of "opacity", a confidence score, and bounding box in the format of (xmin, ymin, xmax ymax). Images may contain more than one object. If there are no findings, a prediction of "none 1 0 0 1 1" ("none" is the class ID for no finding, and this provides a one-pixel bounding box with a confidence of 1.0) is expected.

Further for each image (Figure 3.1), an overall class from 'Negative for Pneumonia', 'Typical Appearance', 'Indeterminate Appearance' and 'Atypical Appearance' needs to be determined.



Figure 3.1 Chest X-ray

Chest radiographs (CXRs) were categorized using a specific grading schema, based on a published paper (Chung et al., 2020). Per the grading schema, chest radiographs were classified into one of four mutually exclusive categories:

- **Negative for Pneumonia:** No lung opacities.
- **Typical Appearance:** Multifocal bilateral, peripheral opacities with rounded morphology, lower lung–predominant distribution.
- **Indeterminate Appearance:** Absence of typical findings AND unilateral, central, or upper lung predominant distribution.
- **Atypical Appearance:** Pneumothorax, pleural effusion, pulmonary edema, lobar consolidation, solitary lung nodule or mass, diffuse tiny nodules, cavity.

Bounding boxes were placed on lung opacities, whether typical or indeterminate. Bounding boxes were also placed on some atypical findings including solitary lobar consolidation, nodules/masses, and cavities. Bounding boxes were not placed on pleural effusions, or pneumothoraces. No bounding boxes were placed for the negative pneumonia category. In cases of multiple adjacent opacities, one large bounding box was opted, rather than multiple adjacent smaller boxes, to improve consistency in the labelling.

Annotators did have access to the COVID status for each patient but were asked to adhere to the above grading system irrespective of the status. As such, some patients who were COVID negative still had chest radiographs with typical appearances. Similarly, some patients who were COVID positive had atypical appearances, or were negative for pneumonia (no lung opacities), because the grading system is based off the chest radiographic findings alone.

3.3 Data Understanding

Number of training samples are 6334 images whereas number of test samples are 2447 images. All the images are in DICOM format.

Following figures reflect some of the images belonging to the different classes in the training data set.

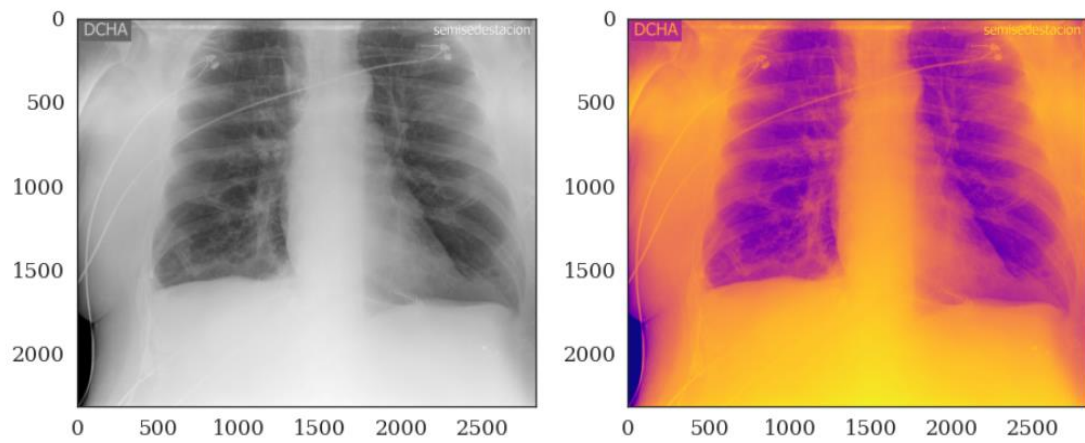


Figure 3.2 Chest X-ray Normal/Negative Class

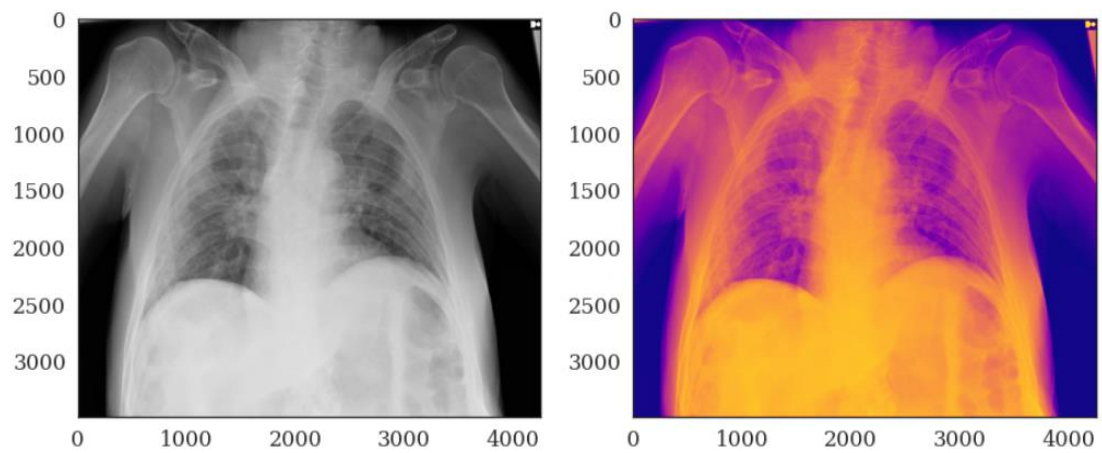


Figure 3.3 Chest X-ray Typical Class

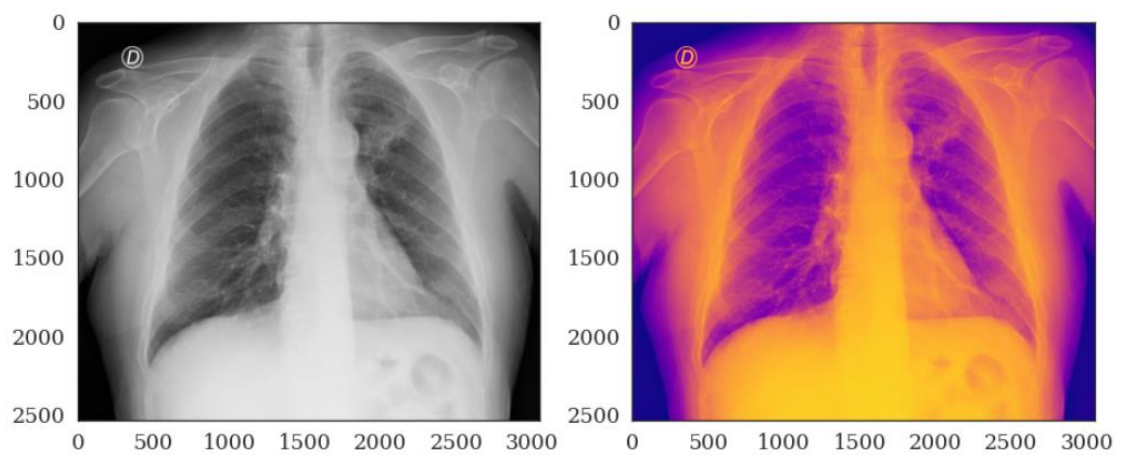


Figure 3.4 Chest X-ray Indeterminate Class

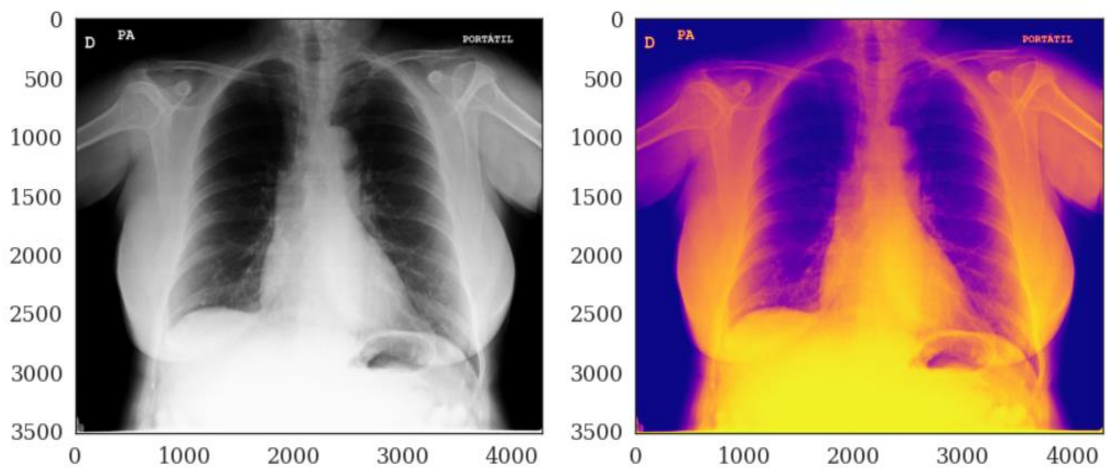


Figure 3.5 Chest X-ray Atypical Class

Training data set has the following distribution among the 4 classes (Figure 3.6). It can be observed that the data set is quite imbalanced.

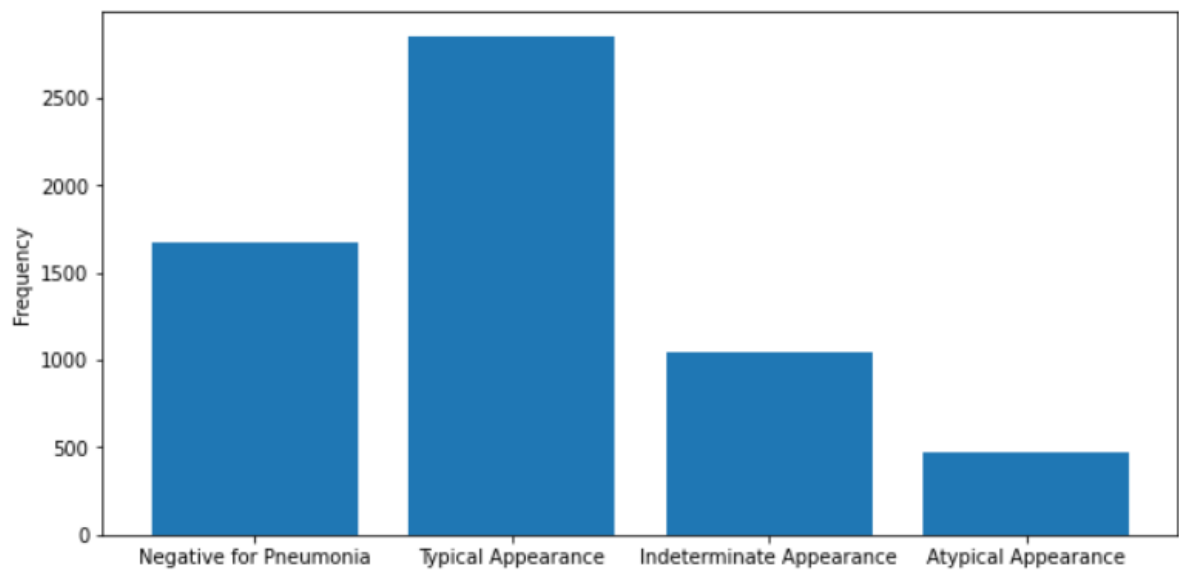


Figure 3.6 Training Dataset Class Distribution

Figure 3.7 reflects the distribution of training images which are annotated with bounding boxes labels.

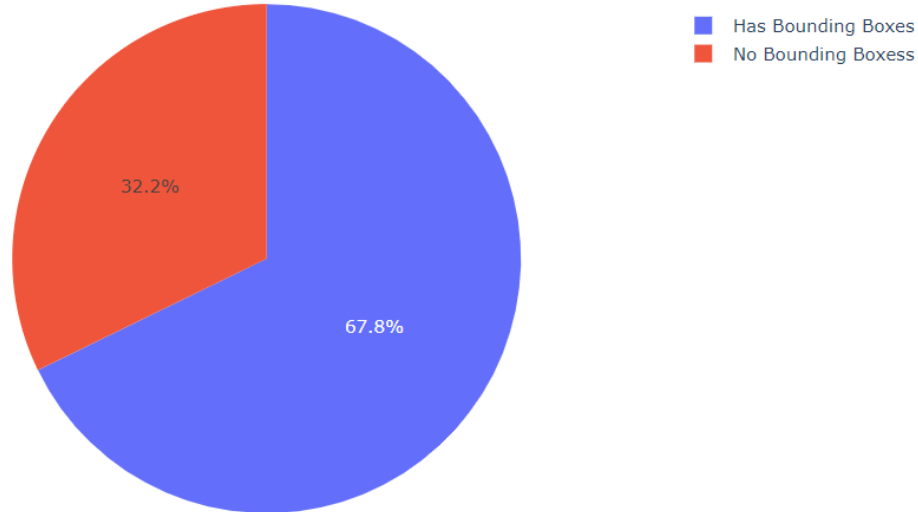


Figure 3.7 Training Dataset Bounding Box Distribution

3.4 Data Preparation and Pre-Processing

As part of data preparation, DICOM images from the original training database will need to be read, resized, and saved to png formats. Images are in DICOM format, which means they contain additional data that might be useful for visualizing and classifying. Training database will be resized to generate training images of different sizes such as 256x256, 384x384, 512x512 and 728x728.

Stratified group k fold validator will be applied on the annotations to preserve the percentage of each class across each fold. Figure 3.8 depicts 5-fold cross validation across 2 classes. Similarly, it will be applied for the 4 target classes in this study. This step will also require removal or correction of erroneous annotations and labels. There are approximately 200 images in the training data set that are duplicate. The images which are removed are the ones that have missing bounding box information. New annotations or columns will be derived such as “bounding box present” which will help in improving the accuracy and generalization of our final models.

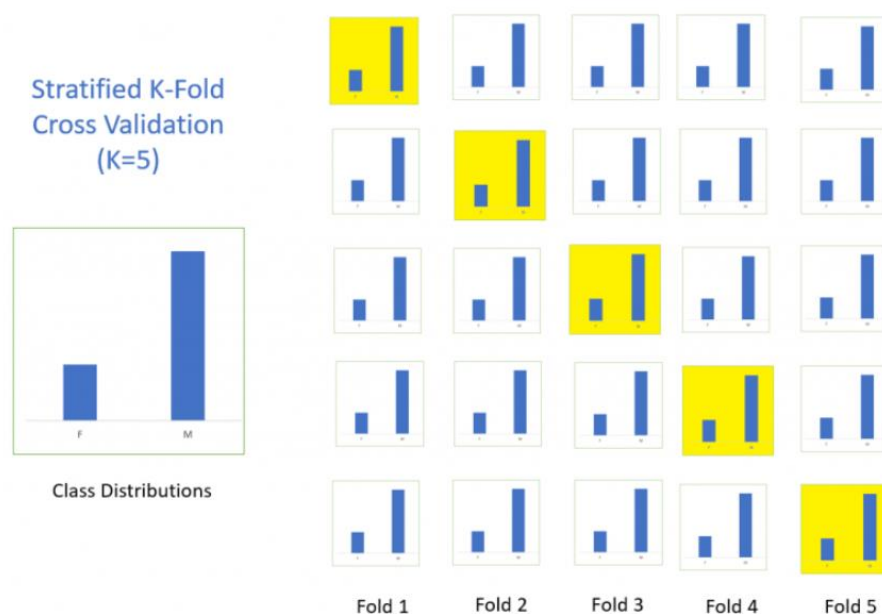


Figure 3.8 Stratified k-fold

Table 3.1 depicts all the original and derived training columns corresponding to the dataset.

Table 3.1 Training Dataset Annotations

Field	Description
Image Id	Unique Image Identification.
Bounding Boxes Labels	In the form of (x, y, width, height)
Bounding Boxes Labels	In the form of (x center, y center, width, height)
Bounding Boxes Labels	In the form of (x1, y1, x2, y2)
Negative for pneumonia	True or False
Typical appearance	True or False
Indeterminate Appearance	True or False
Atypical appearance	True or False
Width of original image	Self-explanatory
Height of original image	Self-explanatory
K fold	If K is 5 then between 0 and 4
Bounding box present	True or False

3.5 Feature Engineering

This is an important step to handle class imbalance along with building a data augmentation pipeline.

- Images will be normalized in the range of 0 to 1.
- Bounding boxes will be normalized in the range of 0 to 1.
- Data augmentation pipeline will include rotation, shear, horizontal zoom, vertical zoom, horizontal shift, and vertical shift. Based upon the rotation, shear, zoom and shift values, a single transformation matrix will be derived and applied on the original images.
- Data augmentation pipeline will also include random flip left and right, random saturation, random contrast, and random brightness.
- Complex data augmentation techniques such as Random Erase, Cut Out Augmentation, Cut Mix Augmentation, or Mix up Augmentation can also be applied.

3.6 Flowchart and Training Pipeline

Figure 3.9 depicts the research methodology flowchart that summarizes the required steps from loading of training dataset followed by data preparation and feature engineering. Classification and object localization models will then be developed to learn on the training dataset. Ensembling results from multiple models will be further researched to reduce the variance of predictions and reduce generalization error and achieve higher accuracy. Final steps would be model evaluation based upon metrics on unseen test dataset for choosing the best set of models to conclude the research.

Figure 3.10 summarizes the full training pipeline for developing models starting from pre-processing of DICOM images to png images of size 1024x1024. Stratified 5-fold will be applied to split into training and validation datasets and models will be trained 5 times. Augmentation pipeline will transform the images before feeding them to the input layer of the network. At the end of each epoch, loss and metrics will be calculated on the validation images, and best checkpoint trained models will be saved for final predictions.

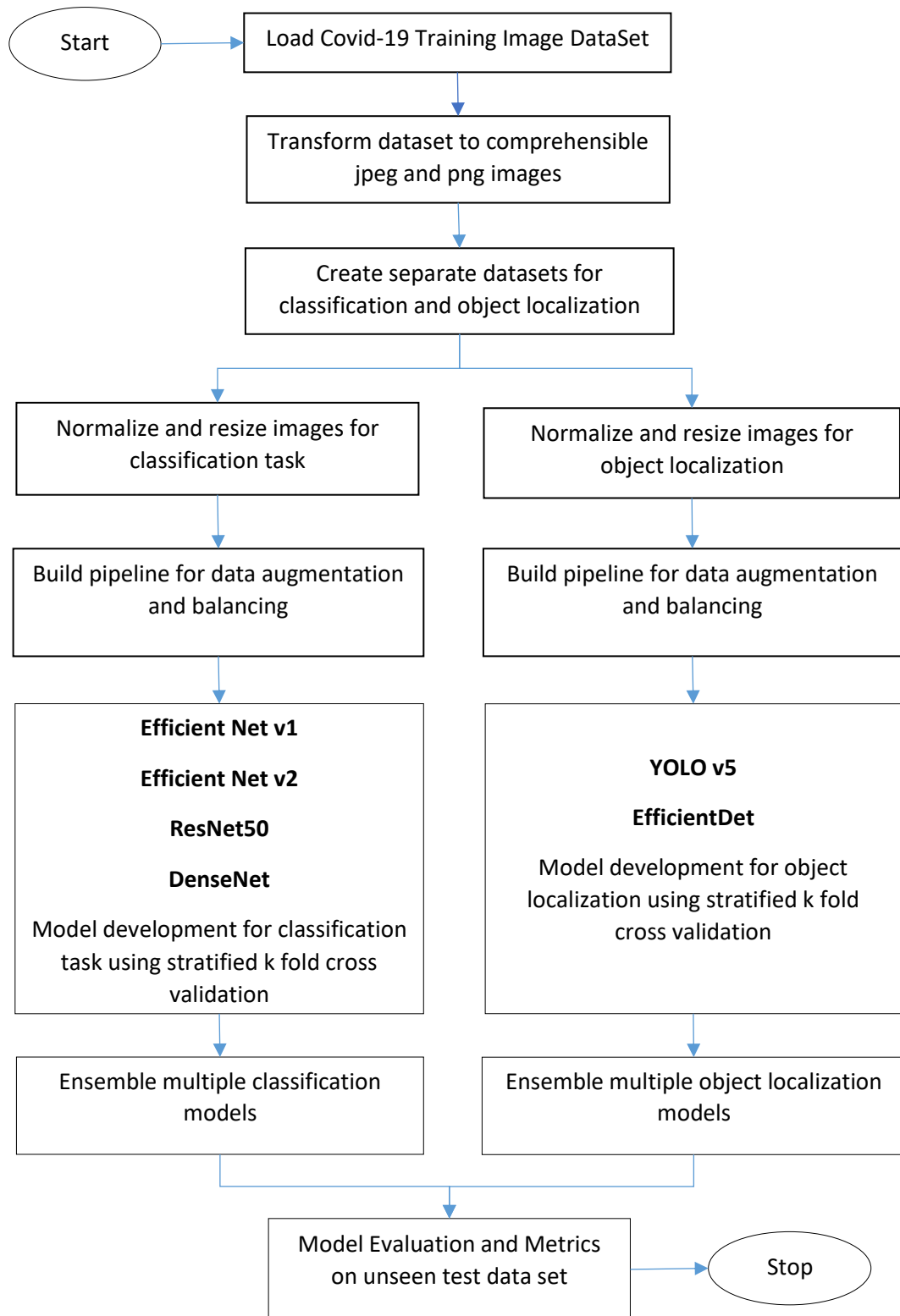


Figure 3.9 Research Methodology Flowchart

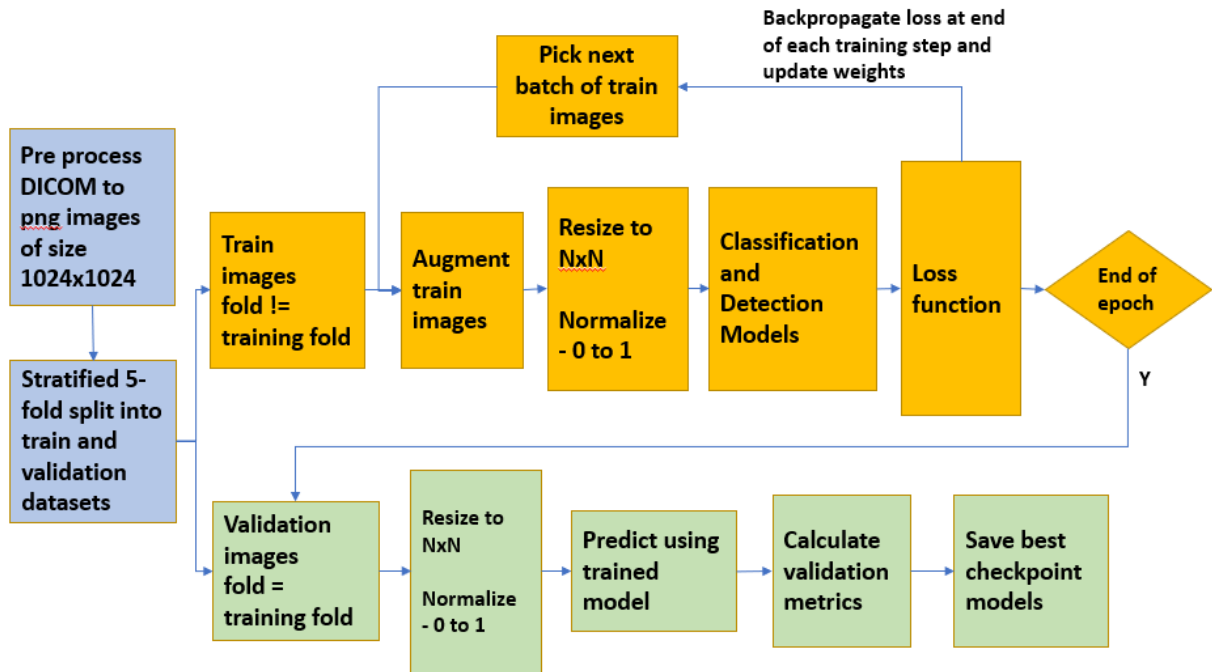


Figure 3.10 Training Pipeline

3.7 Training Classification Models

Following model architectures will need to be tried for classification task on the dataset.

3.7.1 ResNet

ResNet based algorithms have always been a good starting point and preferred option for image classification. Transfer learning on our dataset via ResNet50 pretrained model on ImageNet dataset will serve as a good baseline architecture. All the layers of ResNet50 will be retrained. ResNet50 architecture will be applied for k (usually 5) folds to generate k models. Accuracy and AUC metrics on out of fold data samples will be evaluated.

3.7.2 DenseNet

DenseNet is another popular and powerful choice for image classification. DenseNet will require lesser number of training parameters as compared to ResNet50. Transfer learning on our dataset via DenseNet pretrained model on ImageNet dataset will also serve as a good architecture. All the layers of DenseNet will be retrained. Transfer learning via DenseNet will be applied for k (usually 5) folds to generate k models. Accuracy and AUC metrics on out of fold data samples will be evaluated.

3.7.3 EfficientNet

EfficientNetv1/v2 - EfficientNet architectures are the latest state of architecture in terms of quality as well as faster execution for image classification. EfficientNets do well in transfer learning. We have a whole family of EfficientNetv1 architectures namely B0 to B7 to try out on our data set. Similarly, EfficientNetv2 have small, medium, and large implementations that can be applied. All the layers of EfficientNets will be retrained. We expect EfficientNet models to perform the best on our dataset.

3.7.4 Parameters for Training Classification Models

Softmax activation function (Figure 3.11) will be applied on the output layer to convert the output logits to probability values between 0 and 1 for the 4 classes. Categorical cross entropy (CCE) loss function as defined in Figure 3.12 will be used to train all the models for our 4-class classification problem. Target or golden truth will need to be provided in one hot encoded format. Parameters for training classification models are defined in Table 3.2. Class weights will be passed to handle class imbalance so that more attention is paid to the under sampled classes of indeterminate and atypical categories. Indeterminate and atypical classes will be given more weightage as compared to negative and typical classes.

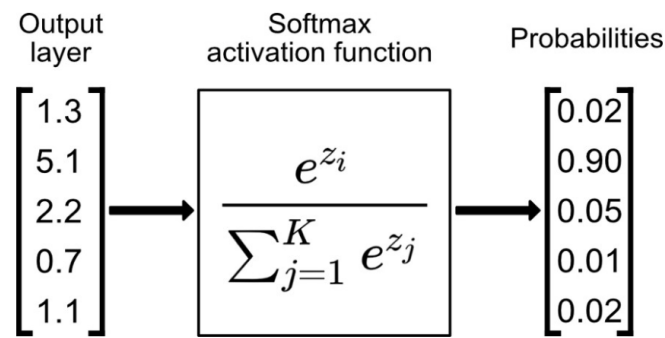


Figure 3.11 Softmax Activation

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Figure 3.12 Categorical Cross Entropy

Table 3.2 Parameters for training classification models

Models	Architecture	Loss function and Class Weights	Batch Size and No of epochs	Callbacks
ResNet50	Models pretrained on ImageNet + Dropout + Dense Layer with 4 neurons and softmax activation	Loss function : Categorical cross entropy	Batch Size - 16 Epochs - 15	Monitor validation loss and reduce learning rate by a factor if the loss stops improving for some epochs.
DenseNet201				
EfficientNetv1-B5		Class Weights : 1:1:2:2 ratio for Negative:		
EfficientNetv2-Medium		Typical: Indeterminate: Atypical		

3.7.5 Ensembling Classification Models

Predictions on the test data set from multiple classification k-fold models will be ensembled or combined using simple average mean or weighted mean depending upon their out of fold validation scores on the training data set. Ensembling often results to better performance and generalization rather than relying on a single model. Approx. 10 to 15 models may be combined for better performance on the test data set as depicted in Figure 3.13.

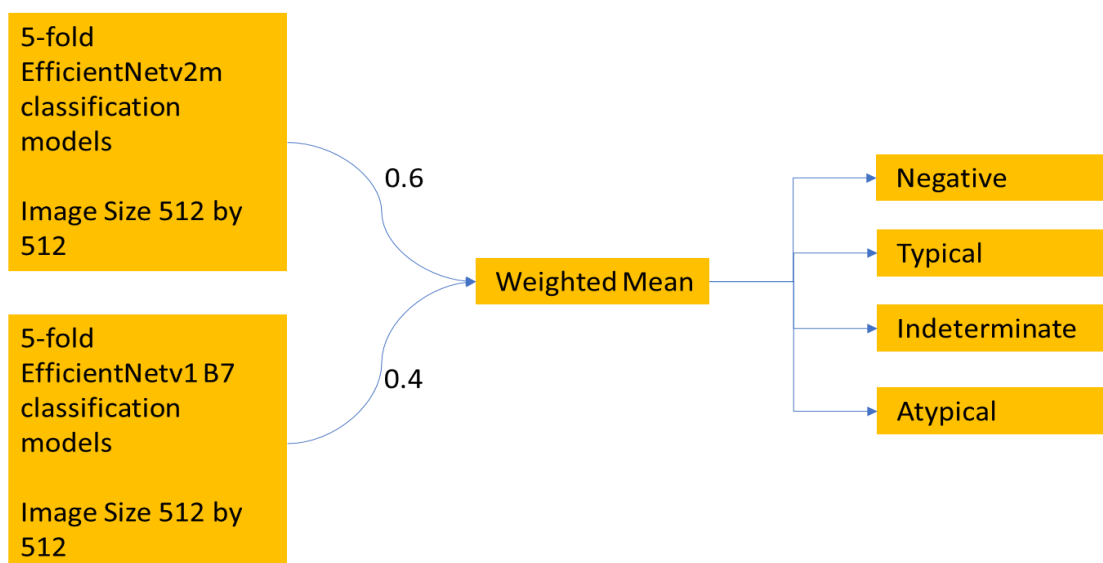


Figure 3.13 Ensembling Classification Models

3.7.6 Binary Classification Model to predict presence of bounding boxes

A training column indicating the presence of bounding boxes will be derived and a separate classification model based upon EfficientNet will be trained to predict this binary class. This model can be leveraged in the final post processing step to power blend with the results of detection models to tune the confidence scores of the detection classes that is either opacity or None.

3.8 Training Object Detection Models

Following model architectures will need to be tried for object detection task on the dataset.

3.8.1 YOLO

YOLO or “You Only Look Once” is one of the most famous and versatile object detection models. YOLO algorithms have been the first choice for data scientists for any object detection tasks. YOLO will require csv annotations to be converted to COCO format and configuration files will be created specifying the path to train, test and validation data sets, number of object classes and the class names. YOLO models will be trained on the images annotated with bounding boxes labels.

As an example, YOLO configuration file:

```
# train and val datasets (image directory or *.txt file with image paths)
train:
  - data/covid/fold_1/images
  - data/covid/fold_2/images
  - data/covid/fold_3/images
  - data/covid/fold_4/images

val: data/covid/fold_0/images
test: data/covid/fold_0/images

# number of classes
nc: 1

# class names
names: [opacity]
```

3.8.2 EfficientDet

These are new family of detection architectures that are efficient and can achieve high accuracy results. EfficientDet variants will be trained on the images annotated with bounding boxes labels. Configuration files will need to be created for training EfficientDet models.

As an example, EfficientDet configuration file:

```
data: data/grouped_df.csv
root: data/images
print_every: 50
device: cuda
bs: 12
epochs: 20
mosaic: 0.0
mixup: 0.0
set_bn_eval: false
lr: 1e-04
initial_checkpoint: null
output: runs/
model: tf_efficientdet_d5_ap
model_ema: true
model_ema_decay: 0.999
image_size: 512
name: exp1
```

3.8.3 Parameters for Training Object Detection Models

Table 3.3 reflects some of the key hyperparameters for training object detection models.

Table 3.3 Parameters for training object detection models

Models	Architecture	Learning rate and IOU threshold for non max suppression	Batch Size and No of epochs
YOLOv5	YOLOv5 pretrained on COCO dataset	Learning rate : 1e-04 IOU threshold : 0.5	Batch Size : 16 Epochs : 20
EfficientDet-D5	EfficientDet-D5 pretrained on COCO dataset	Learning rate : 1e-04 IOU threshold : 0.5	Batch Size : 12 Epochs : 20

3.8.4 Weighted Box Fusion

Weighted box fusion will be applied to predict the final bounding boxes on an image which has been predicted by N different models as depicted in Figure 3.14. For example, if we have bounding boxes predictions from both EfficientDet and YOLO models then we can assign weights to each of the model predictions and ensemble results from both the architectures using weighted box fusion.

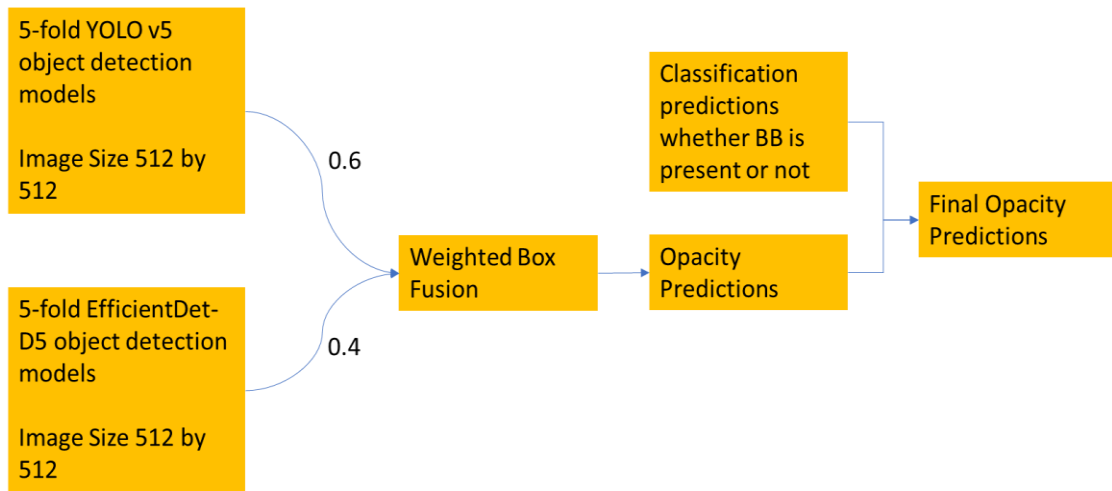


Figure 3.14 Ensembling Object Detection Models

3.9 Evaluation Metrics

Following metrics will be used to evaluate the models:

3.9.1 Accuracy

Accuracy of the model is the proportion of the total number of predictions across all classes that were correct. Confusion Matrix for the classification predictions is depicted in Table 3.4 where TP stands for True Positives and FN stands for False Negatives. Accuracy based upon the report will be sum of all elements along the diagonal divided by all data points.

3.9.2 AUC-ROC

Area under the curve (AUC) is computed for each class label and then averaged across each label. AUC-ROC curve is a plot between true positive rate and false positive rate as shown in Figure 3.15.

Table 3.4 Classification Report

Actual/Predicted	Negative	Typical	Indeterminate	Atypical
Negative	TP	FN	FN	FN
Typical	FN	TP	FN	FN
Indeterminate	FN	FN	TP	FN
Atypical	FN	FN	FN	TP

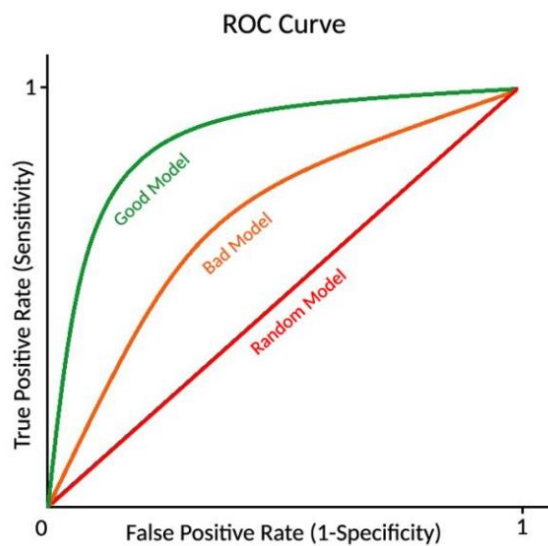


Figure 3.15 AUC ROC

3.9.3 Precision/Recall/F1 score

Precision is basically proportion of positive predictions that are correct whereas Recall is proportion of actual positive cases that are correctly identified. F1 score is basically harmonic mean of precision and recall. F1 score, precision and recall for each class label will be evaluated as shown in Table 3.5.

Table 3.5 Precision Recall Report

Pneumonia Class	Precision	Recall	F1 Score
Negative	X	X	X
Typical	X	X	X
Indeterminate	X	X	X
Atypical	X	X	X

3.9.4 Mean average precision (mAP)

mAP is calculated for both classification and bounding boxes predictions. mAP is basically the area under the precision/recall curve. For detection task, evaluation metric of Intersection Over Union (IOU) determines the precision and recall by comparing the overlap area between the predicted bounding boxes and the ground truth bounding boxes. Detections are considered true or false positives based on the area of overlap with ground truth bounding boxes. For example, if IOU threshold is set as 0.5 and IOU is calculated as 0.75 then the prediction is considered as true positive. If IOU threshold is set as 0.5 and IOU is calculated as 0.35 then the prediction is considered as false positive. When there is no detection at all against the golden truth or when the predicted object class is incorrect for $\text{IOU} > \text{IOU threshold}$ then those data points are counted as false negatives. In the equation for calculating IOU, a_o is the area of overlap between the predicted bounding box B_p and ground truth bounding box B_{gt} .

$$a_o = \frac{\text{area}(B_p \cap B_{gt})}{\text{area}(B_p \cup B_{gt})}$$

An example Precision-Recall curve is shown in Figure 3.16.

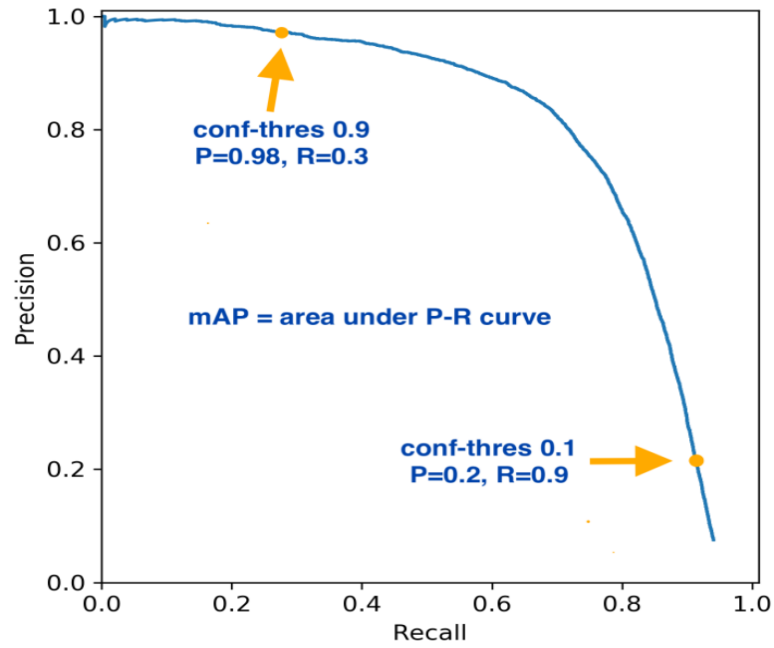


Figure 3.16 Precision Recall Curve

3.9.5 Summary of evaluation metrics for various models

Table 3.6 highlights the overall evaluation for classification models that needs to be targeted in this research.

Table 3.6 Evaluation of Classification Models

Models	Image Size	Accuracy			mAP		
		Train	Validation	Test	Train	Validation	Test
ResNet50	512 by 512	X	X	X	X	X	X
DenseNet201	512 by 512	X	X	X	X	X	X
EfficientNetv2 EfficientNetv1	512 by 512	X	X	X	X	X	X
Ensemble of EfficientNets	512 by 512	NA	X	X	NA	X	X

Table 3.7 highlights the overall evaluation for object detection models that needs to be targeted in this research.

Table 3.7 Evaluation of Object Detection Models

Model	Image Size	Train Dataset mAP	Validation Dataset mAP	Test Dataset mAP
YOLOv5	640 by 640	X	X	X
EfficientDet	640 by 640	X	X	X
Weighted ensemble of YOLOv5 and EfficientDet	640 by 640	NA	NA	X
Fine tune confidence scores of bounding boxes following weighted ensemble using binary classification model	640 by 640	NA	NA	X

3.10 Requirements Resources

Hardware/Cloud:

- 32 GB DDR.
- 180 GB Free space.
- Desktop/Laptop with GPU.
- GPU on Google Colab and Kaggle.
- TPU v3-8 on Google Colab and Kaggle.

Software:

- Dataset
- Python ≥ 3.6
- Python packages - numpy, pandas, scikit, matplotlib, seaborn, PyYAML, tqdm etc.
- TensorFlow ≥ 2.5
- PyTorch = 1.8.1
- Torchvision = 0.9.1
- Albumentations = 1.0.3
- Pytorch-lightning = 1.3.8

3.11 Summary

To summarize, goal of this research is to identify and localize COVID-19 abnormalities on chest radiographs. This is an object detection and classification problem. Classification models will be able to predict overall appearance class of the chest X-ray images which are negative for pneumonia or typical, indeterminate, or atypical. Object detection models will be able to predict and localize the opacities in terms of bounding boxes on chest X-ray images.

Various state of the art architectures such as ResNets, DenseNets, EfficientNets for image classification and YOLOs, EfficientDets for object detection will be applied on this dataset and results from multiple models will be ensembled for better performance. Out of fold cross validation scores will help in deciding the weights for ensembling multiple models to perform predictions on a test dataset. Finally, models will be evaluated and compared based upon various metrics such as accuracy, F1 score, mAP etc to decide the best performing solution that can be deployed in real world.

Chapter 4 ANALYSIS AND IMPLEMENTATION

4.1 Introduction

Dataset used in this research comprises of 6334 chest X-ray scans in medical imaging format and the task is to learn the classification class among 4 classes that are negative, typical, indeterminate, and atypical. The task also involves localizing the opacities in the form of bounding boxes. Section 4.2 covers data analysis in depth and highlights all the challenges in the dataset. Sections 4.3 and 4.4 cover data preparation and data partitioning respectively. Data augmentation techniques are explored in section 4.5. Implementation and experiments of various image classification models are discussed in section 4.6. Implementation and experiments of object detection models are discussed in section 4.7.

4.2 Exploratory Data Analysis

Chest radiographs (CXRs) are categorized using a specific grading schema (Chung et al., 2020; Kanne et al., 2021). CXRs belonging to negative class are depicted in Figure 4.1. Negative class comprises of the images with no lung opacities and hence are not labelled for any opacities in terms of bounding boxes. CXRs belonging to typical class are depicted in Figure 4.2 along with bounding boxes annotations. Typical class comprises of images with multifocal bilateral, peripheral opacities with rounded morphology, or lower lung–predominant distribution of opacities. CXRs belonging to indeterminate class are shown in Figure 4.3 along with bounding boxes annotations. Indeterminate class comprises of images that do not have any typical findings and unilateral, central, or upper lung predominant distribution of opacities. CXRs belonging to atypical class are shown in Figure 4.4 along with bounding boxes annotations. Atypical class comprises of images with Pneumothorax, Pleural effusion, Pulmonary edema, Lobar consolidation, solitary lung nodule or mass diffuse tiny nodules, cavity. Bounding boxes are not placed for atypical class images with pleural effusion or pneumothorax. For all other atypical images, bounding boxes are labelled.

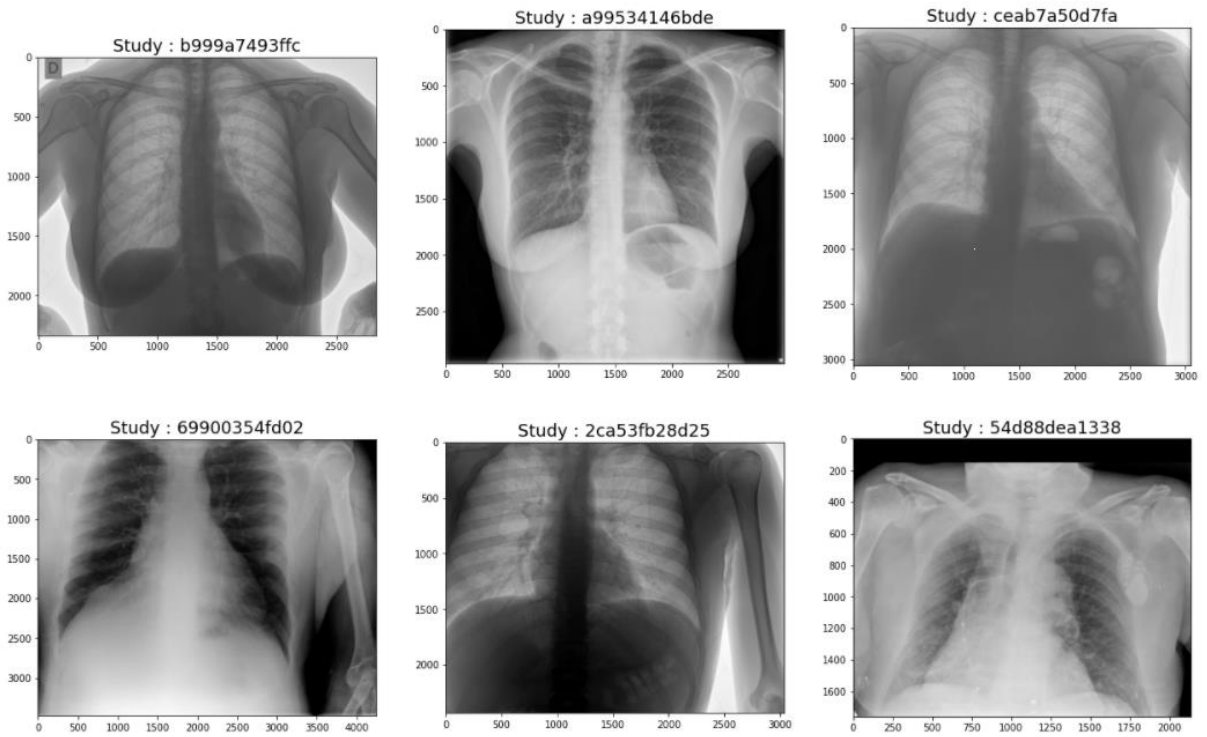


Figure 4.1 Visualizing training CXRs labelled as negative class

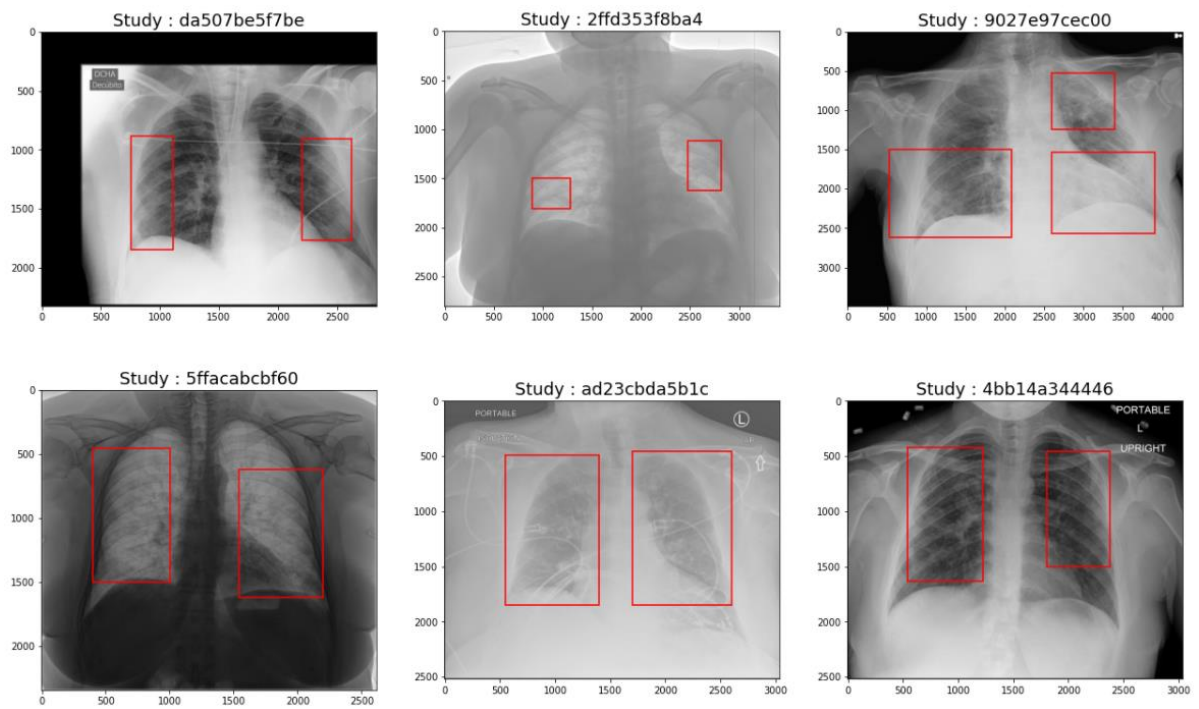


Figure 4.2 Visualizing training CXRs labelled as typical class with bounding boxes

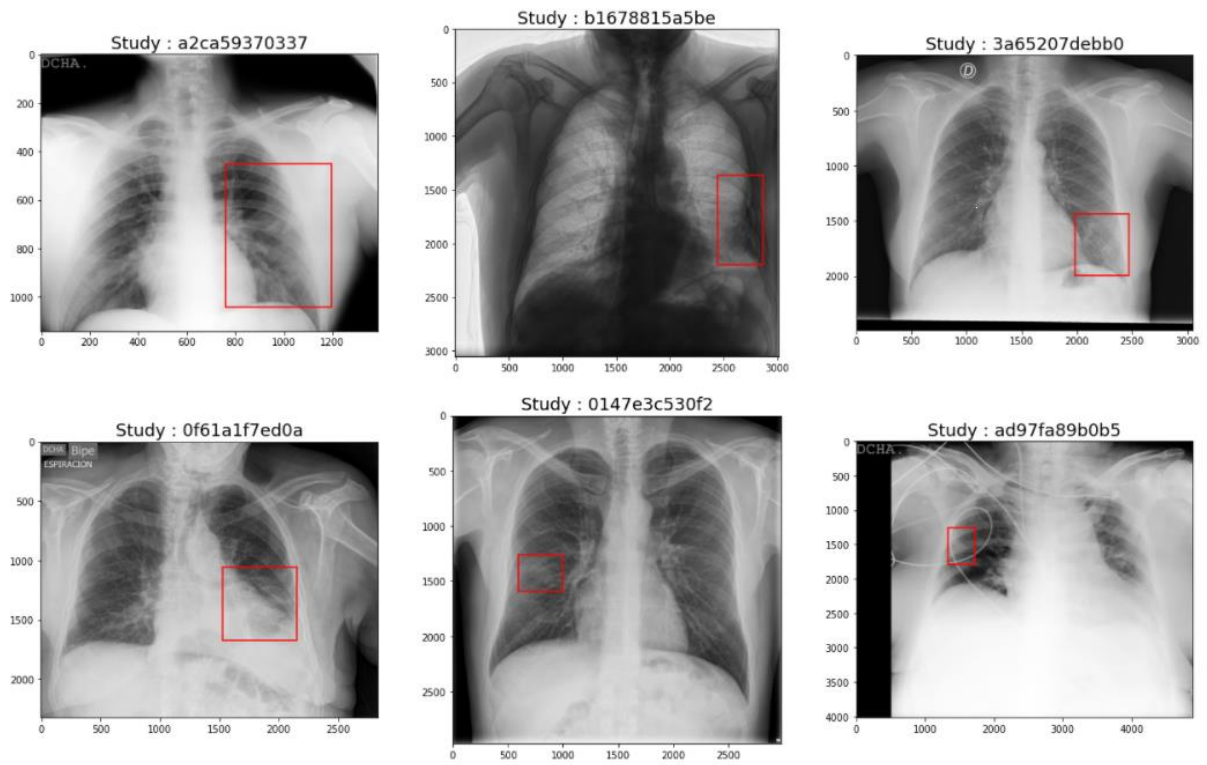


Figure 4.3 Visualizing training CXRs labelled as indeterminate class with bounding boxes

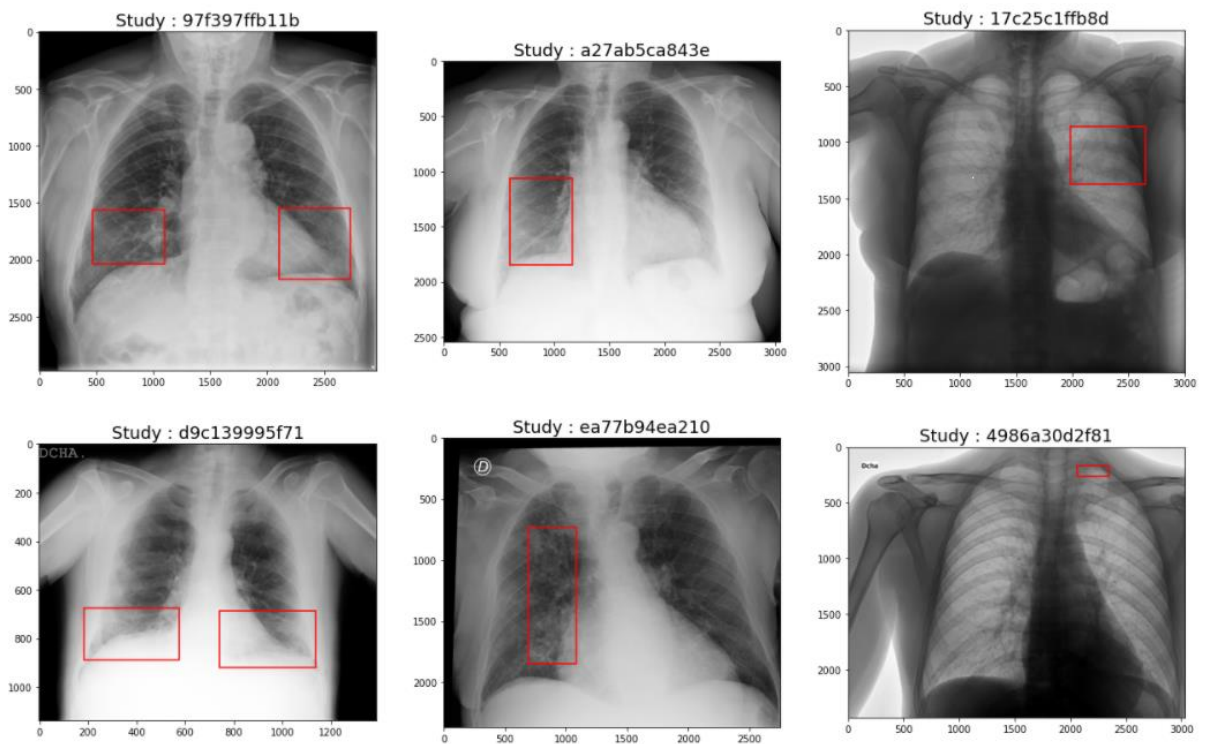


Figure 4.4 Visualizing training CXRs labelled as atypical class with bounding boxes

Figure 4.5 reflects the scatter plot of the original image sizes in the dataset with image rows or heights in x axis and image columns or widths in y axis. It can be inferred that most of the images have dimensions between 2000 and 3200. Majority of the images do not have width same as height, so it is important to maintain the aspect ratio when converting and resizing original DICOM images to png or jpeg formats.

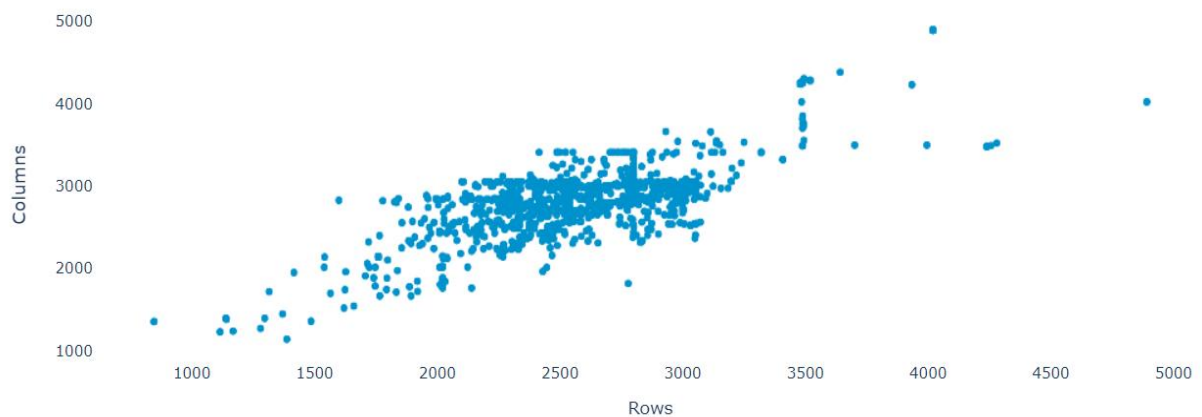


Figure 4.5 Scatter plot of image size

Figure 4.6 indicates the distribution of classification classes across the training dataset. Training images with negative class are 1736, images with typical class are 3007, images with indeterminate class are 1108 and images with atypical class are 483. Ratio of negative: typical: indeterminate: atypical images is approximately 0.28:0.47:0.17:0.08 as indicated in Figure 4.7. It can be inferred that the dataset is quite imbalanced specially for indeterminate and atypical classes which constitute approximately 17% and 8% of training dataset respectively.

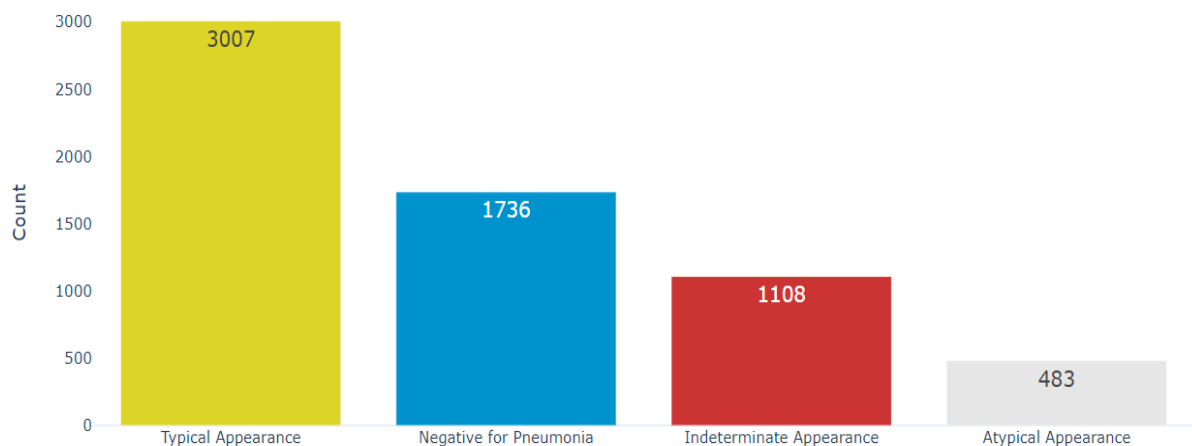


Figure 4.6 Distribution of classification classes in training dataset

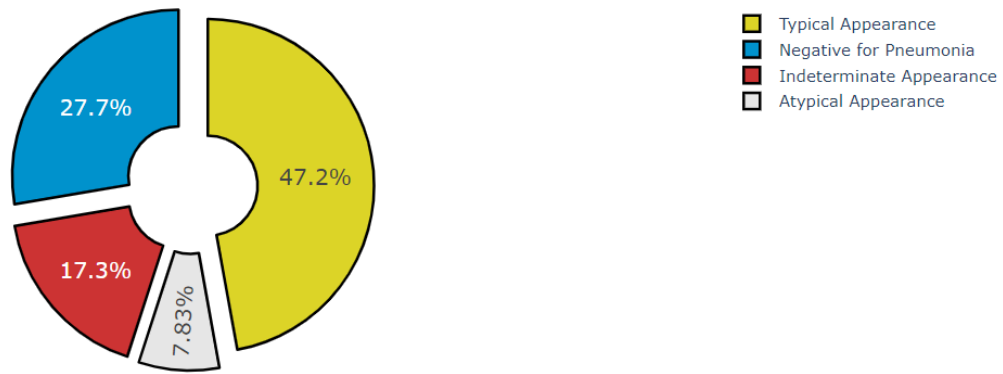


Figure 4.7 Distribution % of classification classes in training dataset

Figure 4.8 indicates the count of training images with bounding boxes annotations across the classification classes on the training dataset. CXRs with opacities are mostly labelled with one or two bounding boxes whereas fewer CXRs with opacities are labelled with either three or four bounding boxes. Indeterminate and atypical class images are more likely to be labelled with one bounding box whereas typical class images are more likely to be labelled with two bounding boxes. There are few images which belong to non-negative classes but are not annotated with bounding boxes. Those are duplicate images having missing annotations of bounding boxes and are removed in data cleaning step as explained later in section 4.3.2.

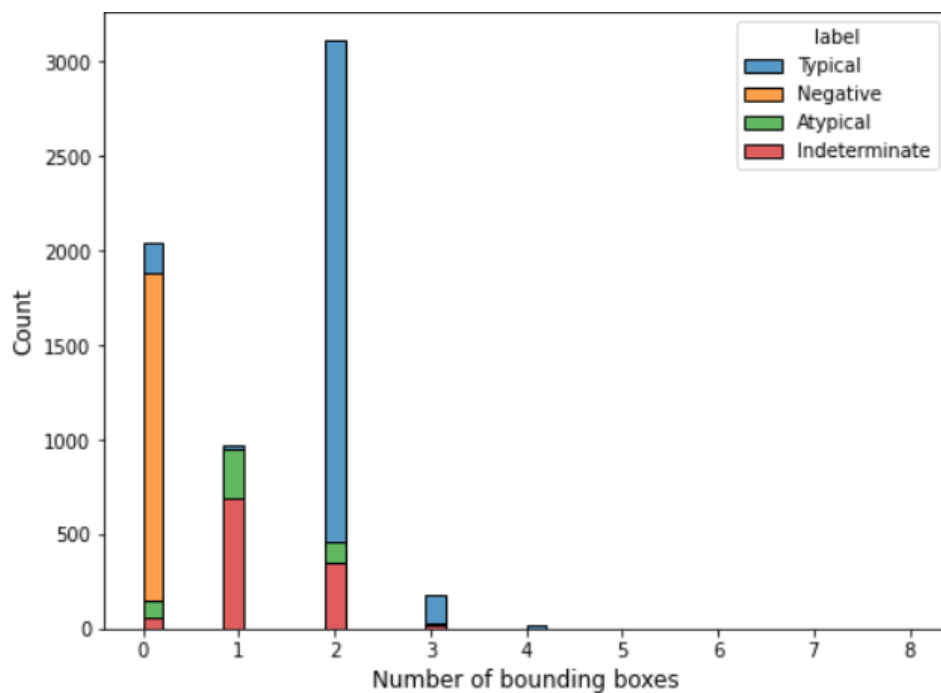


Figure 4.8 Count of train images with number of bounding boxes

Figure 4.9 indicates the overall bounding boxes annotations across the classification classes in the training dataset. Typical class images which are 3007 have in all 5868 bounding boxes labels, indeterminate class images which are 1108 have in all 1453 bounding boxes labels and atypical class images which are 483 have in all 537 bounding boxes labels. As expected, images with negative class are not labelled with any bounding boxes as they do not contain any lung opacities.

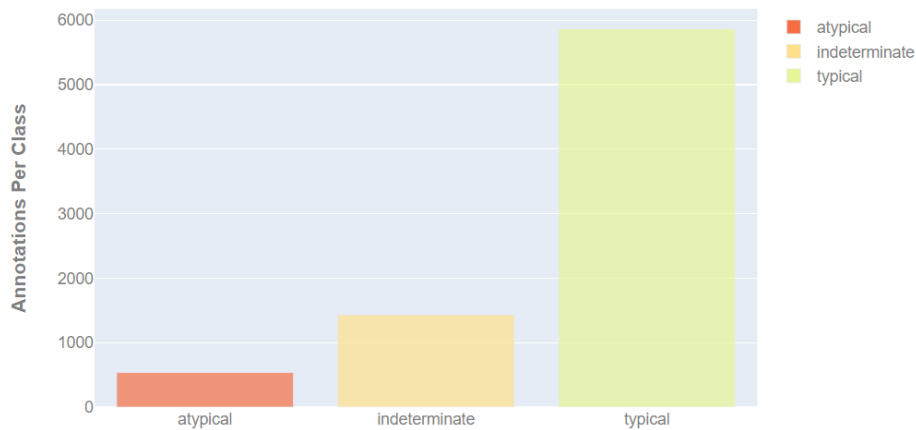


Figure 4.9 Distribution of total bounding boxes across the classification classes

Figure 4.10 reflects the box plots of bounding boxes dimensions categorized across the number of bounding boxes that are between one and four per image. It can be inferred that the images are labelled with both large as well as small boxes suggesting some inconsistency in labelling of opacity boxes. Figure 4.11 visualizes the CXRs with large boxes annotations whereas Figure 4.12 visualizes the CXRs with small boxes annotations.

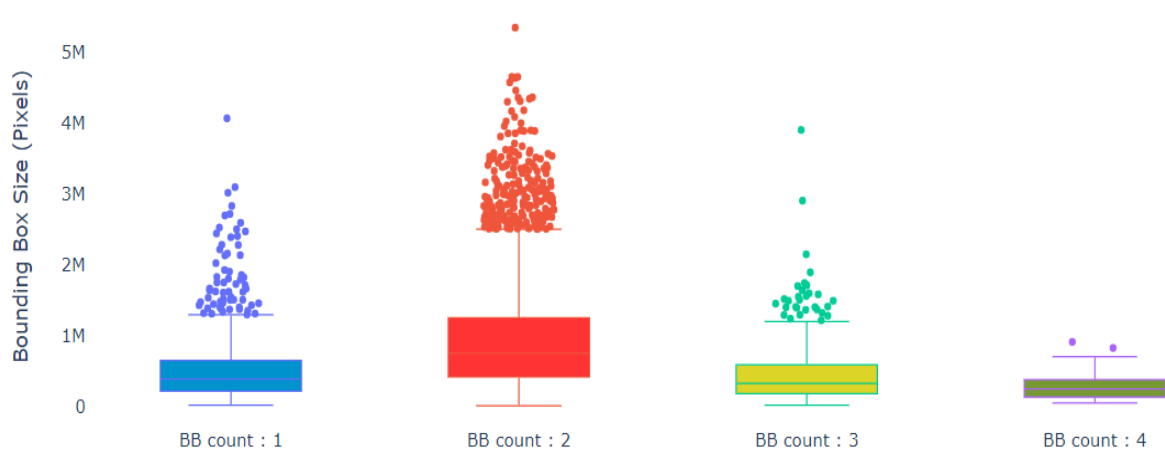


Figure 4.10 Box plot of bounding boxes sizes

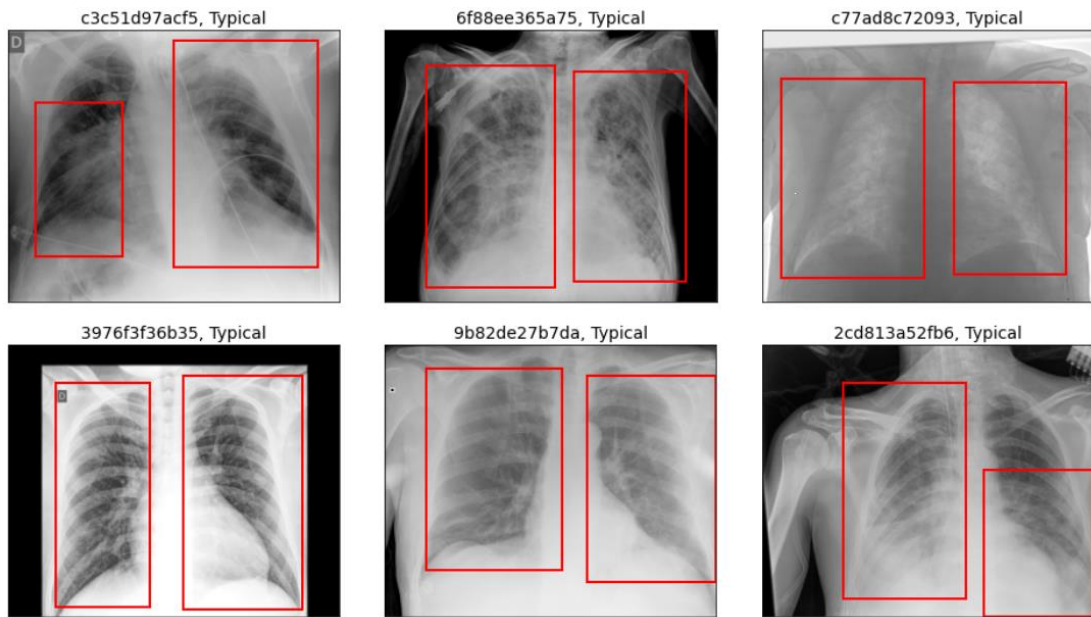


Figure 4.11 Images with large bounding boxes annotations

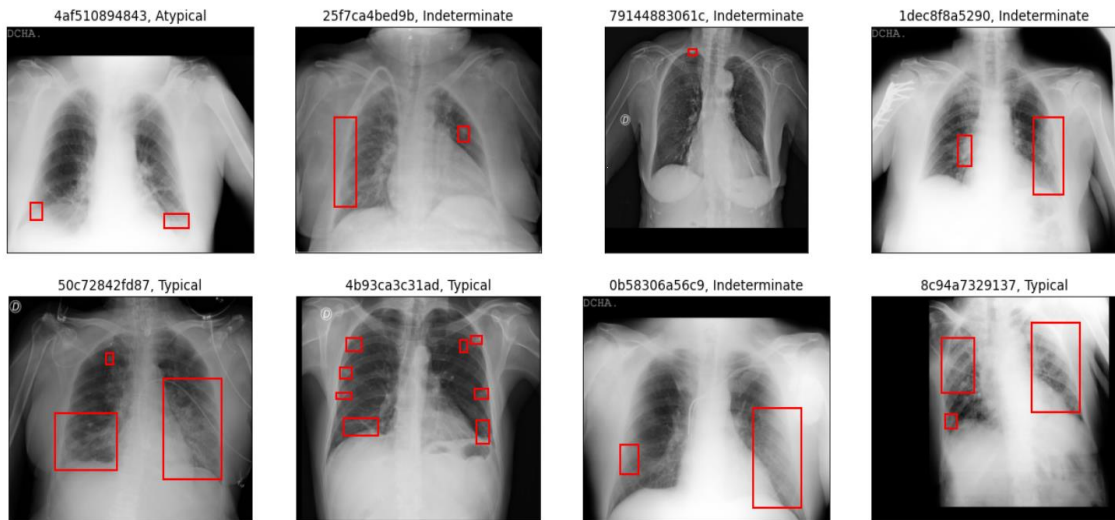


Figure 4.12 Images with small bounding boxes annotations

Figure 4.13 indicates the overall heatmap of bounding boxes labels across the training dataset. Maximum width of an opacity box is approx. 50% of the total image size and maximum height of an opacity box is approx. 80% of the total image size. Total number of bounding boxes instances across all the CXRs is approx. 8000.

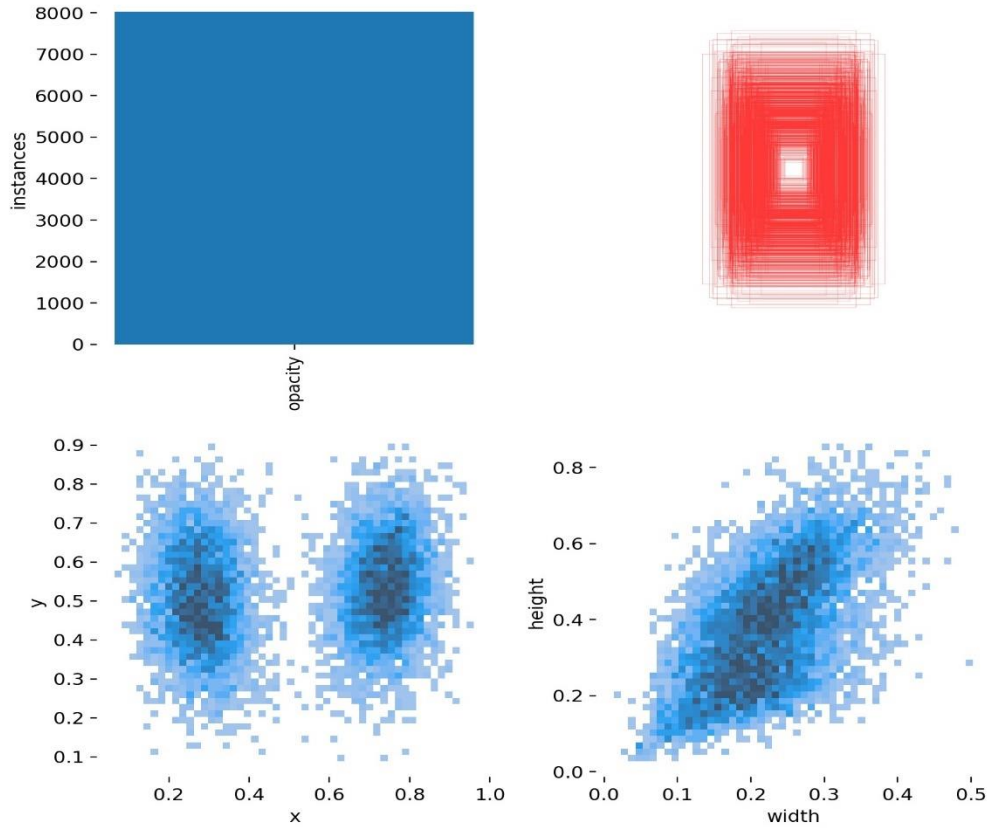


Figure 4.13 Bounding boxes heatmap

Training images are in DICOM medical imaging format and thus contain extra metadata regarding the images. Figure 4.14 count plot indicates the distribution of gender used for collection of chest X-ray images to build the training dataset. Number of male CXRs is 3564 whereas number of female CXRs is 2770. Figure 4.15 count plot indicates the distribution of modality used for collection of chest X-ray images to build the training dataset. Number of computed tomography (CT) CXRs is 3094 whereas number of Digital Radiography CXRs (DX) is 3240. DX images are generally of higher quality than CT images. Figure 4.16 count plot indicates the distribution of photometric interpretation used for collection of chest X-ray images to build the training dataset. Number of monochrome-1 CXRs is 1704 whereas number of monochrome-2 CXRs is 4630. If the photometric interpretation is monochrome-1 then CXRs become inverted after converting from DICOM to png or jpeg formats and hence need to be fixed after conversion. Information such as patient gender, modality and photometric interpretation are not used for model development but can be used as an interesting post data analysis step regarding the performance of models across these categories.

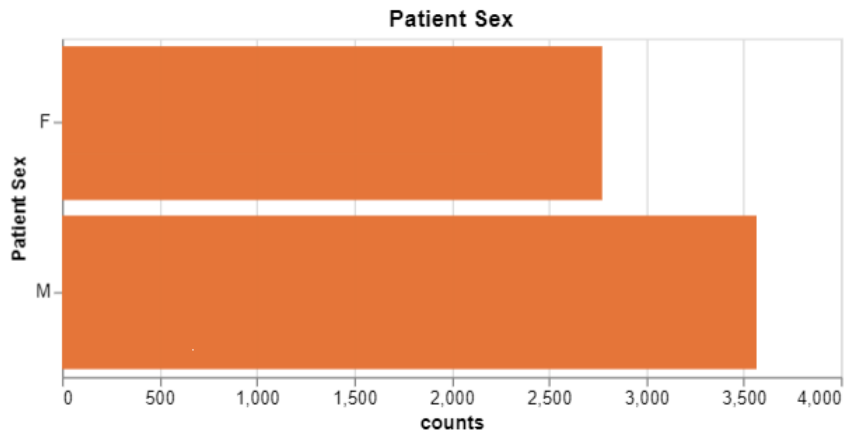


Figure 4.14 Sampling of training images across patient gender

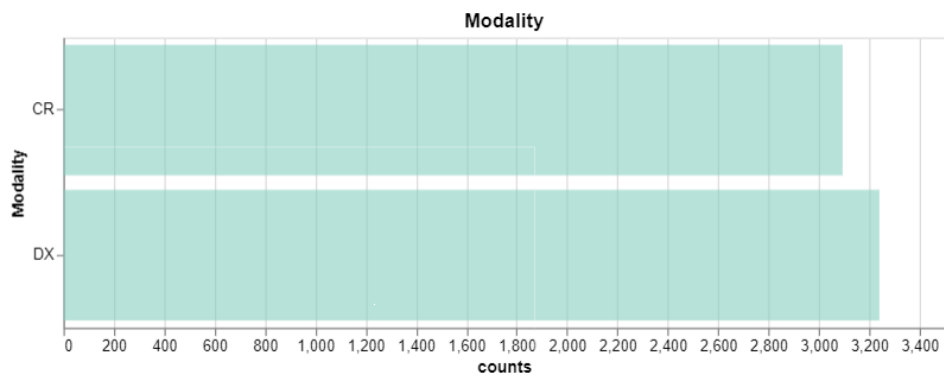


Figure 4.15 Sampling of training images across modality

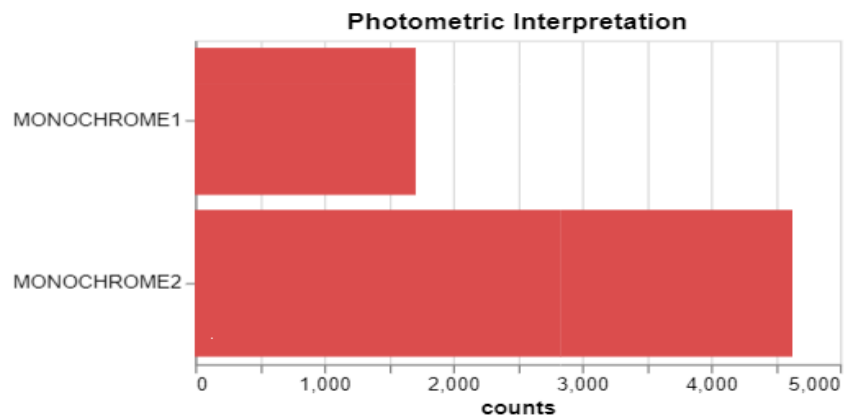


Figure 4.16 Sampling of training images across monochrome

4.3 Data Preparation and Cleaning

4.3.1 Converting DICOM to png images

In computer vision, medical databases are usually in the form of DICOM images. It is the case for the dataset used in this research too. DICOM images are read into numpy arrays using pydicom python library. Image arrays are rescaled in the range of 0 to 255 and converted to unsigned int-8 data type. Additionally, for DICOM images which have photometric interpretation as monochrome-1, inverted X-ray images are fixed. Image arrays are saved as png extension images of sizes 1024*1024 to form the training and test datasets which are used by augmentation and modelling pipelines. Further resizing of images is taken care by augmentation pipeline before feeding those images to model training and inference.

4.3.2 Removing duplicate images

Training dataset consists of 6334 images out of which 217 images are duplicate. The duplicate images belong to non-negative classes. Non-negative classes are expected to have bounding boxes information as prescribed by the dataset description so the images that are removed are the ones which are duplicate and belonging to same patient study identifier but have missing bounding boxes labels as indicated in Figure 4.17 for atypical, indeterminate, and typical classes. Following removal of duplicates, train dataset reduces to 6117 images which are finally used for development of the prediction models.

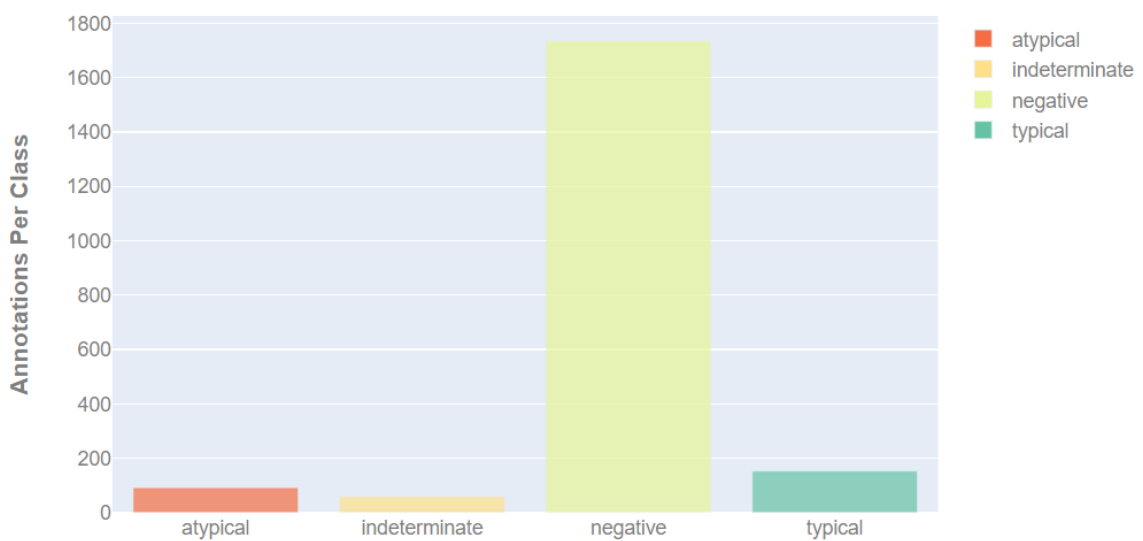


Figure 4.17 No bounding boxes across classification classes

4.3.3 Generation of segmentation masks from bounding boxes annotations

Segmentation masks (Figure 4.18) are generated for the training dataset from the bounding boxes labels. An auxiliary or segmentation head is trained along with classification models to predict these segmentation masks which helps in better generalization and performance of classification models to predict the overall class of an image. White pixels in the segmentation images indicate the opacity boxes of the training images. Normal class images which do not have any opacities contain all black pixels. Auxiliary head is trained on these generated segmentation mask images.

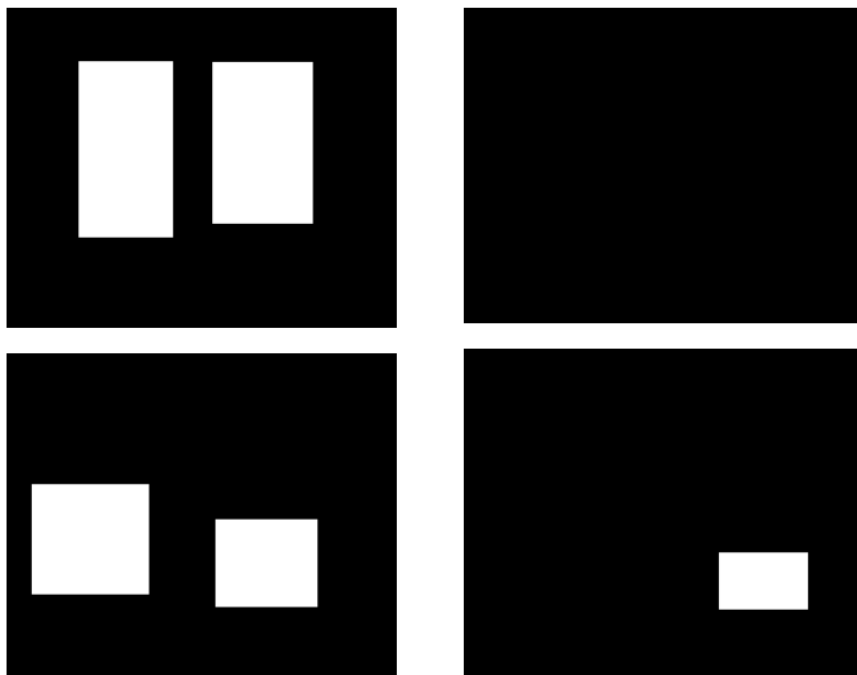


Figure 4.18 Segmentation Masks

4.3.4 Normalization of bounding boxes annotations

Bounding boxes labels are normalized so that training of detection models is transferable to any image resolutions. Training of EfficientDet detection models require normalized coordinates in the form of $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$. Training of YOLO detection models require normalized coordinates in the form of $[x_{\text{center}}, y_{\text{center}}, \text{width}, \text{height}]$. Bounding box coordinates for the x and y axis are divided by the width and the height of the images for normalization. As an example, coordinates of a bounding box $[98, 345, 420, 462]$ with image resolution of $[480, 640]$ become $[0.153125, 0.71875, 0.65625, 0.9625]$ in $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ normalized form .

4.4 Data Partitioning

Stratified 5-fold is applied on the dataset to split into train and validation datasets and preserve the percentage of classes across all the folds. Training dataset of 6117 images is divided into 5 non overlapping folds and each of the classification and detection architectures are trained 5 times and ensembled later for the final predictions on a test dataset. Number of training and validation images are indicated for each of the 5 folds in Table 4.1 after applying stratified k-fold and achieving same distribution ratio as in the full training dataset. Models are trained for 5 folds wherein for each fold, training images are the ones which don't belong to the fold and validation images are the ones which belong to the fold. Validation images act as out of fold unseen data for that fold trained model to evaluate and iteratively improve model performance before feeding the final trained models on an unseen private test dataset. As an example, when a model architecture is trained on fold-0 then the training images are the ones which are tagged for fold-1 to fold-4 and validation images are the ones which are tagged for fold-0. Similarly, when a model architecture is trained on fold-1 then the training images are the ones which are tagged for fold-0 and fold-2 to fold-4 and validation images are the ones which are tagged for fold-1. Stratified k-fold generally achieves better performance and less bias specially for imbalanced dataset than a simple train-validation split but at the cost of fitting and evaluating more models.

Table 4.1 Data Partitioning – Stratified 5-fold

Class	Number of Training Images (fold != training fold)	Number of Validation Images (fold == training fold)
Negative	1391	345
Typical	2284	571
Indeterminate	839	210
Atypical	380	97
Total	4894	1223

4.5 Data Augmentation Pipeline

Image augmentation techniques help in controlling overfitting of deep models by applying random transformations on the train dataset and creating new training samples from the existing ones. Augmentation pipeline includes the following transformations:

- Shift Rotate Scale – Randomly translate, scale, and rotate the input image as depicted in Figure 4.19.



Figure 4.19 Augmentation - Shift Rotate Scale

- Horizontal flip – Flips the input horizontally along the y axis as depicted in Figure 4.20.



Figure 4.20 Augmentation – Horizontal Flip

- Random Brightness Contrast - Randomly change brightness and contrast of the input image as depicted in Figure 4.21.
- One of Blur, Gaussian Blur or Median Blur - Blur the input image using a random size kernel or a Gaussian filter or a median filter.
- One of emboss or sharpen – Sharpen or Emboss the input image and overlays the result with the original image.



Figure 4.21 Augmentation – Random Brightness Contrast

- Coarse Dropout – Apply Coarse Dropout on the rectangular regions in the image as depicted in Figure 4.22.



Figure 4.22 Augmentation – Coarse Dropout

- Random Resized Crop - Crop a random part of the input and rescale it.

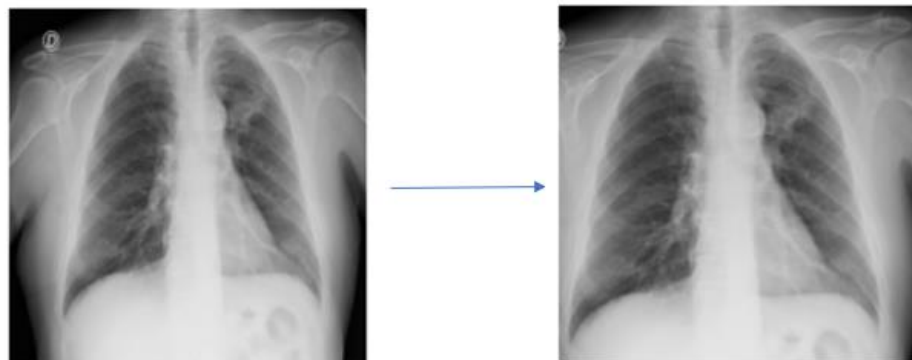


Figure 4.23 Augmentation – Random Resized Crop

- Resize images to desired input sizes for the input layer of the models.
- Normalize images in the range of 0 to 1.

4.6 Image Classification Models Implementation

EfficientNet-v2m and EfficientNet-B5 state of art architectures are trained on the dataset to predict the overall classification class among the 4 classes that are either negative, typical, indeterminate, or atypical.

4.6.1 EfficientNet-v2m

EfficientNet (Tan and Le, 2021) architecture is explored in detail in literature review section 2.3.4. The family of models achieved approx. 4-7% higher accuracy than existing ConvNets such as ResNets or DenseNets and with reduced number of training parameters and faster inference speed. EfficientNets transfer well to new domains. EfficientNet-v2m pretrained model weights on ImageNet are retrained for 20 epochs to fit on the training dataset and trained 5 times based upon the stratified k-fold partitions.

EfficientNet-v2m training configuration:

- Image size: 512 x 512
- Batch size: 16
- Optimizer: AdamW
- Initial Learning rate: 0.0003
- Scheduler: Cosine
- Loss function: Binary Cross Entropy with Logits Loss and class weights
- Epochs: 20
- K folds: 5
- Train images per fold: 4902
- Valid images per fold: 1225
- Training time per fold: 2 hrs 20 mins
- Total training time: ~12 hrs
- Param count: 61.6 M
- Param size: 246 MB
- GPU: Tesla P100-PCIE-16GB

- ATTENTION MAP: Dual Attention with Global Avg. pooling
- SEGMENTATION HEAD: 128 filters Conv architecture
- RELU activation replaced with MISH activation

4.6.1.1 Segmentation Head

An auxiliary or segmentation head is forked out of the 3rd block of EfficientNetv2m 6 blocks as shown in Figure 4.24 and a couple of convolution layers with 128 filters are added to predict the segmentation masks during training. Some of the mid-level features which are already learnt by the EfficientNet architecture are reused by the segmentation head. Such a concept of auxiliary head was first introduced in GoogleNet (Szegedy et al., 2015) and the primary reason behind it is that a secondary or auxiliary loss helps improve the performance and generalization of primary classification task. An auxiliary head also improves the convergence of deeper networks by combatting the vanishing gradient problem.

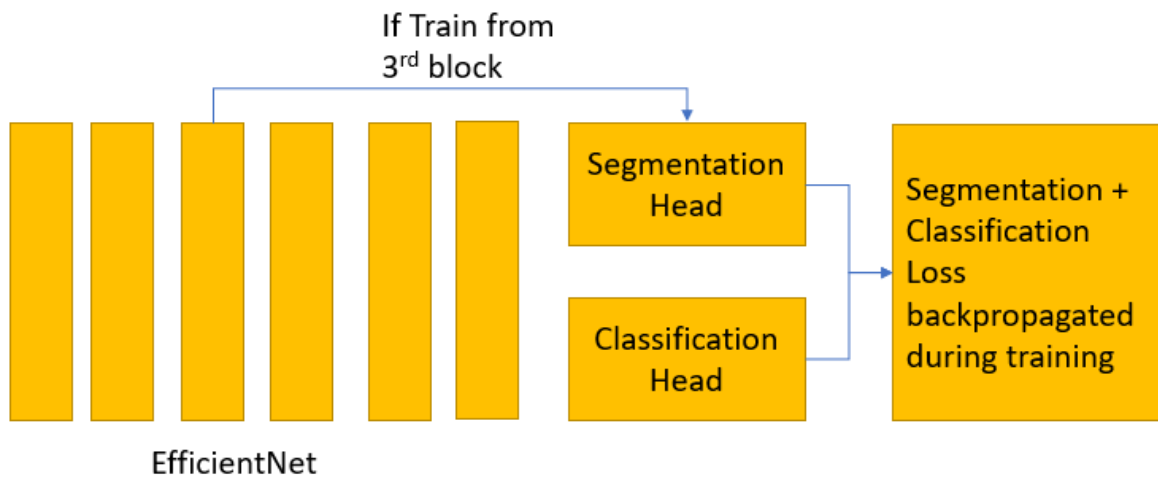


Figure 4.24 Segmentation Head during training of classification models

Table 4.2 and Table 4.3 highlight the mAP, AUC-ROC and Accuracy scores achieved on out of fold validation datasets across all the folds. Average mAP score is 0.565 for models trained with segmentation head whereas average mAP score is 0.55 for models trained without segmentation head. Average AUC-ROC score is 0.813 for models trained with segmentation head whereas average AUC-ROC score is 0.803 for models trained without segmentation head. Based upon the improved results on out of fold validation dataset, segmentation auxiliary head is used by default in further classification models.

Table 4.2 EfficientNetv2m mAP comparison with and without segmentation head

Metric	Fold	Value without segmentation head	Value with segmentation head
mAP	0	0.549	0.561
	1	0.548	0.572
	2	0.549	0.546
	3	0.57	0.57
	4	0.539	0.572
Avg mAP of all folds		0.55	0.565 (+1.5%)

Table 4.3 EfficientNetv2m AUC and Accuracy comparison with and without segmentation head

Metric	Fold	Value without segmentation head	Value with segmentation head
AUC-ROC	0	0.798	0.805
	1	0.809	0.828
	2	0.795	0.797
	3	0.822	0.822
	4	0.803	0.814
Avg AUC-ROC of all folds		0.803	0.813 (+1%)
Accuracy	0	0.65	0.66
	1	0.65	0.68
	2	0.66	0.66
	3	0.68	0.68
	4	0.65	0.66
Avg Accuracy of all folds		0.658	0.668 (+1%)

4.6.1.2 MISH activation

RELU activation is replaced by MISH activation (Misra, 2019) in EfficientNetv2m architecture. Mish activation improved top-1 accuracy of ResNet50 architecture on ImageNet by approx. 1% keeping all other network parameters constant. Similar experiments have proven successful in other CNN architectures too. With respect to average mAP metric across all the validation folds of the dataset used in this research, results with and without MISH activation in EfficientNetv2m are almost similar.

4.6.1.3 Attention layer

EfficientNet backbone is appended with a self-attention layer (Wang and Xia, 2018) to better learn spatial and channel interdependencies followed by global average pooling of the feature maps from the attention layer. Attention mechanism is inspired from transformer architecture which learns better embeddings using nearby embeddings. Similarly, better feature maps can be learnt for each of the classes by self-attention mechanism. Output of attention layer is finally connected to output layer of 4 neurons to calculate the logits and SoftMax probabilities corresponding to the 4 classes.

4.6.1.4 Class weights

Passing class weights to loss function enable the models to pay more attention to the under-represented classes. Indeterminate and atypical classes are the less represented classes in the training dataset. Ratio of negative: typical: indeterminate: atypical images in the training dataset is approximately 0.28:0.47:0.17:0.08. Models are trained by passing more weightage to indeterminate and atypical classes and this helped to achieve higher F1 scores on out of fold validation datasets as indicated in Table 4.4 for folds 0, 1 and 2. Other metrics such as mAP, AUC-ROC and accuracy metrics are indicated in Table 4.5 and Table 4.6. Overall mAP and accuracy scores further improved by approx. 1% when class weights are used in loss function along with training for segmentation loss.

Table 4.4 EfficientNetv2m F1 scores on validation dataset with and without class weights

Metric	Fold	Class	Value without class weights	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)
F1 score	0	Negative	0.73	0.73	0.72
		Typical	0.77	0.77	0.76
		Indeterminate	0.20	0.21	0.21
		Atypical	0.21	0.31	0.28
F1 score	1	Negative	0.75	0.76	0.77
		Typical	0.79	0.79	0.79
		Indeterminate	0.19	0.24	0.24
		Atypical	0.26	0.38	0.42
F1 score	2	Negative	0.73	0.74	0.74
		Typical	0.78	0.78	0.79
		Indeterminate	0.20	0.23	0.21
		Atypical	0.25	0.32	0.29

Table 4.5 EfficientNetv2m mAP scores on validation dataset with and without class weights

Metric	Fold	Value without class weights	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)	Best Epoch
mAP	0	0.561	0.557	0.567	14
	1	0.572	0.594	0.59	18
	2	0.546	0.558	0.555	14
	3	0.570	0.582	0.581	12
	4	0.572	0.577	0.581	15
Avg mAP		0.565	0.573 (+0.8%)	0.575 (+1%)	

Table 4.6 EfficientNetv2m AUC/Accuracy on validation data with and without class weights

Metric	Fold	Value without class weights	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)
AUC-ROC	0	0.805	0.805	0.806
	1	0.828	0.83	0.832
	2	0.797	0.80	0.802
	3	0.822	0.828	0.822
	4	0.814	0.82	0.82
Avg AUC-ROC		0.813	0.816 (+0.3%)	0.816 (+0.3%)
Accuracy	0	0.66	0.66	0.66
	1	0.68	0.68	0.685
	2	0.66	0.68	0.68
	3	0.68	0.68	0.69
	4	0.66	0.67	0.67
Avg Accuracy		0.668	0.674 (+0.6%)	0.677 (+0.9%)

4.6.1.5 Training and Validation Loss

Figure 4.25 and Figure 4.26 highlight the training and validation loss of EfficientNet-v2m using segmentation head, MISH activation, attention layers and class weights across 20 epochs for a couple of folds that are fold-0 and fold-3. It is important to observe the increasing mAP metric on the validation dataset as the number of training epochs increases. Best checkpoints of the model are saved based upon the highest mAP score achieved on the validation dataset and these checkpoints will be finally used for evaluation on an unseen private test dataset. Training loss is not monotonically decreasing because segmentation loss is backpropagated along with the classification loss during training of the model. Lower batch size of 16 and cosine scheduler which varies the learning rate across the epochs also have potential impact on the loss leading to a wavy plot. Learning rate decay across the epochs is plotted in Figure 4.27.

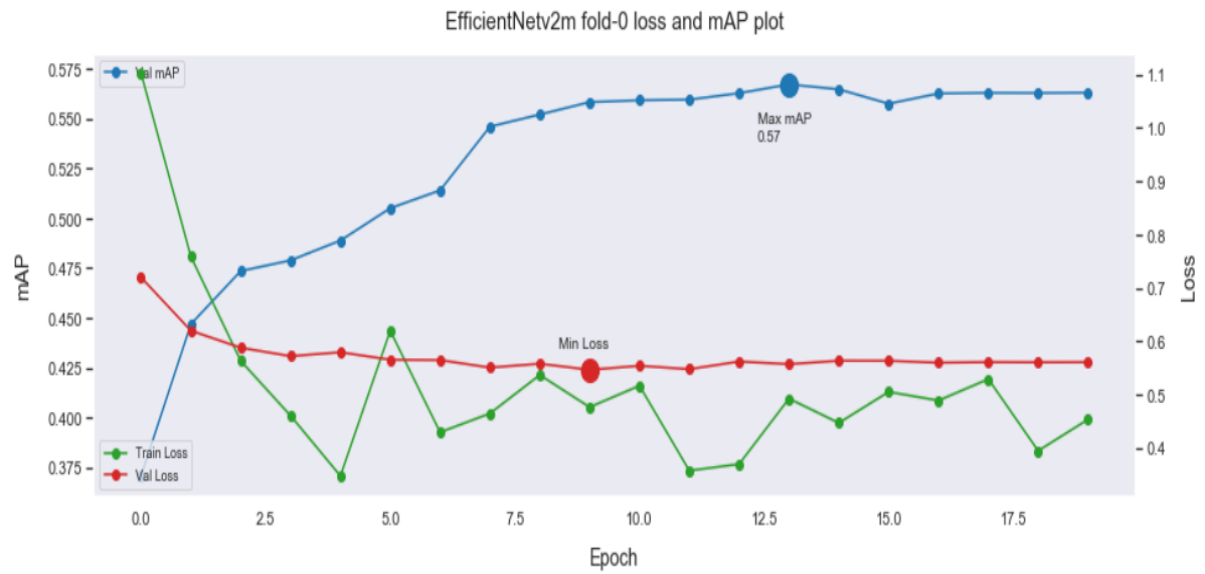


Figure 4.25 EfficientNetv2m Training/Validation loss and Validation mAP across epochs-fold0

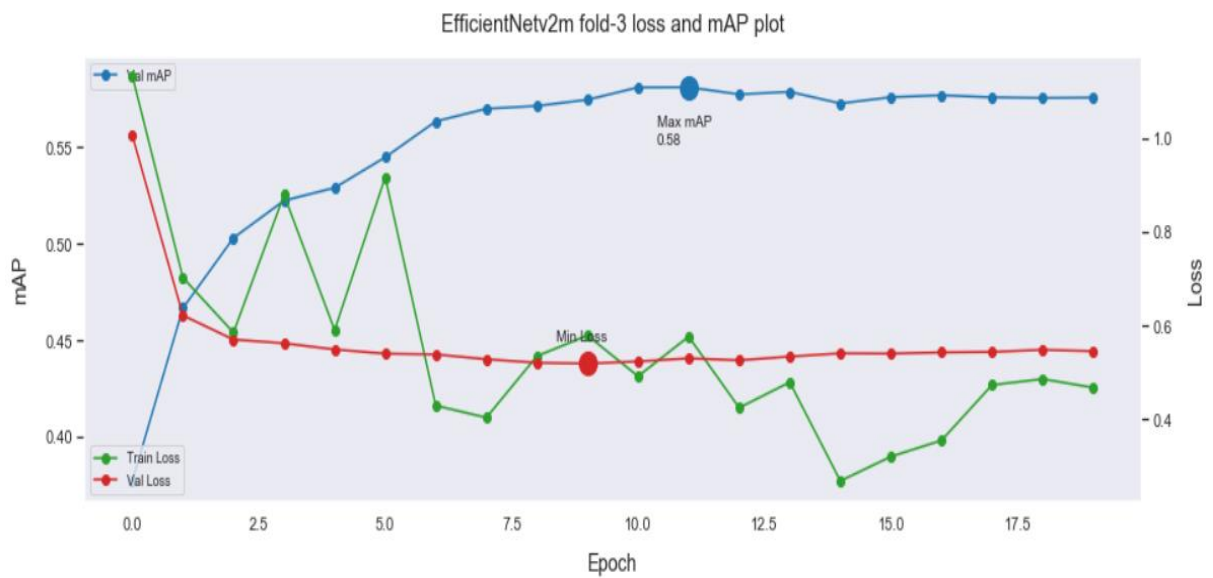


Figure 4.26 EfficientNetv2m Training/Validation loss and Validation mAP across epochs-fold3

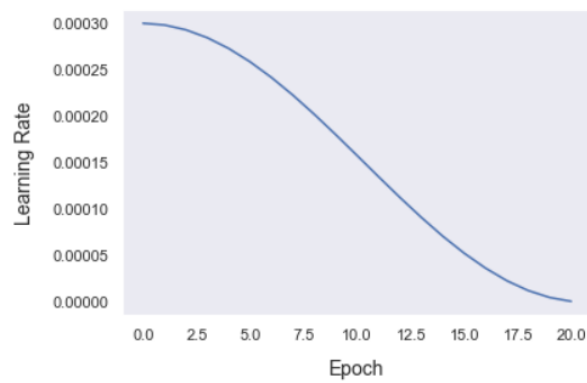


Figure 4.27 EfficientNet-v2m Learning Rate per epoch

4.6.1.6 Full Training Pipeline

Figure 4.28 summarizes the full training pipeline for classification task starting from pre-processing of DICOM images to png images of size 1024x1024. Stratified 5-fold is applied to split into training and validation datasets and models are trained 5 times. Augmentation pipeline transforms the images and resizes and normalizes the transformed images before feeding them to the input layer of the network. At the end of each epoch, validation loss and metrics are calculated on the validation images, and best checkpoint trained models are saved.

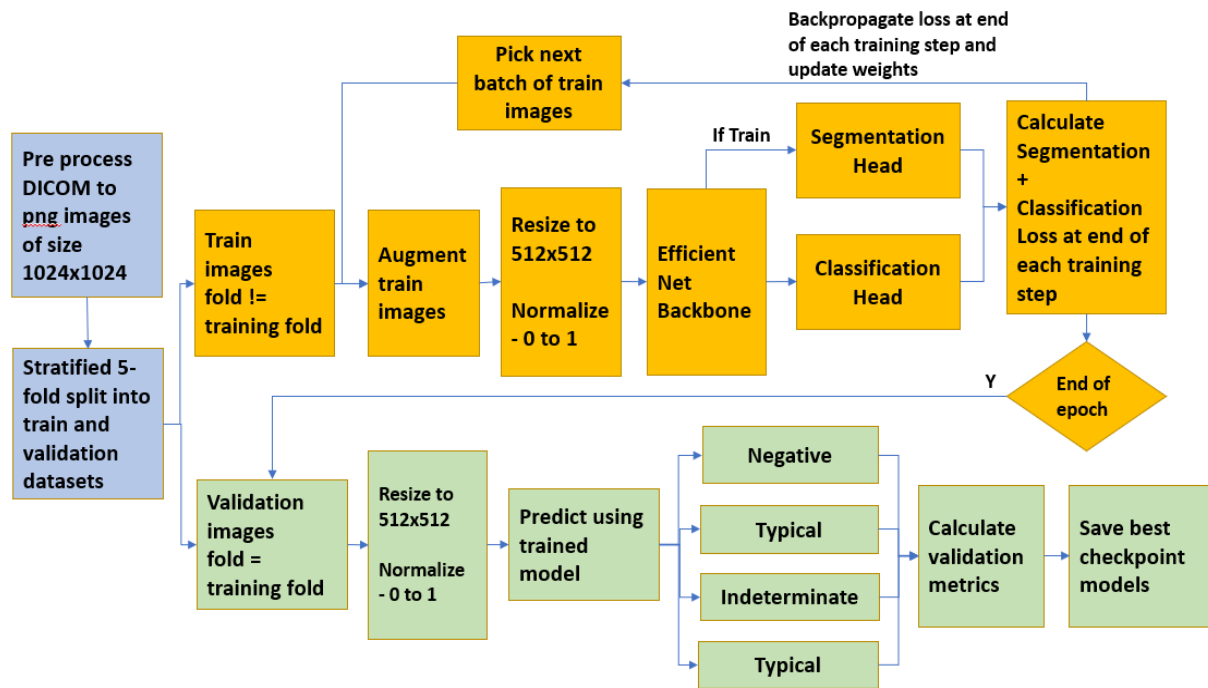


Figure 4.28 EfficientNet Full Training Pipeline

4.6.1.7 Other experiments

EfficientNet-v2m using class weights and segmentation head is also trained for 5 folds for image sizes 480*480 which is the recommended image size for this variant of EfficientNet. Training on different image sizes can add more diversity when ensembling and combining multiple models for predictions on a test dataset.

4.6.2 EfficientNet-v1 B5

EfficientNet-v1 B5 (Tan and Le, 2019) pretrained model weights on ImageNet are retrained for 15 epochs to fit on the dataset and trained 5 times based upon the stratified k-fold partitions. Based upon the experiments conducted for EfficientNet-v2m, segmentation head is added by default to train along with primary classification task. Loss function includes class weights to handle class imbalance.

EfficientNet-v1 B5 training configuration:

- Image size: 512 x 512
- Batch size: 16
- Optimizer: AdamW
- Initial Learning rate: 0.0003
- Scheduler: Cosine
- Loss function: Binary Cross Entropy with Logits Loss and class weights
- Epochs: 15
- K folds: 5
- Train images per fold: 4902
- Valid images per fold: 1225
- Training time per fold: 2 hrs per fold
- Total training time: 10 hrs
- Param count: 47.8M
- Param size: 191 MB
- ATTENTION MAP: Dual Attention with Global Avg. pooling
- SEGMENTATION HEAD: 128 filters Conv architecture
- GPU: Tesla P100-PCIE-16GB

4.6.2.1 Validation Dataset Metrics

Table 4.7 highlights the best mAP scores achieved on out of fold validation datasets using class weights in the loss function. Best checkpoints are saved which will be used for inferencing on a private test dataset. A couple of varying class weights experiments are attempted for EfficientNet-v1 B5. Table 4.8 and Table 4.9 highlight the best AUC and accuracy scores respectively using class weights. Table 4.10 reflects the best F1 scores using class weights. Note that segmentation or auxiliary head is by default used in all these experiments and results.

Table 4.7 EfficientNet B5 mAP scores on validation dataset with class weights

Metric	Fold	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)	Best Epoch
mAP	0	0.554	0.556	11
	1	0.587	0.587	9
	2	0.564	0.564	10
	3	0.588	0.576	9
	4	0.591	0.592	10
Avg mAP of all folds		0.576	0.575	

Table 4.8 EfficientNet B5 AUC scores on validation dataset with class weights

Metric	Fold	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)
AUC-ROC	0	0.806	0.802
	1	0.832	0.830
	2	0.805	0.802
	3	0.821	0.816
	4	0.821	0.825
Avg AUC-ROC of all folds		0.817	0.815

Table 4.9 EfficientNet B5 Accuracy scores on validation dataset with class weights

Metric	Fold	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)
Accuracy	0	0.65	0.66
	1	0.68	0.68
	2	0.67	0.66
	3	0.68	0.68
	4	0.67	0.68
Avg Accuracy of all folds		0.67	0.672

Table 4.10 EfficientNet B5 F1 scores on validation dataset with class weights

Metric	Fold	Class	Value with class weights (1.0, 1.0, 1.5, 2.0)	Value with class weights (1.0, 1.0, 2.0, 2.0)
F1 score	0	Negative	0.73	0.73
		Typical	0.77	0.77
		Indeterminate	0.15	0.18
		Atypical	0.27	0.31
F1 score	1	Negative	0.77	0.77
		Typical	0.78	0.78
		Indeterminate	0.17	0.15
		Atypical	0.37	0.31
F1 score	2	Negative	0.74	0.73
		Typical	0.78	0.77
		Indeterminate	0.23	0.17
		Atypical	0.32	0.31

4.6.2.2 Training and Validation Loss

Figure 4.29 and Figure 4.30 highlight the training and validation loss of EfficientNet-v1 B5 using segmentation head and class weights across 15 epochs for a couple of folds that are fold-0 and fold-1. It is important to observe the increasing mAP metric on the validation dataset as the number of training epochs increases. Best checkpoints of the model are saved based upon the highest mAP score achieved on the validation datasets and these checkpoints will be finally used for evaluation on a test dataset.

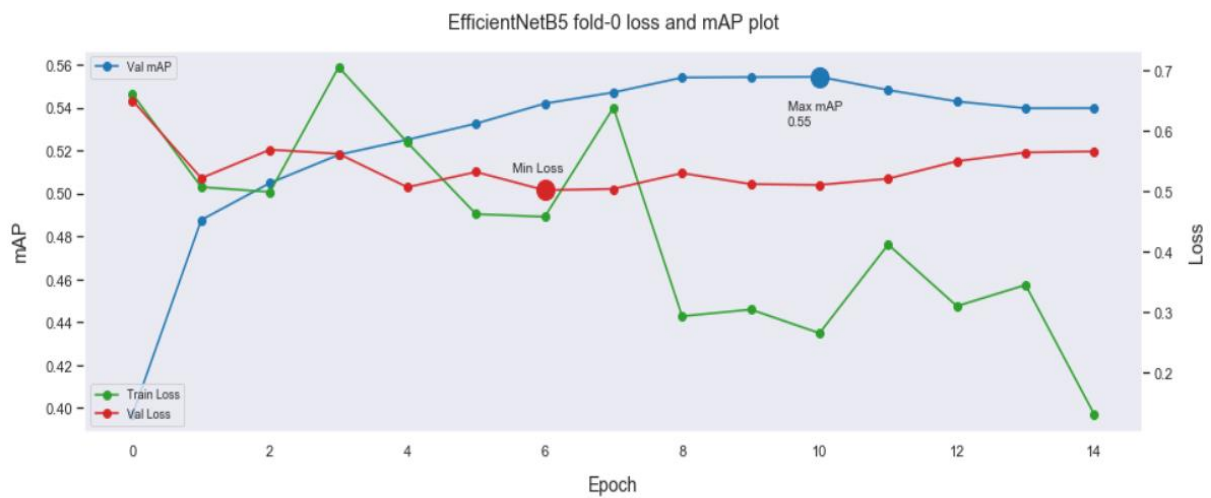


Figure 4.29 EfficientNet-B5 Training/Validation loss and Validation mAP across epochs-fold0

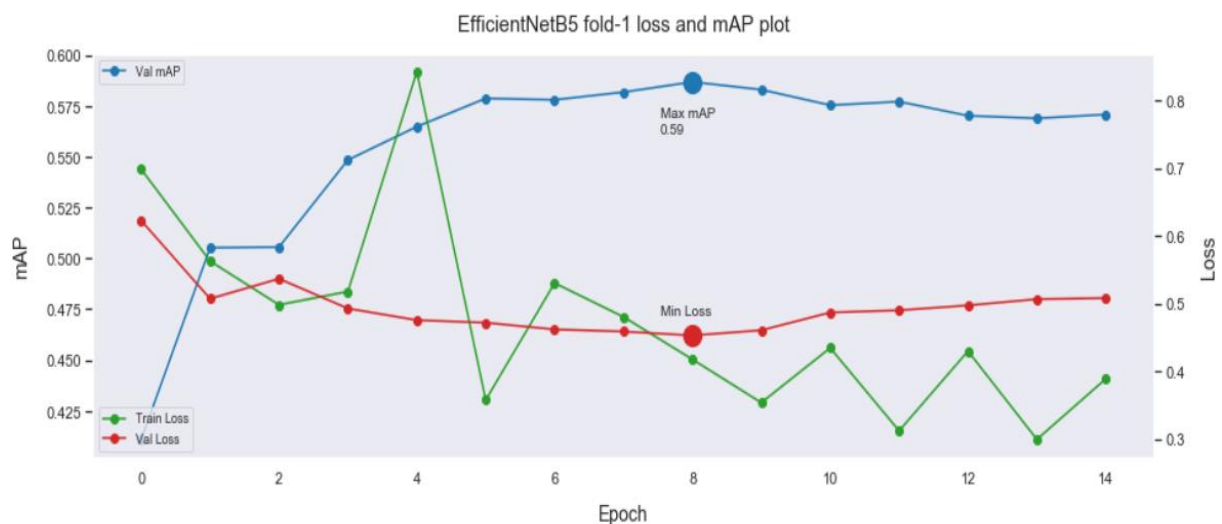


Figure 4.30 EfficientNet-B5 Training/Validation loss and Validation mAP across epochs-fold4

4.6.2.3 Other experiments

EfficientNet-B5 using class weights and segmentation head is also trained for 5 folds for image sizes 456*456 which is the recommended image size for this variant of EfficientNet. Training on different image sizes can add more diversity when ensembling and combining multiple models for predictions on a test dataset. Regarding updating activations, Relu activations are not replaced with Mish activations in EfficientNet-B5 but can be done as a future experiment.

4.6.3 ResNet50 and DenseNet

ResNet50 (He et al., 2016) and DensenNet201 (Huang et al., 2017) pretrained models on ImageNet are also attempted for learning classification classes on the dataset but both achieved lower results than EfficientNet models as indicated in Table 4.11. EfficientNetv2m achieves an average mAP and AUC-ROC score of 57.5% and 81.6% respectively on out of fold validation dataset. ResNet50 with approximately 42.4M training parameters achieves an average mAP and AUC score of 51.7% and 78.4% respectively on out of fold validation dataset. DenseNet201 with approximately 34.7M training parameters achieves an average mAP and AUC score of 55.2% and 80.1% respectively on out of fold validation dataset. Results of ResNet and DenseNet architectures seems to fall in line with the standard ImageNet results where both architectures stand approx. 6% lower in top-1 accuracy results as compared to EfficientNet architectures.

Table 4.11 ResNet50 and DenseNet201 results on validation dataset

Metric	EfficientNetv2m	ResNet50	DenseNet201
Avg mAP of all folds	0.575	0.517	0.552
Avg AUC-ROC of all folds	0.816	0.784	0.801

4.6.4 EfficientNet-v2m for binary classification

An EfficientNet-v2m model is trained for binary classification task to predict negative class vs the rest. Segmentation head is not used to train this model, but class weight is passed in the ratio of 2:1 to support the negative class imbalance. EfficientNetv2m binary model for prediction of negative class achieves an average mAP and AUC-ROC score of 87.2% and 90.8%

respectively on out of fold validation dataset as indicated in Table 4.12. Best checkpoints of the model are saved based upon the highest mAP score achieved on the out of fold validation datasets. Table 4.13 highlights the confusion matrix and F1 scores achieved using this binary classification model for one of the folds. This model will be leveraged in post processing following detection model predictions on test dataset to refine the confidence scores of the bounding boxes predictions for opacity class and to add a confidence score of negative class indicating that no bounding box is present or predicted. It is interesting to observe that a binary classification model also has similar F1 score (73%) for negative class that a 4-class classification model attained for this class, so the usage of this model has a scope of pruning in future.

Table 4.12 Metrics of EfficientNet-v2m binary classification model to predict negative class vs rest on validation dataset

Metric	EfficientNetv2m binary classification model
Avg mAP of all folds	0.872
Avg AUC-ROC of all folds	0.908

Table 4.13 Confusion Matrix and F1 score of EfficientNet-v2m binary model (fold-0) on validation dataset

Actual/Predicted	Non-Negative class (classes with opacities)	Negative class (class with no opacity)	F1 score
Non-Negative class (classes with opacities)	774	103	0.89
Negative class (class with no opacity)	92	255	0.73

4.7 Object Detection Models Implementation

EfficientDet-D3, EfficientDet-D5, YOLO-v5l and YOLO-v5x state of art architectures are trained on the dataset to localize the opacities in the form of bounding boxes.

4.7.1 YOLO-v5l

YOLO-v5 (Jocher et al., 2021) are a family of object detection networks pretrained on COCO dataset and achieved state of art results. YOLO is one of the most famous object detection algorithms due to its speed and accuracy. It divides images into a grid system and each cell in the grid is responsible for detecting objects within itself. YOLO architecture is explored in detail in literature review section 2.4.2. YOLO-v5l is retrained for 20 epochs to fit on the training images labelled with bounding boxes and trained 5 times based upon the stratified k-fold partitions.

YOLO-v5l training configuration:

- Image Size: 640 x 640
- Batch size: 16
- warmup_epochs: 3.0
- iou_threshold: 0.2 # IoU training threshold
- translate: 0.1 # image translation (+/- fraction)
- scale: 0.5 # image scale (+/- gain)
- flipplr: 0.5 # image flip left-right (probability)
- mosaic: 1.0 # image mosaic (probability)
- number of classes: 1
- Class names: [opacity]
- Params: 46.5 M
- Training time per fold: 2.2 hrs
- Total training time: ~12 hrs
- GPU: Tesla P100-PCIE-16GB

4.7.1.1 Validation Dataset Metrics

Table 4.14 highlights the highest mAP@IOU-0.5 achieved by the trained YOLO-v5l models on the validation dataset across all the folds. mAP metric is explained in detail in literature review section 2.4.1 and is one of the most important metrics to evaluate the accuracy of object detection models. Figure 4.31 shows the improvement of mAP metrics across the training epochs.

Table 4.14 YOLO-v5l Highest mAP @IOU=0.5 on Validation Dataset

Metric	Fold	Value	Best Epoch
mAP@IOU=0.5	0	0.538	8
	1	0.56	18
	2	0.585	16
	3	0.551	10
	4	0.595	11
Avg mAP		0.566	

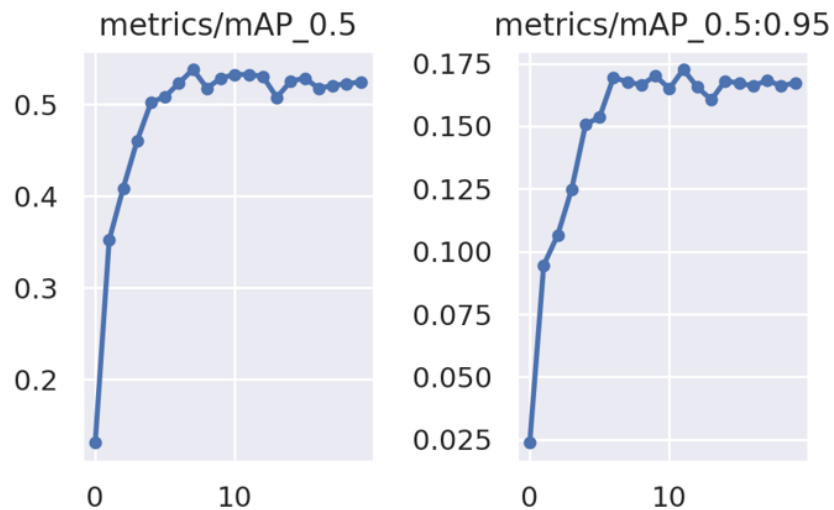


Figure 4.31 YOLO-v5l mAP metric across training epochs

4.7.1.2 Training and Validation Loss

Figure 4.32 shows the bounding box loss and object loss across the training epochs. Bounding box loss is the regression loss to predict the coordinates of the boxes whereas object loss indicates the improvement in the object presence likelihood score within a grid cell. YOLOs are single-stage object detectors, and all the combined losses are calculated in one shot at the output layer.

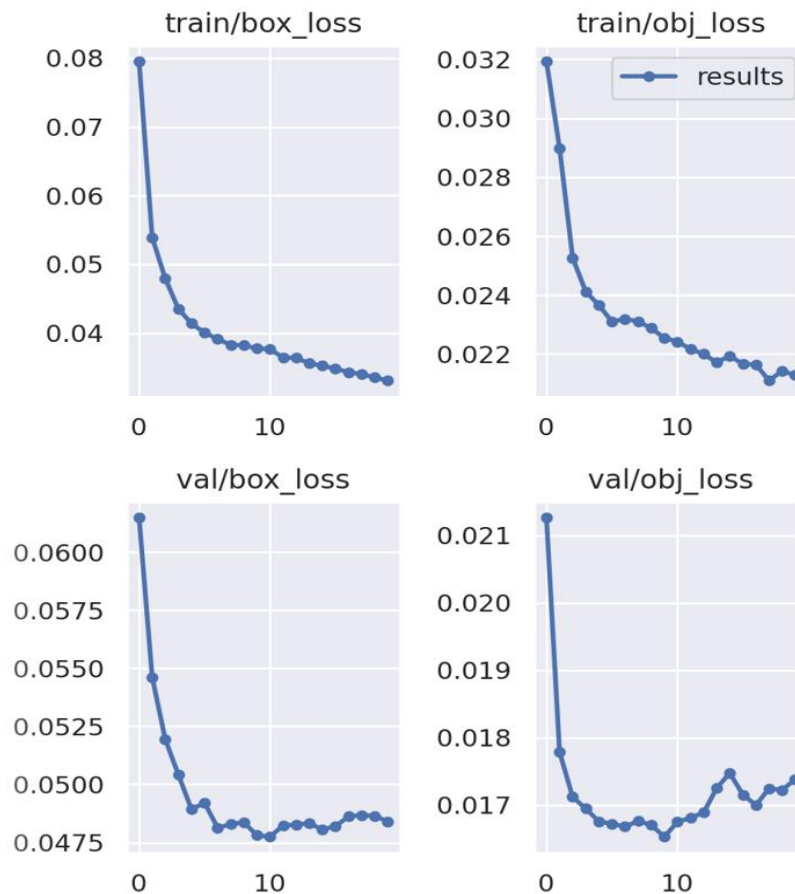


Figure 4.32 YOLO-v5l Box and Object loss across training epochs

4.7.1.3 Mosaic Augmentation

Mosaic augmentation is a key technique in training of YOLO. It combines 4 images into 4 tiles of random ratios as depicted in Figure 4.33. It helps the model learn to not overemphasize activations for large objects and try to pay equal attention to smaller objects. Dataset consists of both smaller and larger bounding boxes labelled as opacities as studied in EDA section 4.2.

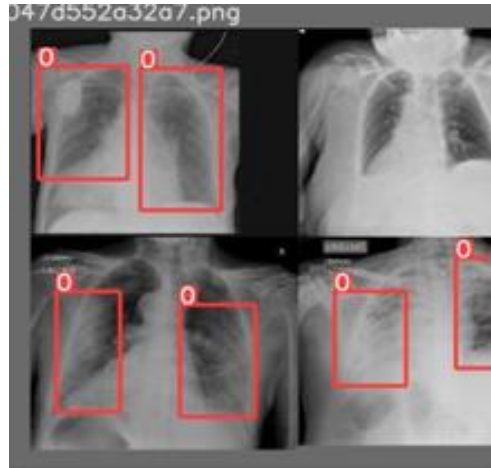


Figure 4.33 YOLO-Mosaic Augmentation

4.7.1.4 Full Training pipeline

Figure 4.34 summarizes the full training pipeline for detection task starting from pre-processing of DICOM images to png images of size 1024x1024. Stratified 5-fold is applied to split into training and validation datasets and models are trained 5 times. Augmentation pipeline transforms the images and resizes and normalizes transformed images before feeding them to the input layer of the network. At the end of each epoch, validation loss and metrics are calculated on the validation images, and best checkpoint trained models are saved.

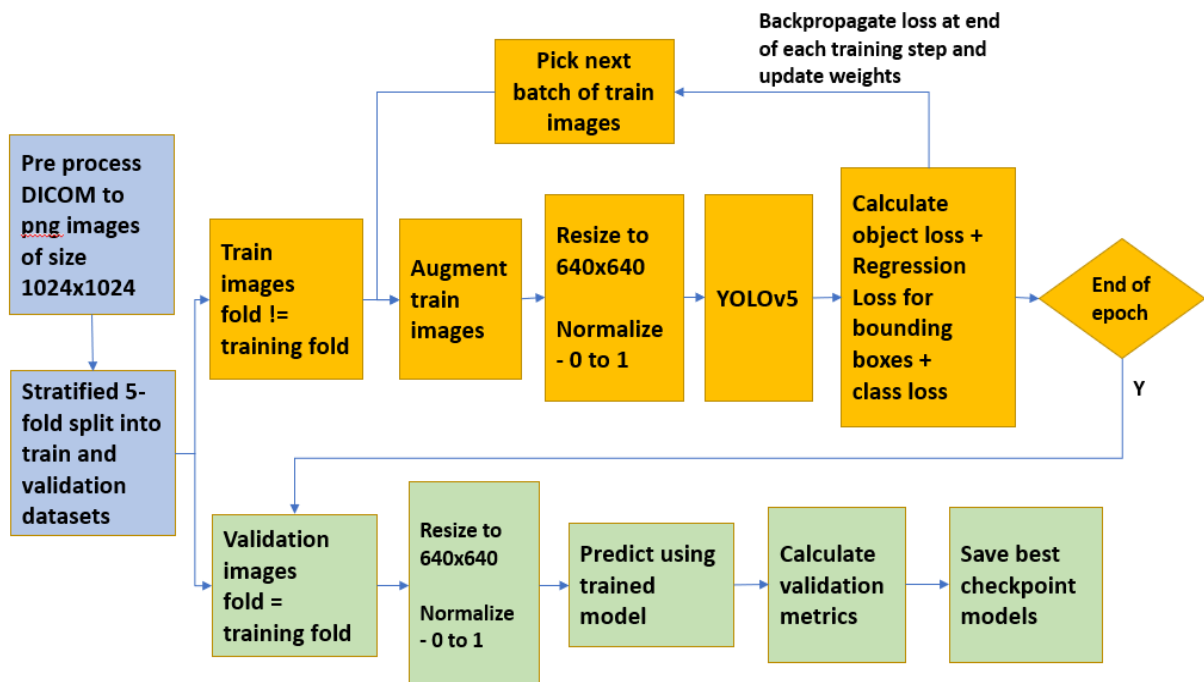


Figure 4.34 YOLO Full Training Pipeline

4.7.2 YOLO-v5x

YOLO-v5x (Jocher et al., 2021) pretrained model weights on COCO dataset are retrained for 15 epochs to fit on the training images labelled with bounding boxes and trained 5 times based upon the stratified k-fold partitions.

YOLO-v5x training configuration:

- Image Size: 640 x 640
- Batch size: 16
- Training epochs: 15
- warmup_epochs: 3.0
- iou_threshold: 0.2 # IoU training threshold
- translate: 0.1 # image translation (+/- fraction)
- scale: 0.5 # image scale (+/- gain)
- flip_lr: 0.5 # image flip left-right (probability)
- mosaic: 1.0 # image mosaic (probability)
- number of classes: 1
- Class names: [opacity]
- Params: 86.7 M
- Training time per fold: 2.2 hrs
- Total training time: ~12 hrs
- GPU: Tesla P100-PCIE-16GB

4.7.2.1 Validation Dataset Metrics

Table 4.15 highlights the highest mAP@IOU-0.5 achieved by the trained models on the validation dataset across all the folds. Figure 4.36 shows the improvement of mAP metrics across the training epochs.

Table 4.15 YOLO-v5x Highest mAP @IOU=0.5 on Validation Dataset

Metric	Fold	Value	Best Epoch
mAP@IOU=0.5	0	0.534	8
	1	0.55	8
	2	0.58	12
	3	0.542	9
	4	0.562	11
Avg mAP		0.554	

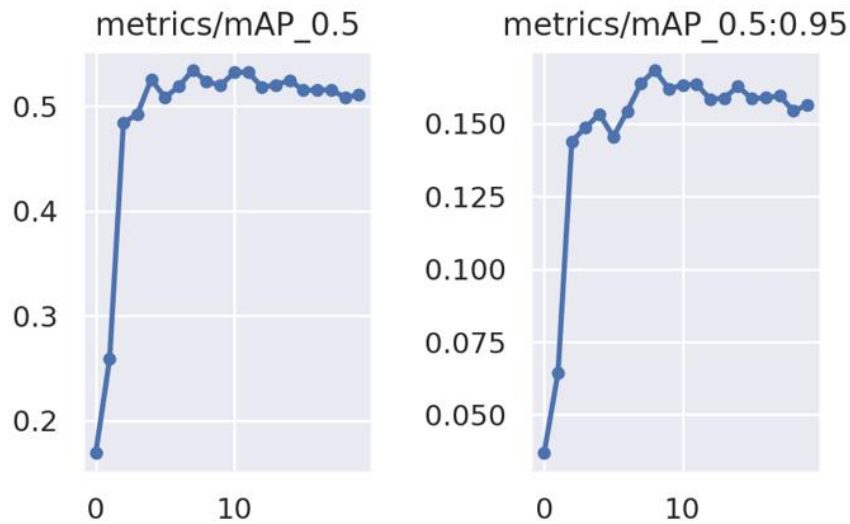


Figure 4.35 YOLO-v5x mAP metric across training epochs

4.7.2.2 Training and Validation Loss

Figure 4.36 shows the bounding box loss and object loss across the training epochs. YOLO-v5x due to higher number of parameters seems to start overfitting after approx. epoch-12 as can be observed in validation loss. Since we are monitoring the validation loss and metrics, corresponding best checkpoint models are saved which have lowest validation loss and are selected as the final models for predictions on a test dataset.

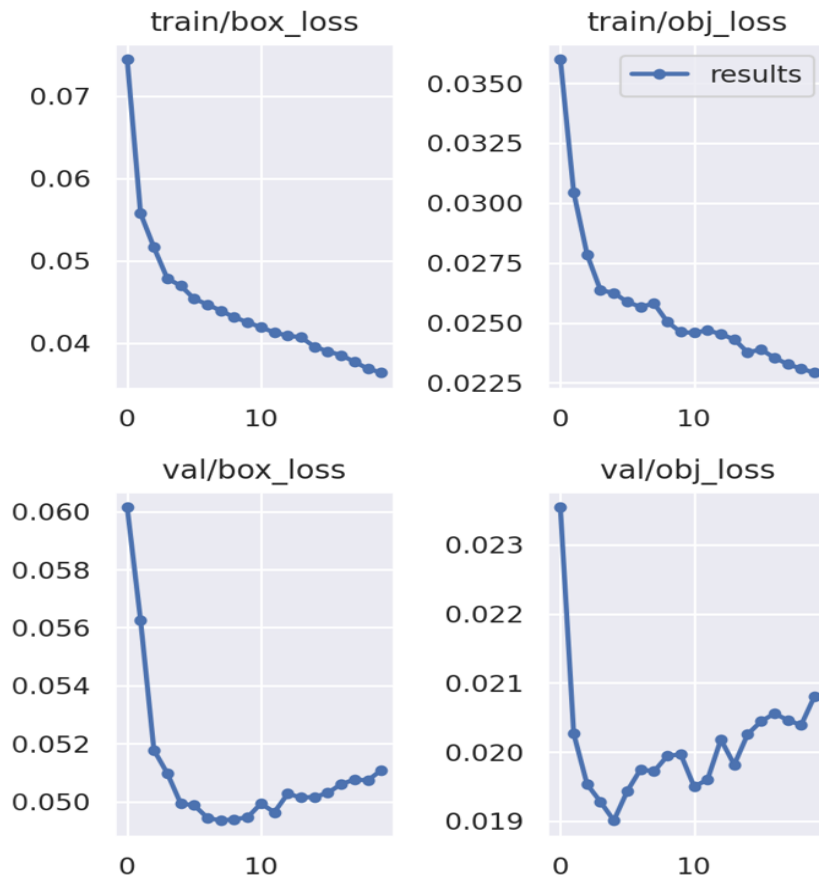


Figure 4.36 YOLO-v5x Box and Object loss across training epochs

4.7.3 EfficientDet-D3

EfficientDet (Tan et al., 2020) are a family of object detection networks that achieved state of art results on standard COCO dataset. EfficientDet architecture is explored in detail in literature review section 2.4.3. Backbone network of EfficientDet architecture is EfficientNet which feeds the extracted features to the bidirectional feature pyramid subnetwork. EfficientNets have performed well on the classification task so choosing EfficientDet for object detection task is a natural choice. EfficientDet-D3 pretrained model weights on COCO dataset are retrained for 20 epochs to fit on the training images labelled with bounding boxes and trained 5 times based upon the stratified k-fold partitions.

EfficientDet-D3 training configuration:

- Image Size: 640 x 640
- Initial Learning rate: 0.0005
- Batch size: 12

- Params: 11.9 M
- Training Images: 3431
- Validation Images: 857
- Training time per fold: 2 hrs
- Total training time: ~10 hrs
- GPU: Tesla P100-PCIE-16GB

4.7.3.1 Validation Dataset Metrics

Table 4.16 highlights the highest mAP@IOU-0.5 achieved by Efficient-D3 trained models on the validation dataset across all the folds.

Table 4.16 EfficientDet-D3 Highest mAP @IOU=0.5 on Validation Dataset

Metric	Fold	Value	Best Epoch
mAP@IOU=0.5	0	0.55	9
	1	0.54	12
	2	0.57	14
	3	0.54	11
	4	0.57	11
Avg mAP		0.554	

4.7.3.2 Training and Validation Loss

Figure 4.37 and Figure 4.38 shows the bounding box loss of both training and validation data across the training epochs. It also shows the improvement of mAP metric during training. Best checkpoint models are selected as final models for predictions on a private test dataset. Figure 4.39 depicts the decay in learning rate over the training epochs.

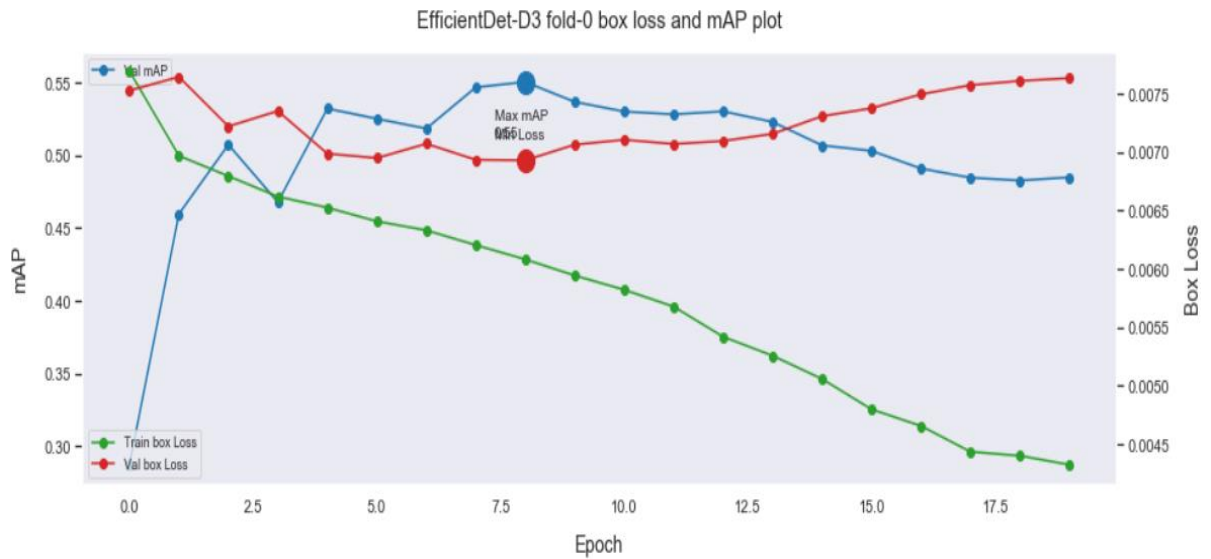


Figure 4.37 EfficientDet-D3 fold-0 Box loss and mAP plot across training epochs

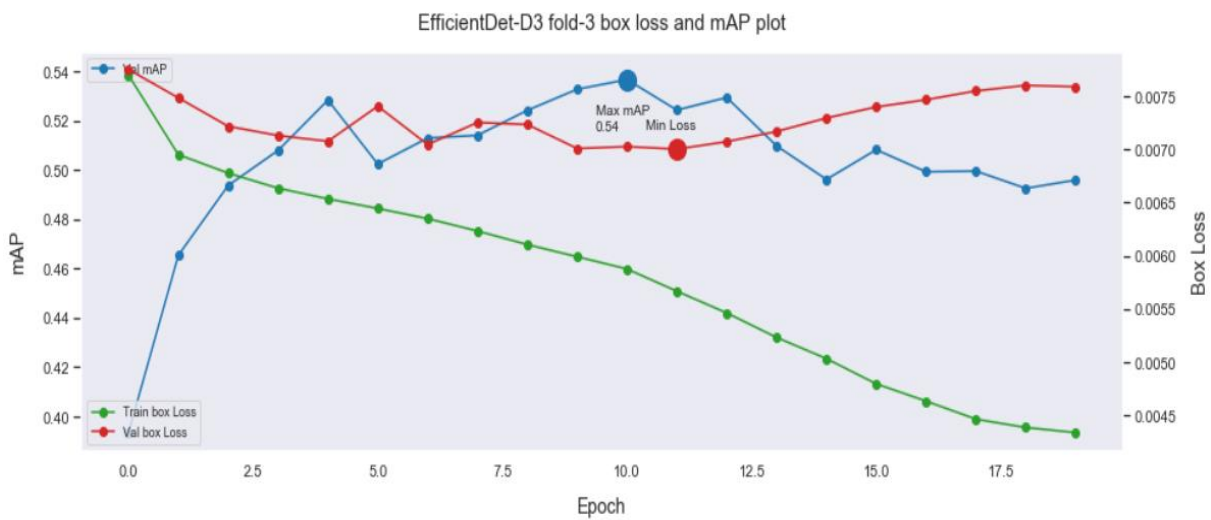


Figure 4.38 EfficientDet-D3 fold-3 Box loss and mAP plot across training epochs

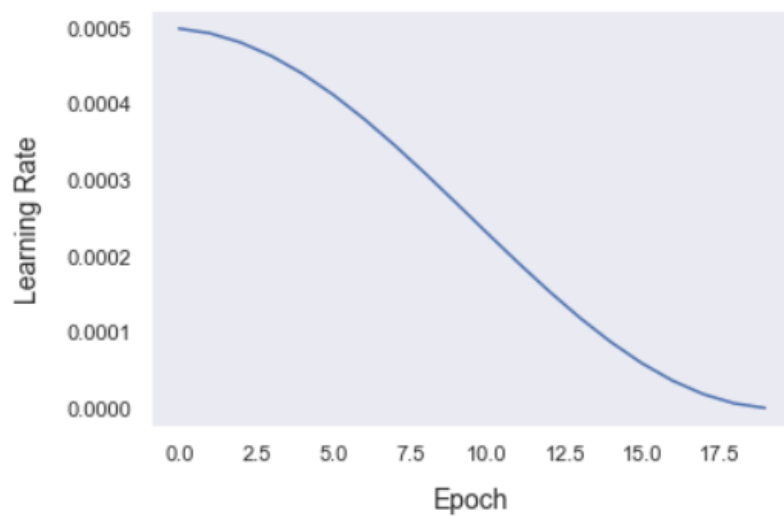


Figure 4.39 EfficientDet-D3 Learning Rate per epoch

4.7.4 EfficientDet-D5

EfficientDet-D5 (Tan et al., 2020) pretrained model weights on COCO dataset are retrained for 15 epochs to fit on the training images labelled with bounding boxes and trained 5 times based upon the stratified k-fold partitions. EfficientDet-D5 is trained on lower image resolution (384x384) because of GPU memory limitation.

EfficientDet-D5 training configuration:

- Image Size: 384 x 384
- Learning rate: 0.0001
- Batch size: 12
- Params: 33.4 M
- Training Images: 3431
- Validation Images: 857
- Training time per fold: 1 hr 50 mins
- Total training time: ~9 hrs
- GPU: Tesla P100-PCIE-16GB

4.7.4.1 Validation Dataset Metrics

Table 4.17 highlights the highest mAP@IOU-0.5 achieved by Efficient-D5 trained models on the validation dataset across all the folds.

Table 4.17 EfficientDet-D5 Highest mAP @IOU=0.5 on Validation Dataset

Metric	Fold	Value	Best Epoch
mAP@IOU=0.5	0	0.54	9
	1	0.55	9
	2	0.57	10
	3	0.52	10
	4	0.59	8
Avg mAP		0.554	

4.7.4.2 Training and Validation Loss

Figure 4.40 and Figure 4.41 shows the bounding box loss of both training and validation data across the training epochs. It also shows the improvement of mAP metric during training. Figure 4.42 depicts the decay in learning rate over the training epochs. Initial learning rate of EfficientDet-D5 is lower as compared to EfficientDet-D3 due to higher number of trainable parameters in D5.

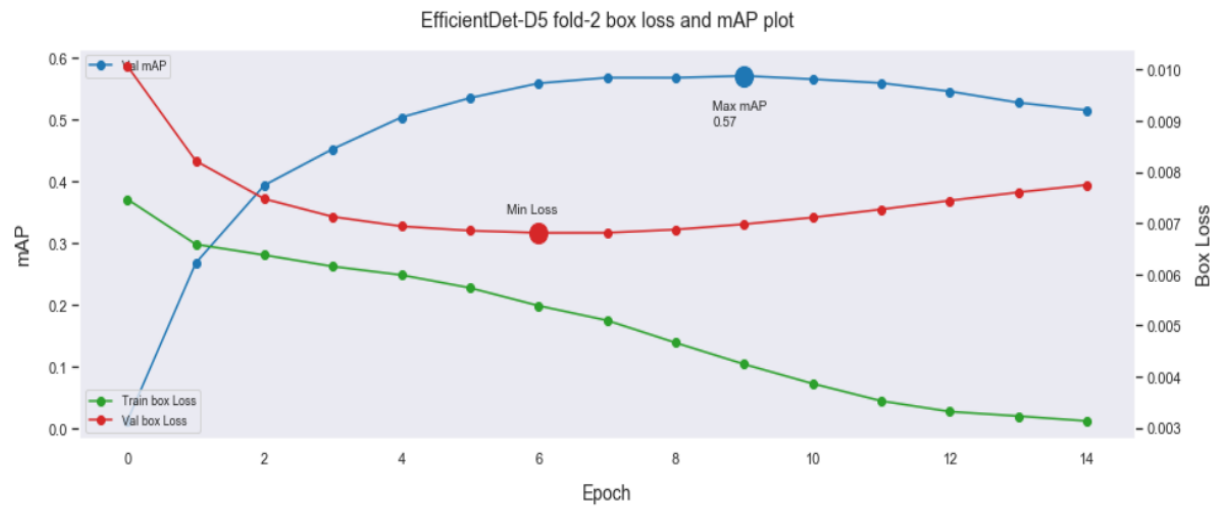


Figure 4.40 EfficientDet-D5 fold-2 Box loss and mAP plot across training epochs

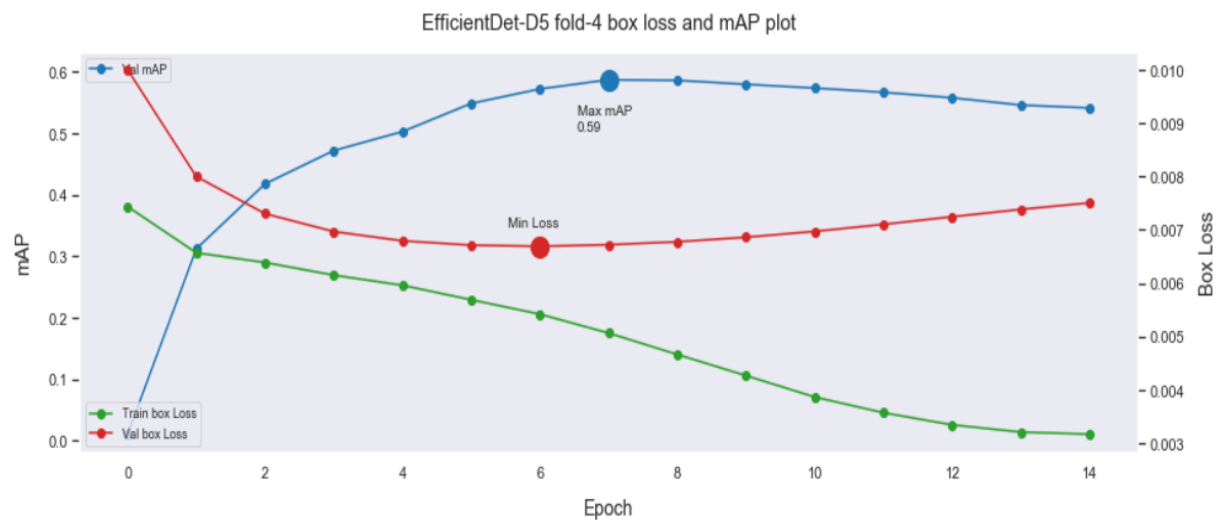


Figure 4.41 EfficientDet-D5 fold-4 Box loss and mAP plot across training epochs

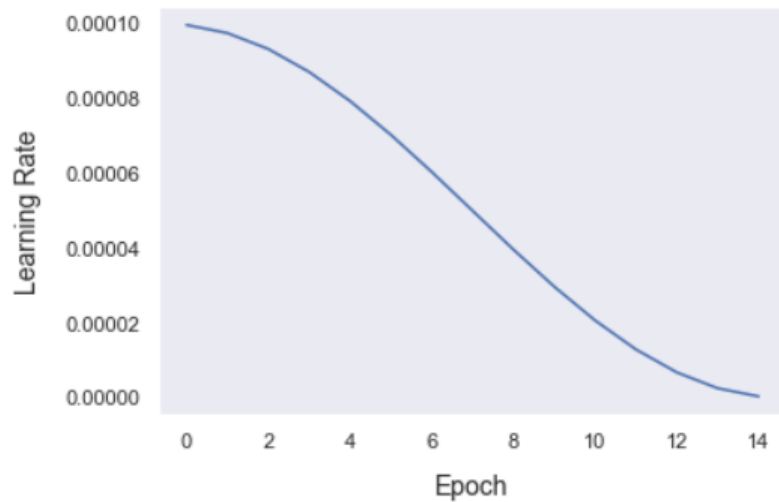


Figure 4.42 EfficientDet-D5 Learning Rate per epoch

4.8 Summary

EfficientNet based image classification models have been successfully trained on the dataset to learn the overall classification class. Segmentation head and class weights helped the classification networks to learn and generalize better. YOLO and EfficientDet object detection models have been successfully developed to localize the opacities on CXRs. All the models are trained using stratified 5-fold technique and hence there are 5 models for each of the variants of these architectures that have been trained on the dataset. Out of fold validation scores are monitored during training and best checkpoint models are saved based upon those scores. Trained models will be further used to predict on a private test dataset.

Chapter 5 RESULTS AND DISCUSSIONS

5.1 Introduction

Trained models are used for prediction on a private test dataset and overall mean average precision (mAP) score is calculated for both classification and object detection tasks. mAP score is one of the most popular metrics used in many competitions and in benchmarking state of art algorithms. mAP score is the area under precision/recall curve and averaged across all the calculated values for each class. Private test dataset comprises of approximately 8000 chest X-ray images held by the competition organizers. Final mAP score is only revealed upon submission of the trained models and their predictions on the private test dataset. An inference notebook is prepared that can load all the trained models and required dependencies to perform predictions on the test dataset.

EfficientNet v2m and v1-B5 classification models are used for predicting the overall classification class on the private test dataset as discussed in section 5.2. EfficientDet D3 and D5 detection models and YOLO v5l and v5x detection models are used for predicting the opacities in the form of bounding boxes on the private test dataset as discussed in section 5.3. Submissions are evaluated and ranked based upon the highest mAP score on the test dataset. Weighted ensembles of the trained models are iteratively submitted for evaluation and best combination of models are finalized and recommended based upon the highest attained mAP score as discussed in section 5.4. Highest mAP score that has been achieved up till now on the private test dataset is 63.5%. Prediction errors of classification and detection models on validation dataset are examined in section 5.5.

5.2 Final Classification Models

EfficientNet-v2m and EfficientNet-v1 B5 trained classification models are used for final predictions on the test dataset.

5.2.1 EfficientNet v2m - Test Dataset Evaluation

EfficientNet-v2m models that are trained using segmentation head, attention layer, MISH activation, weighted loss function and on image sizes 512x512 are used for final predictions on the test dataset. Details of all the EfficientNet-v2m experiments such as training the network with and without segmentation head or training the model with and without weights in loss function are discussed in detail in section 4.6.1. Segmentation head aided the classification task

to learn better mapping functions for predicting the overall class. Weighted loss function improved the learning of minority classes by the classification networks. Results of some of the key experiments of EfficientNet-v2m on out of fold (OOF) validation dataset are summarized in Table 5.1. Figure 5.1 and Figure 5.2 highlight the overall improvements in mAP and AUC-ROC scores respectively between the models trained with and without segmentation head and when class weights are included. Figure 5.3 illustrates the improvements in F1 scores of minority classes when class weights are used.

Table 5.1 Results of EfficientNet-v2m experiments on OOF Validation Dataset

Metrics	Average Value of all folds without segmentation head	Average Value of all folds with segmentation head	Average Value of all folds with segmentation head and class weights (1.0 - Negative, 1.0 - Typical, 2.0 - Indeterminate, 2.0 - Atypical)
mAP (Mean Average Precision)	0.55	0.565 (+1.5%)	0.575 (+2.5%)
AUC-ROC (Area under the Curve)	0.803	0.813 (+1%)	0.816 (+1.3%)
F1 score of Indeterminate class	NA	0.19	0.23 (+4%)
F1 score of Atypical class	NA	0.24	0.34 (+10%)

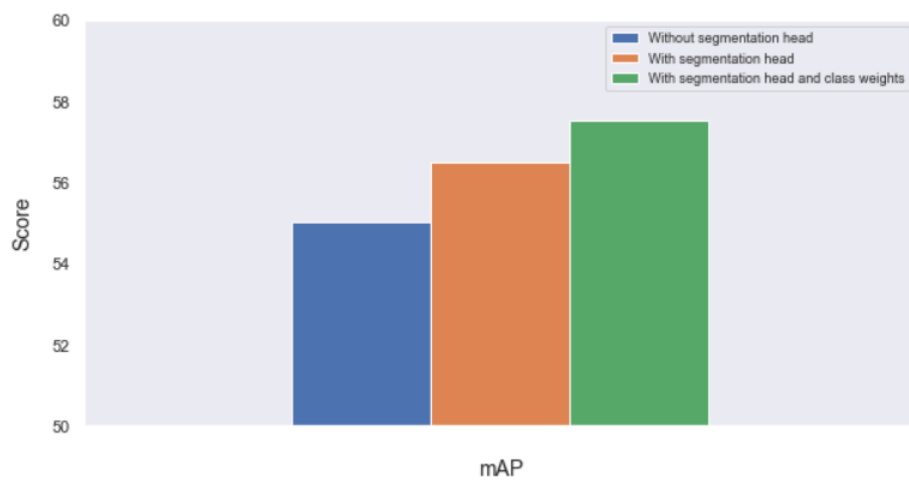


Figure 5.1 mAP Results of EfficientNet-v2m experiments on OOF Validation Dataset

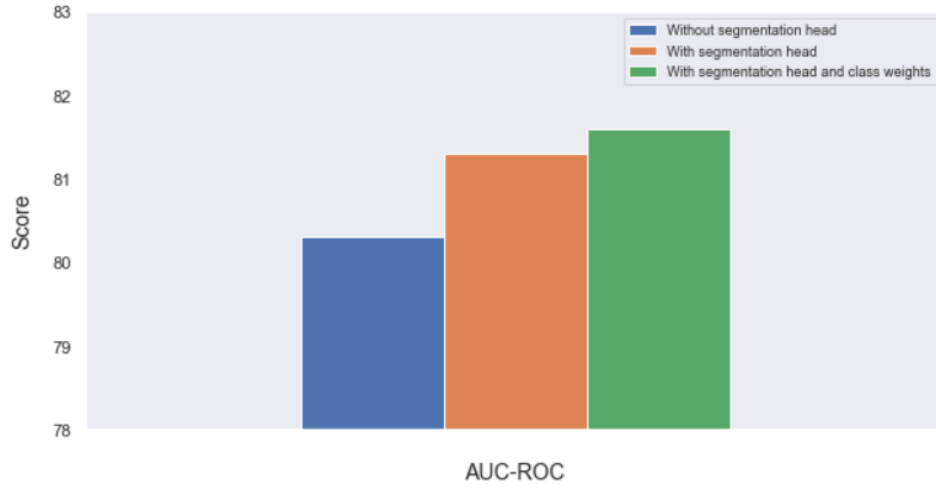


Figure 5.2 AUC-ROC Results of EfficientNet-v2m experiments on OOF Validation Dataset

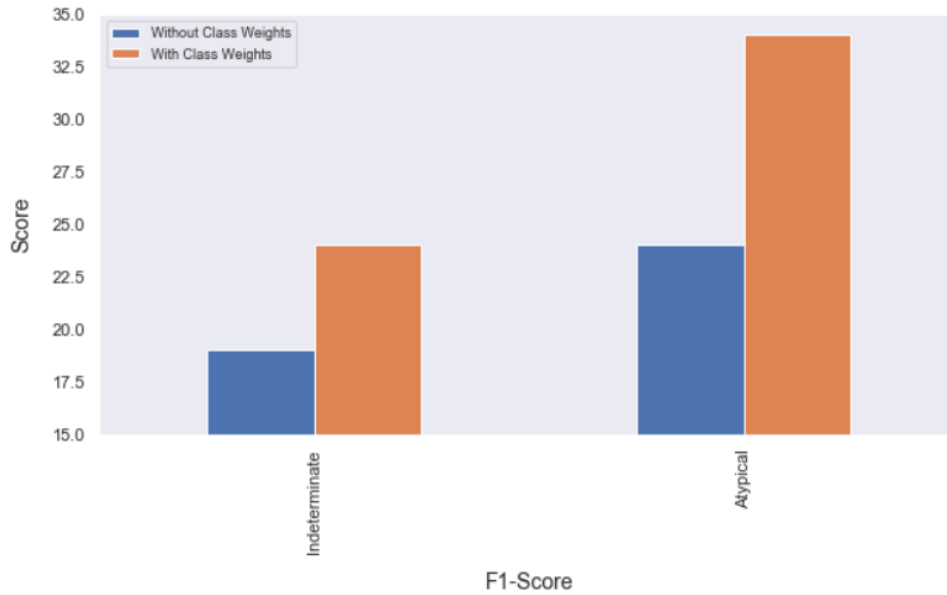


Figure 5.3 F1-Score Results of EfficientNet-v2m experiments on OOF Validation Dataset

EfficientNet-v2m 5-fold models trained using segmentation head and class weights attained an average mAP score of 57.5%, average accuracy score of 68% and average AUC-ROC score of 81.6% on OOF validation dataset as shown in Table 5.2 and corresponding best checkpoint models are used for evaluation on the test dataset. Test predictions are ensembled and averaged across the 5-fold trained models. Fixing the detection models as YOLO and submitting the classification predictions using only the EfficientNet-v2m models achieved an overall mAP score of 61.7% on the private test dataset in the leader board ranking as depicted in Figure 5.4. EfficientNet-v2m was trained for image sizes 512x512 as well as for 480x480 but models trained on image sizes 512x512 are only used for final submission on the test dataset. Combining predictions from trained models on these different image sizes was also attempted to add diversity but did not have a significant impact on the final score.

Table 5.2 Results of final EfficientNet-v2m models on OOF Validation Dataset

Metrics	Average Value of all folds with segmentation head and class weights [1.0, 1.0, 2.0, 2.0]
mAP (Mean Average Precision)	0.575
Accuracy	0.68
AUC-ROC (Area under the Curve)	0.816

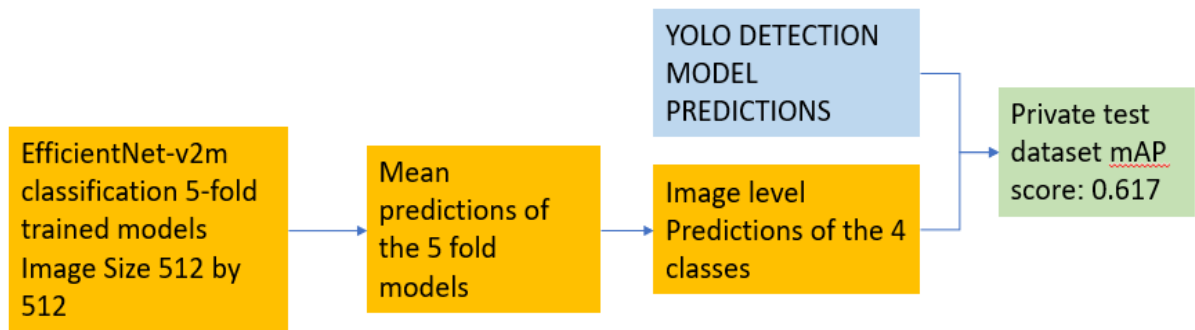


Figure 5.4 Test Dataset mAP using final EfficientNet-v2m 5-fold models

5.2.2 EfficientNet v1 B5 - Test Dataset Evaluation

EfficientNet-v1 B5 models that are trained using segmentation head and weighted loss function and on image sizes 512x512 are used for final predictions on the test dataset. Details of all the EfficientNet-v1 B5 experiments such as with different configuration of weights in loss function are discussed in detail in section 4.6.2. Results of some of the key experiments of EfficientNet-v1 B5 on out of fold validation dataset are summarized in Table 5.3. There is a minor difference in validation dataset metrics when model is trained using a weight of 1.5 vs 2.0 for indeterminate class. F1 scores of each of the 4 classes are slightly higher when weights for negative: typical: indeterminate: atypical classes are in the ratio of 1:1:1.5:2 and hence those trained models are used for final submission on the test dataset. EfficientNet-v1 B5 5-fold models trained using segmentation head and class weights attained an average mAP score of 57.6%, average accuracy score of 67% and average AUC-ROC score of 81.7% on out of fold validation dataset as shown in Table 5.4.

EfficientNet-v1 B5 5-fold best checkpoint models are used for evaluation on the test dataset. Test predictions are ensembled and averaged across the 5-fold trained models. Fixing the detection models as YOLO and submitting the classification predictions using only the EfficientNet-v1 B5 models achieved an overall mAP score of 61.6% on the private test dataset in the leader board ranking as depicted in Figure 5.5. EfficientNet-v1 B5 was trained for image sizes 512x512 as well as for 456x456 but models trained on image sizes 512x512 are only used for final submission on the test dataset. Combining predictions from trained models on different image sizes was also attempted to add diversity but didn't have a significant impact on the final score.

Table 5.3 Results of EfficientNet-v1 B5 experiments on OOF Validation Dataset

Metrics	Average Value of all folds with segmentation head and class weights (1.0 - Negative, 1.0 - Typical, 2.0 - Indeterminate, 2.0 - Typical)	Average Value of all folds with segmentation head and class weights (1.0 - Negative, 1.0 - Typical, 1.5 - Indeterminate, 2.0 - Typical)
mAP (Mean Average Precision)	0.575	0.576 (+0.1%)
AUC-ROC (Area under the Curve)	0.815	0.817 (+0.2%)

Table 5.4 Results of final EfficientNet-v1 B5 models on OOF Validation Dataset

Metrics	Average Value of all folds with segmentation head and class weights [1.0, 1.0, 1.5, 2.0]
mAP (Mean Average Precision)	0.576
Accuracy	0.67
AUC-ROC (Area under the Curve)	0.817

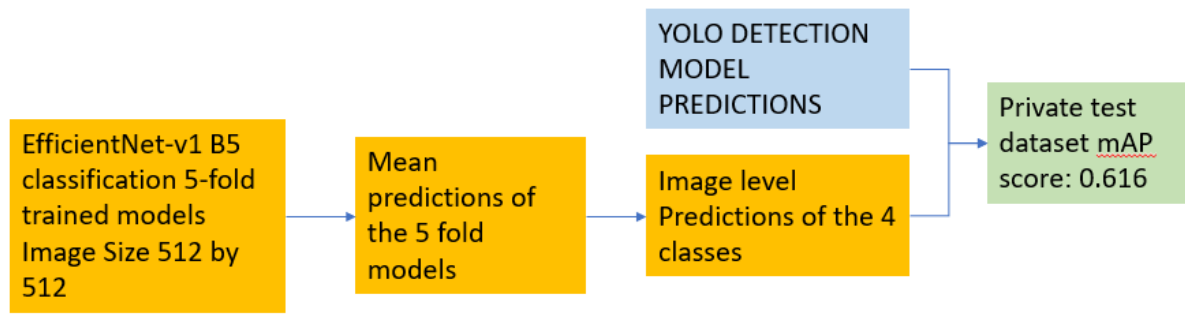


Figure 5.5 Test Dataset mAP using final EfficientNet-v1 B5 5-fold models

5.2.3 Weighted Ensemble of EfficientNet v2m and v1 B5 - Test Dataset Evaluation

Ensembling and combining predictions from multiple models helps in smoothing the mapping function learnt by the contributing members and seeks to reduce the prediction errors made by multiple models trained on the same task. EfficientNet-v2m mean predictions with a weight of 0.6 and EfficientNet-v1 B5 mean predictions with a weight of 0.4 are used for evaluation on the test dataset. Intuition behind giving more weightage to EfficientNet-v2m predictions (60%) as compared to EfficientNet-v1 B5 predictions (40%) is that EfficientNet-v2m models have higher F1-scores than EfficientNet-v1 B5 models on the lower represented classes. Fixing the detection models as YOLO and submitting the weighted ensemble predictions of both EfficientNet-v2m and EfficientNet-v1 B5 architectures achieved an overall mAP score of 61.9% on the private test dataset in the leader board ranking as depicted in Figure 5.6. Weighted ensemble of both architectures improves the mAP score by approx. 0.3% in comparison to the scores of either EfficientNet-v2m or EfficientNetv1-B5.

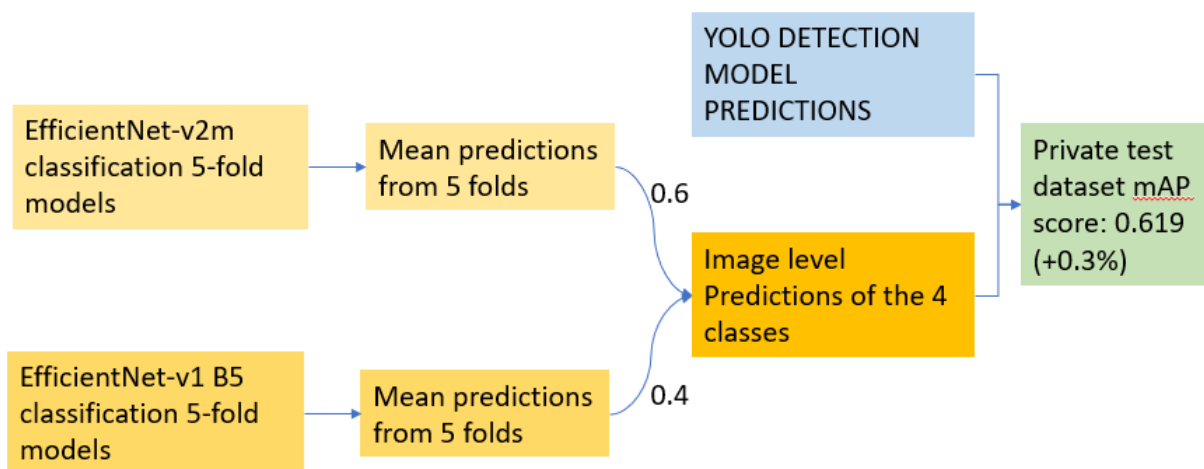


Figure 5.6 Test Dataset mAP: Weighted Ensemble of EfficientNetv2m and EfficientNetv1-B5

5.2.4 EfficientNet v2m comparison with ResNet50/DenseNet

Figure 5.7 and Figure 5.8 highlight the difference in mAP and AUC-ROC scores respectively between EfficientNet-v2m and ResNet-50 and DenseNet-201 on OOF validation dataset. ResNet-50 and DenseNet-201 trained models are not used for final test dataset predictions because of their lower performance on validation dataset as compared to EfficientNet networks. Top-1 accuracy of these models is approx. 6% lower than EfficientNets on standard ImageNet database too.

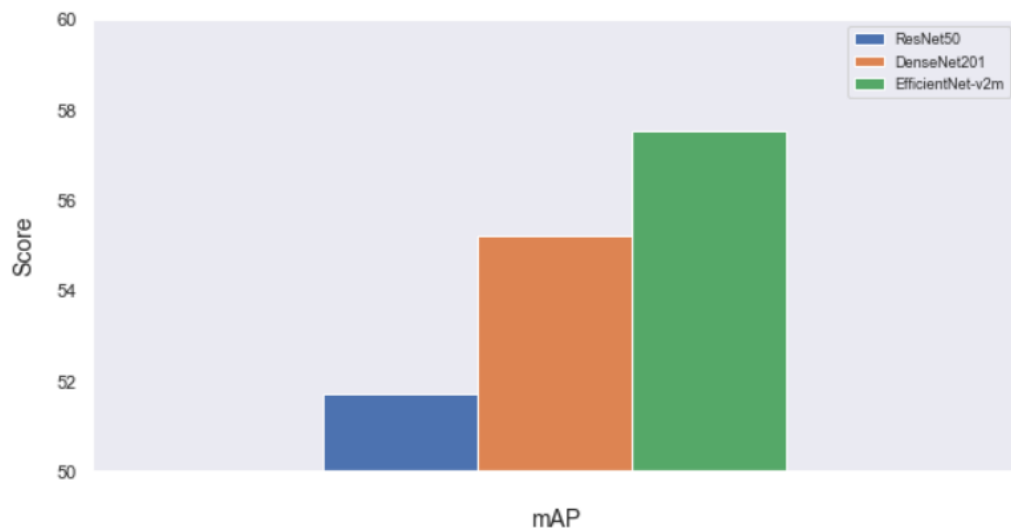


Figure 5.7 mAP Comparison of EfficientNet with ResNet and DenseNet on Validation Dataset

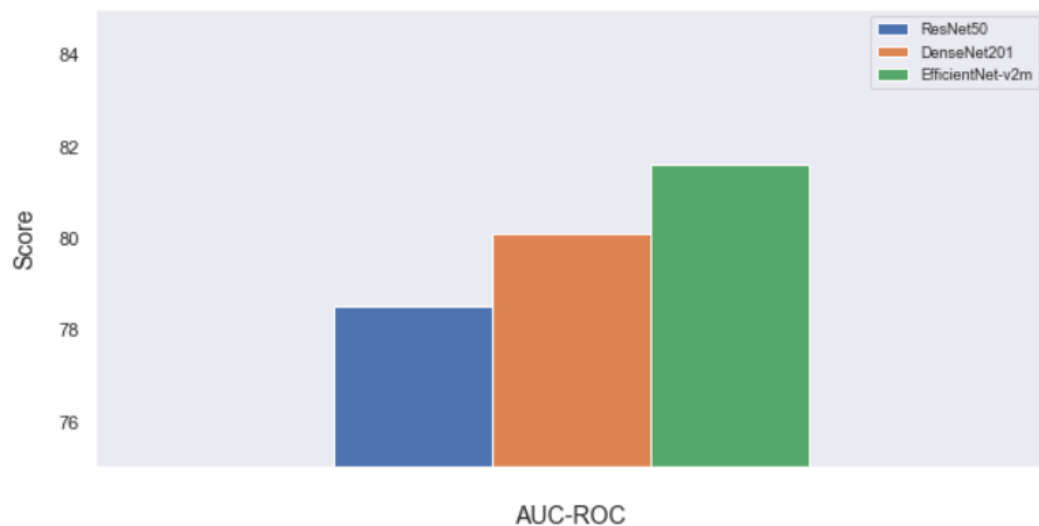


Figure 5.8 AUC-ROC Comparison of EfficientNet with ResNet and DenseNet on Validation Dataset

5.3 Final Detection Models

EfficientDet-D3, EfficientDet-D5, YOLO-v5l, and YOLO-v5x trained detection models are used for final predictions on the test dataset. Predictions from multiple detections models are combined using weighted box fusion (Solovyev et al., 2021). Weighted box fusion is a novel way to combine bounding boxes predictions from multiple models. Unlike standard non max suppressions techniques, it does not discard any proposed bounding boxes but uses confidence scores of all predicted boxes to construct average boxes to boost the overall final predictions.

EfficientNet-v2m binary classification model (section 4.6.4) that was trained specifically to enspredict overall negative class is also power blended with detection model predictions to fine tune the confidence scores of final merged boxes following weighted box fusion. As an example, confidence scores of proposed bounding boxes are increased if the prediction probability of negative class by classification model is very low and vice versa that is confidence scores of proposed bounding boxes are reduced if the prediction probability of negative class by classification model is high.

$$opacity\ pred = (opacity\ pred)^{0.8} * (1 - negative\ pred)^{0.2}$$

5.3.1 EfficientDet-D3 and EfficientDet-D5 - Test Dataset Evaluation

EfficientDet-D3 models that are trained on image sizes 640x640 and EfficientDet-D5 models that are trained on image sizes 384x384 are used for final predictions on the test dataset. EfficientDet-D3 and EfficientDet-D5 5-fold trained models attained an average mAP score of 55.4% on out of fold validation dataset as shown in Table 5.5. EfficientDet-D3 and EfficientDet-D5 5-fold best checkpoint models are used for evaluation on the test dataset. Test predictions from the 5-fold trained models are merged using weighted box fusion followed by tuning of confidence scores using the binary classification model for negative class. Bounding boxes predictions with overlapping area or IOU greater than 60% are combined using weighted box fusion. Fixing the classification models and submitting the detection predictions using only the EfficientDet models achieved an overall mAP score of 62% on the private test dataset in the leader board ranking as depicted in Figure 5.9.

Table 5.5 Results (mAP@IOU=0.5) of EfficientDets on OOF Validation Dataset

Metric	Model	Average Value of all folds
mAP @IOU=0.5	EfficientDet-D3	0.554
	EfficientDet-D5	0.554

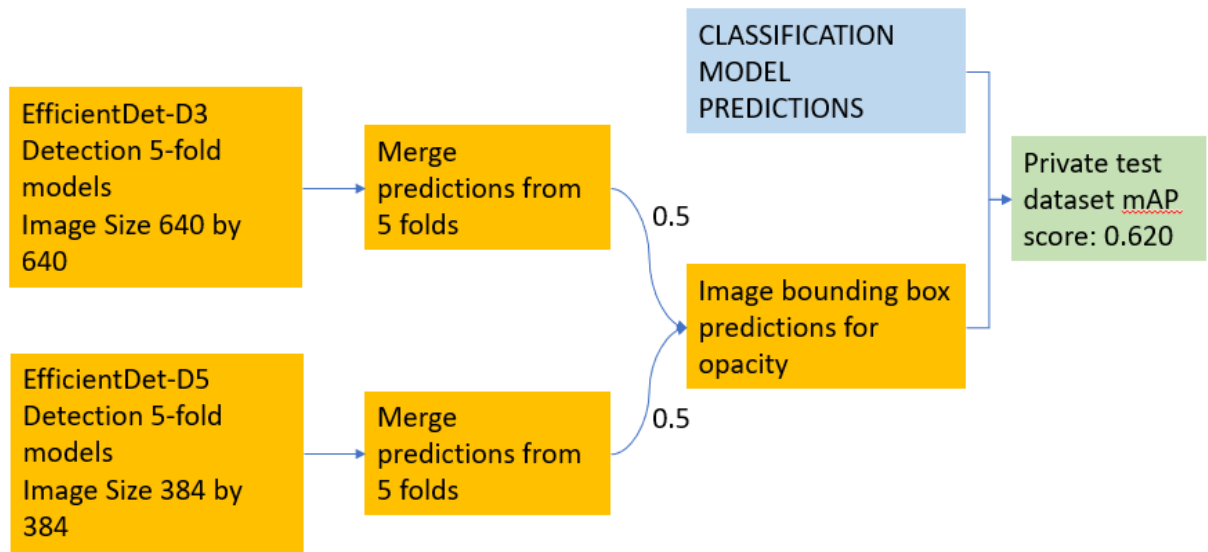


Figure 5.9 Test Dataset mAP using ensembling of EfficientDet-D3 and EfficientDet-D5

5.3.2 YOLO-v5l and YOLO-v5x - Test Dataset Evaluation

YOLO-v5l and YOLO-v5x models that are trained on image sizes 640x640 are used for final predictions on the test dataset. YOLO-v5l and YOLO-v5x 5-fold trained models attained an average mAP score of 56.4% and 55.4% respectively on out of fold validation dataset as shown in Table 5.6. YOLO-v5l and YOLO-v5x 5-fold best checkpoint models are used for evaluation on the test dataset. Test predictions from the 5-fold trained models are merged using weighted box fusion followed by tuning of confidence scores using the binary classification model for negative class. Bounding boxes predictions with overlapping area or IOU greater than 60% are combined using weighted box fusion. Fixing the classification models and submitting the detection predictions using only the YOLO models achieved an overall mAP score of 61.9% on the private test dataset in the leader board ranking as depicted in Figure 5.10.

Table 5.6 Results (mAP@IOU=0.5) of YOLOs on OOF Validation Dataset

Metric	Model	Average Value of all folds
mAP @IOU=0.5	YOLO-v5l	0.564
	YOLO-v5x	0.554

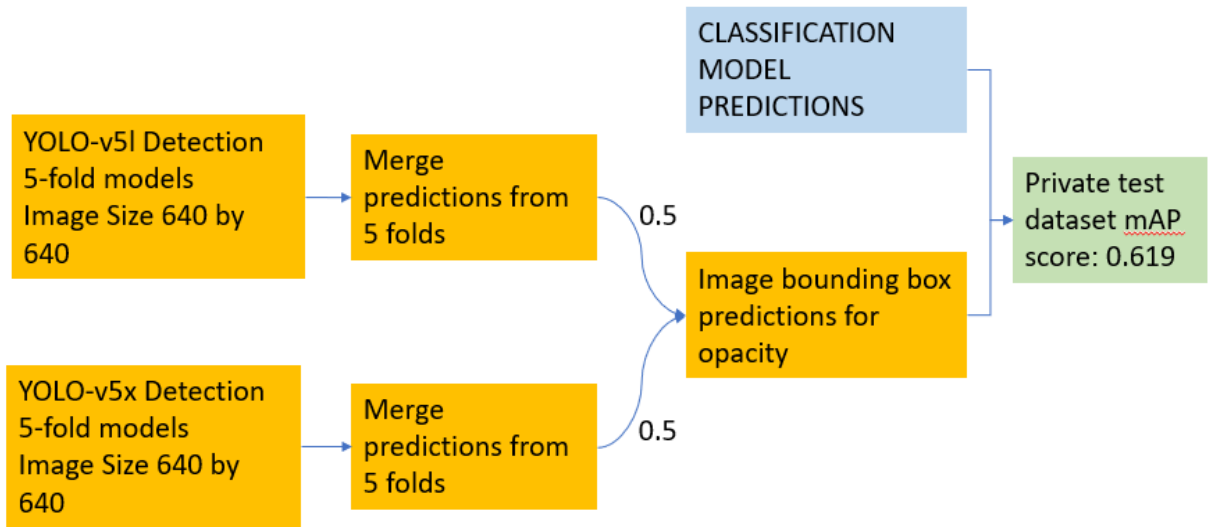


Figure 5.10 Test Dataset mAP using ensembling of YOLO-v5l and YOLO-v5x

5.3.3 Weighted box fusion of EfficientDet and YOLO - Test Dataset Evaluation

EfficientDet-D3, EfficientDet-D5, YOLO-v5l and YOLO-v5x 5-fold model predictions of bounding boxes are merged with equal weightage and are used for evaluation on the test dataset. Intuition behind giving equal weightage to all detection models is that they all performed equally well on validation dataset. Fixing the classification models and submitting the weighted box fusion predictions of both YOLO and EfficientDet detection architectures achieved an overall mAP score of 62.3% on the private test dataset in the leader board ranking as depicted in Figure 5.11. Weighted ensemble of both architectures with equal weightage given to all models improves the mAP score by approx. 0.3% in comparison to the scores of either EfficientDet or YOLO. Increase in final mAP score suggests that weighted ensemble of detection models helped in smoothing the overall detection errors and improving the detection accuracy.

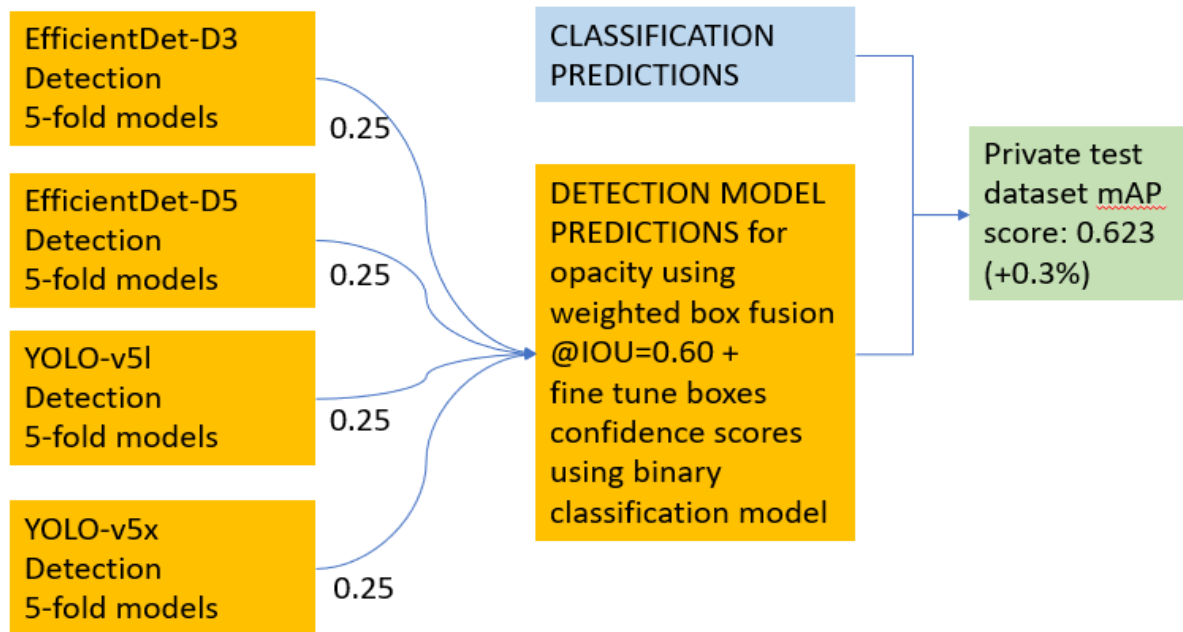


Figure 5.11 Test Dataset mAP: Weighted box fusion of EfficientDet and YOLO

5.4 Final Classification and Detection Models

EfficientNetv2m, EfficientNetv1-B5 classification models and EfficientDet-D3, EfficientDet-D5, YOLOv5l, YOLOv5x detection models achieved an overall mAP score of 62.3% on the private test dataset as shown in Figure 5.13. **Final mAP score of 62.3% on the test dataset stands at 12th position (top 1%) in the leader board ranking** as shown in Figure 5.12 and Figure 5.14.

An attempt is made to prune a couple of detection models by removing EfficientDet-D5 and YOLO-v5x models for final test data predictions. Using only EfficientDet-D3 and YOLO-v5l as detection models along with classification models also achieved same mAP score of 62.3% as depicted in Figure 5.15. Such a pruning is significant as reduced number of models help in improving inference time, reduced cost, and ease of deployment.

Submission (v2m B5 YOLO DET updated) Version 5 (version 5/6) 6 days ago by Puneesh Khanna Notebook Submission (v2m B5 YOLO DET updated) Version 5	Succeeded 	0.623	0.640
---	---	-------	-------

Figure 5.12 Test Dataset Submission Score

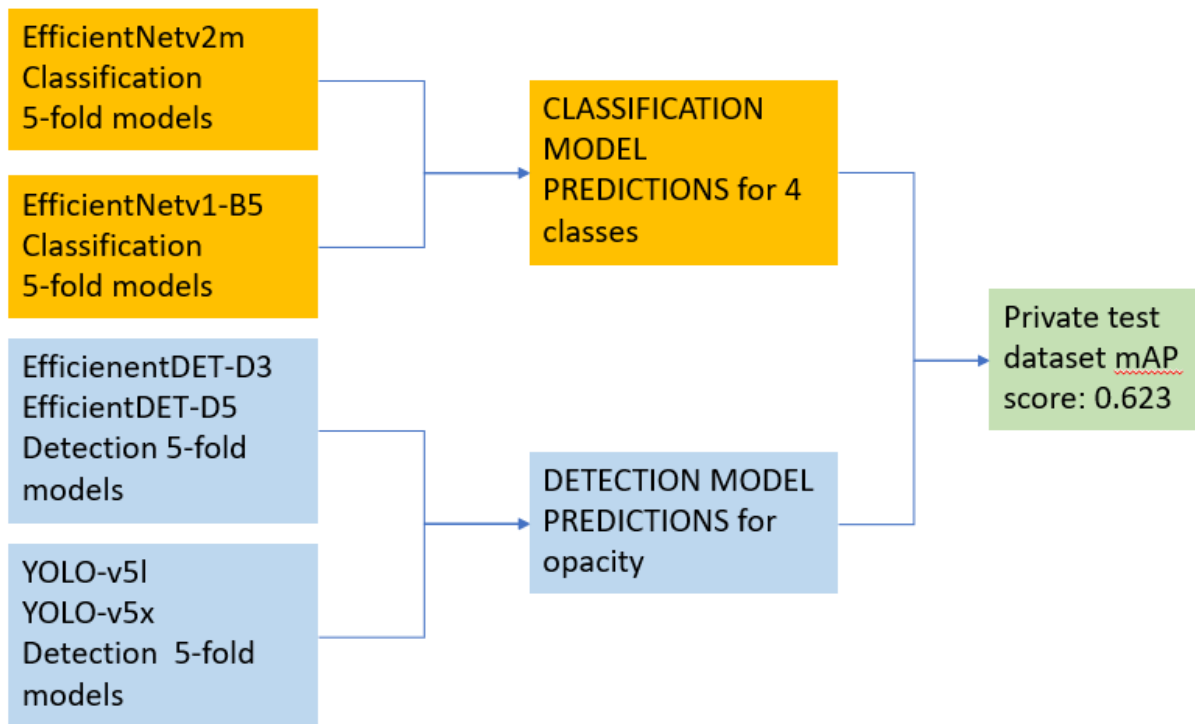


Figure 5.13 Test Dataset mAP: Final Classification and Detection Models

#	Δpub	Team Name	Notebook	Team Members	Score ?	Entries	Last
1	—	VinBigData-VinDr.ai	</> SIIM_COVID19_2021		0.635	262	3mo
2	▲ 4	A Team	</> Siim2021-Final ...		0.634	170	3mo
3	▲ 1	[Aillis] Yuji & Ian			0.634	188	3mo
4	▼ 2	RTX 4090			0.631	239	3mo
5	▼ 2	Ayushman Nischay Shivam	</> 604e8587410a (v2...		0.628	249	3mo
6	▲ 25	Quanta AI Lab	</> Terence_covid19_in...		0.628	19	3mo
7	▼ 2	[dsmlkz] School Zerde	</> validation_pipeline...		0.626	151	3mo
8	▲ 20	RTX 3090	</> siim-covid sub m...		0.625	169	3mo
9	—	Watercooled			0.625	85	3mo
10	▼ 2	Guanshuo Xu	</> notebook68483076ff		0.624	22	3mo
11	▲ 6	∫ √ ∂ ∇ ∇ ∇ ∇			0.624	341	3mo
12	▲ 1	Inoichan	</> [siim] sub145_lp c2...		0.623	159	3mo
13	▼ 3	CGG			0.623	136	3mo
14	▲ 15	D.Imanishi			0.622	158	3mo

Figure 5.14 Test Dataset Leaderboard Ranking

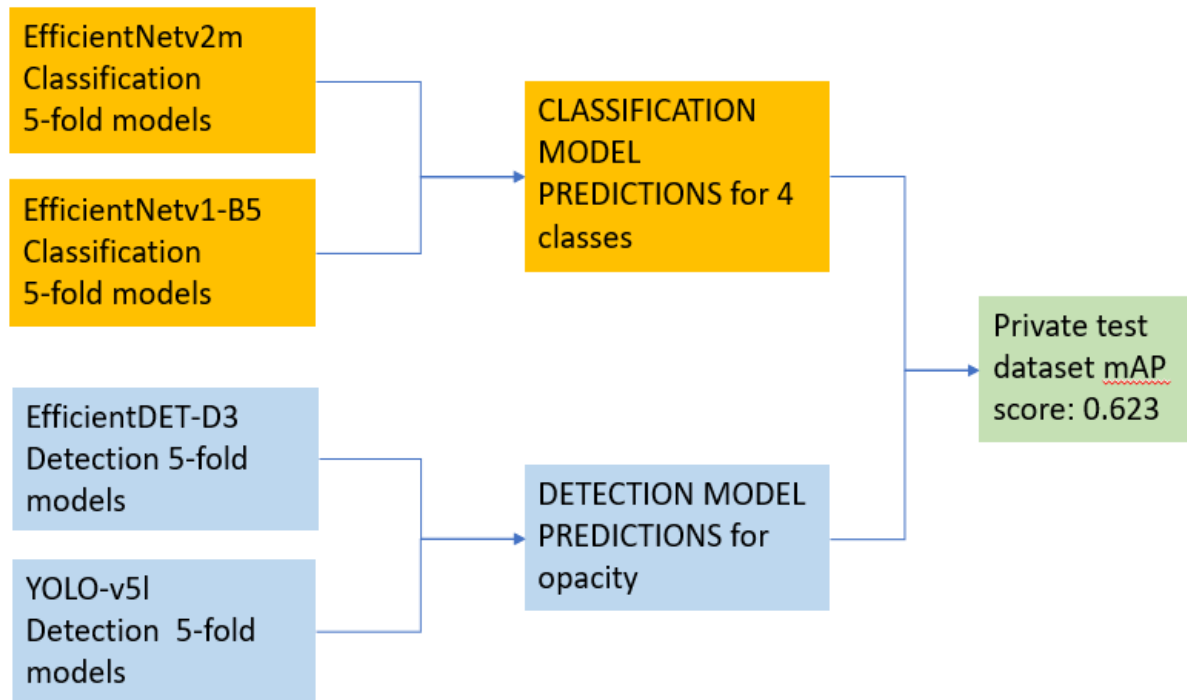


Figure 5.15 Test Dataset MAP: Final Classification and Pruned Detection Models

5.5 Error Analysis

Error analysis is attempted for both classification and detection models on out of fold validation datasets by comparing against the true labels.

5.5.1 Classification Models Errors

Table 5.7 reflects the confusion matrix corresponding to EfficientNet-v2m fold-0 model predictions on the validation dataset. It can be observed that the minority classes of the training dataset that are indeterminate and atypical have lower F1 scores. Usage of segmentation head to predict segmentation mask along with usage of class weights in loss function helped to handle the class imbalance to some extent and improved the accuracy and F1-scores of the under-represented classes. Combining predictions from 5-fold trained models is also expected to give a boost to the scores of these minority classes. At the same time, it can be observed that the recall % is quite low and many of the actual images belonging to indeterminate and atypical classes are being predicted as either typical or negative class. There are a couple of future recommendations highlighted in section 6.4 to further improve the predictions of these classes.

Table 5.7 EfficientNetv2m fold-0 model - confusion matrix and F1 score on validation dataset

Actual/ Predicted	Negative	Typical	Indeterminate	Atypical	Precision	Recall	F1 score
Negative	300	35	11	1	0.61	0.86	0.72
Typical	81	454	23	13	0.73	0.79	0.76
Indeterminate	79	94	29	8	0.37	0.14	0.21
Atypical	25	36	16	19	0.46	0.20	0.28

5.5.2 Detection Models Errors

Figure 5.16 depicts the confusion matrix of YOLO-v5l fold-0 model on validation dataset. Approx. 63% of the opacity label predictions are correctly predicted by the trained model. Combining detections from multiple trained models is expected to give a boost to the localization predictions of the opacities. At the same time, some of the incorrect predictions can be visually observed. Figure 5.17 and Figure 5.19 visualizes the true annotated labels of the validation dataset whereas Figure 5.18 and Figure 5.20 visualizes the predicted bounding boxes by the trained models corresponding to those true annotated labels. Some of the small bounding boxes have not been predicted by the model leading to false negatives. Another case of false negatives that can be observed is when the true number of boxes are more than 2 for an image and the model is not able to predict all of them. Model has also predicted 2 bounding boxes for few images where the number of true labels is just 1 leading to false positives. There are a couple of future recommendations highlighted in section 6.4 to further improve the localization predictions for the opacities.

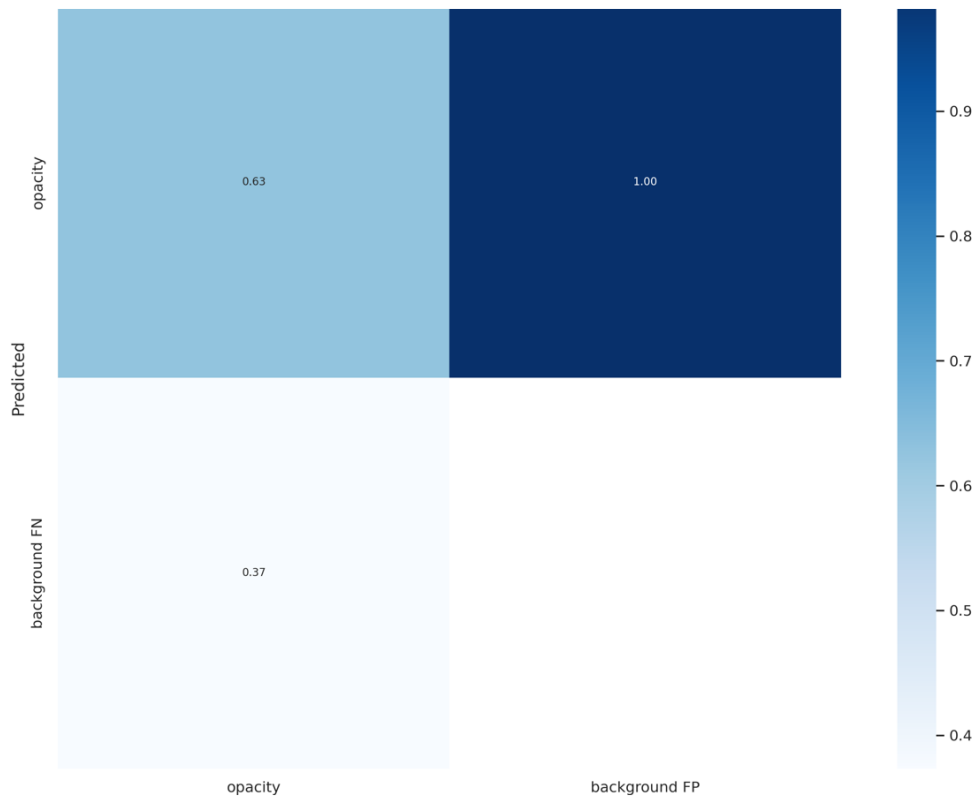


Figure 5.16 YOLO-v5l fold-0 model - confusion matrix on validation dataset

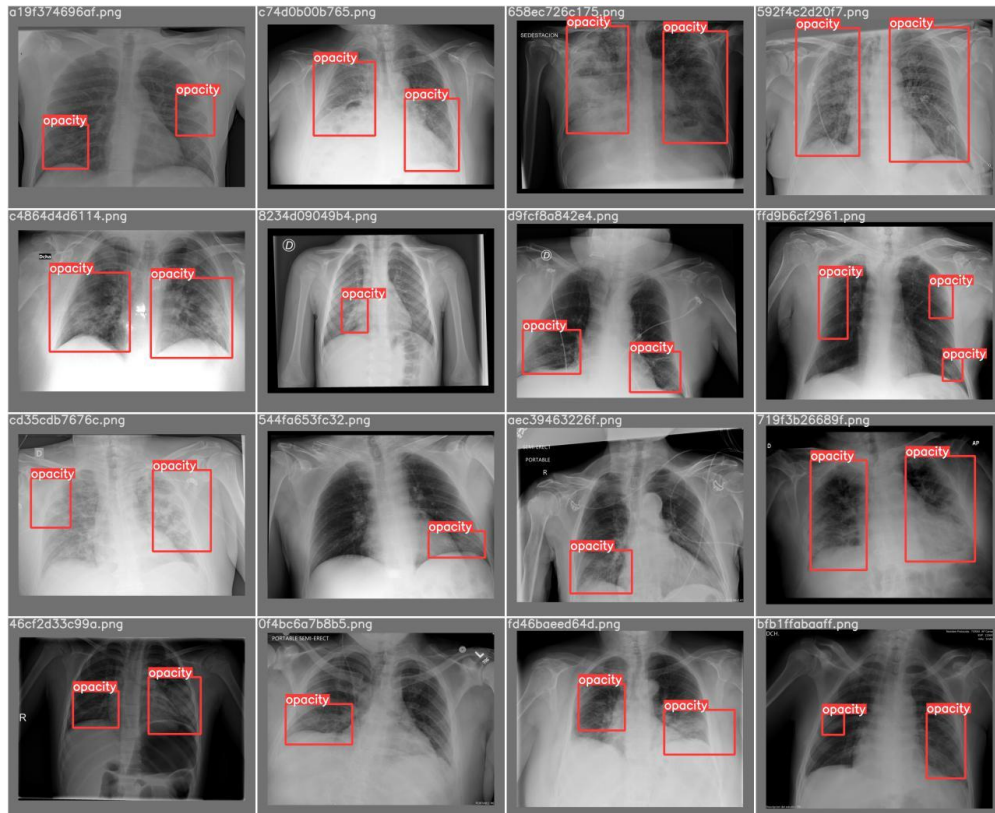


Figure 5.17 Visualization of actual opacities on validation dataset

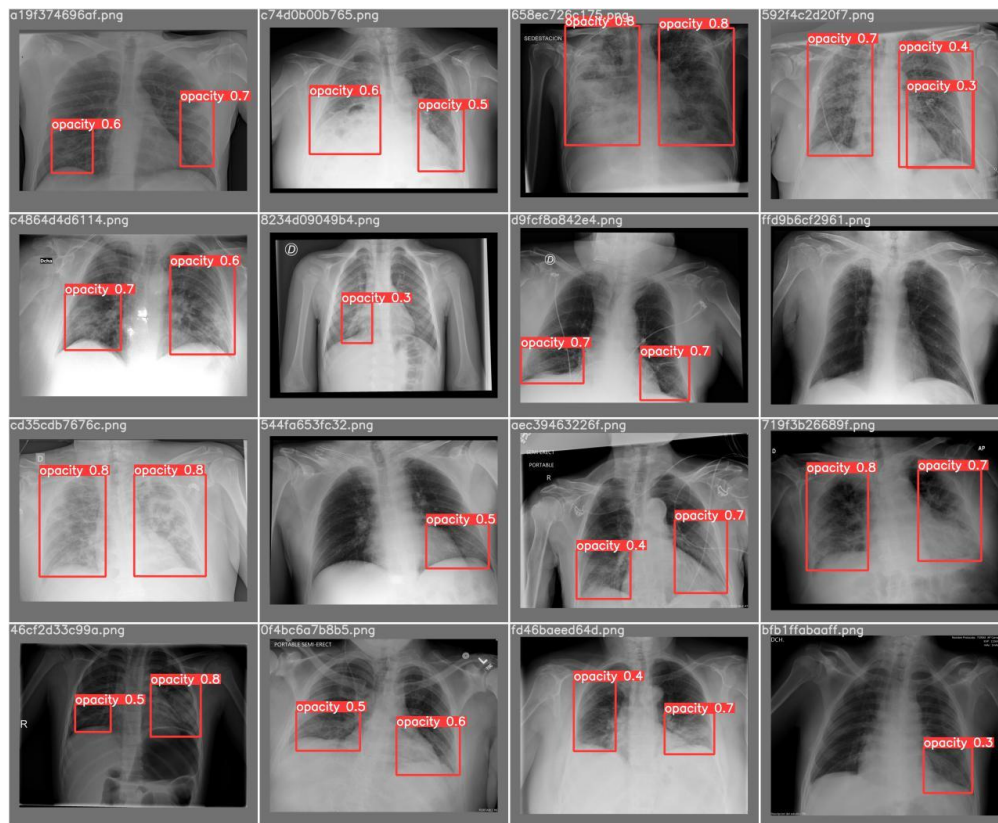


Figure 5.18 Visualization of predicted opacities by detection model on validation dataset

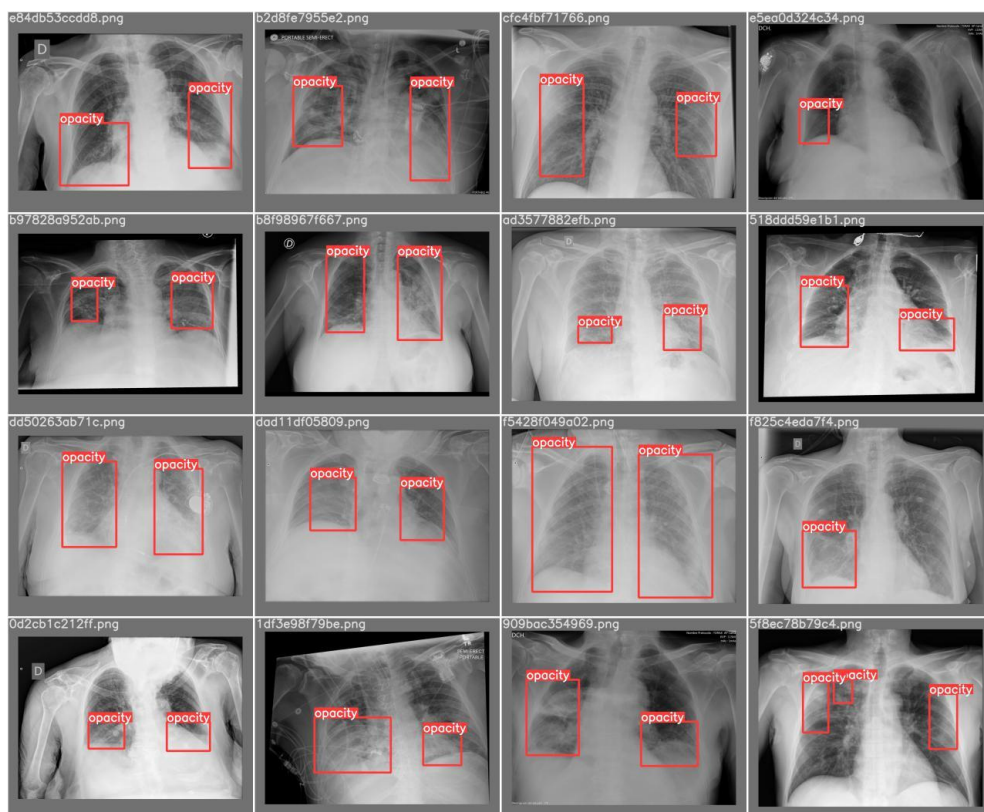


Figure 5.19 Visualization of actual opacities on validation dataset

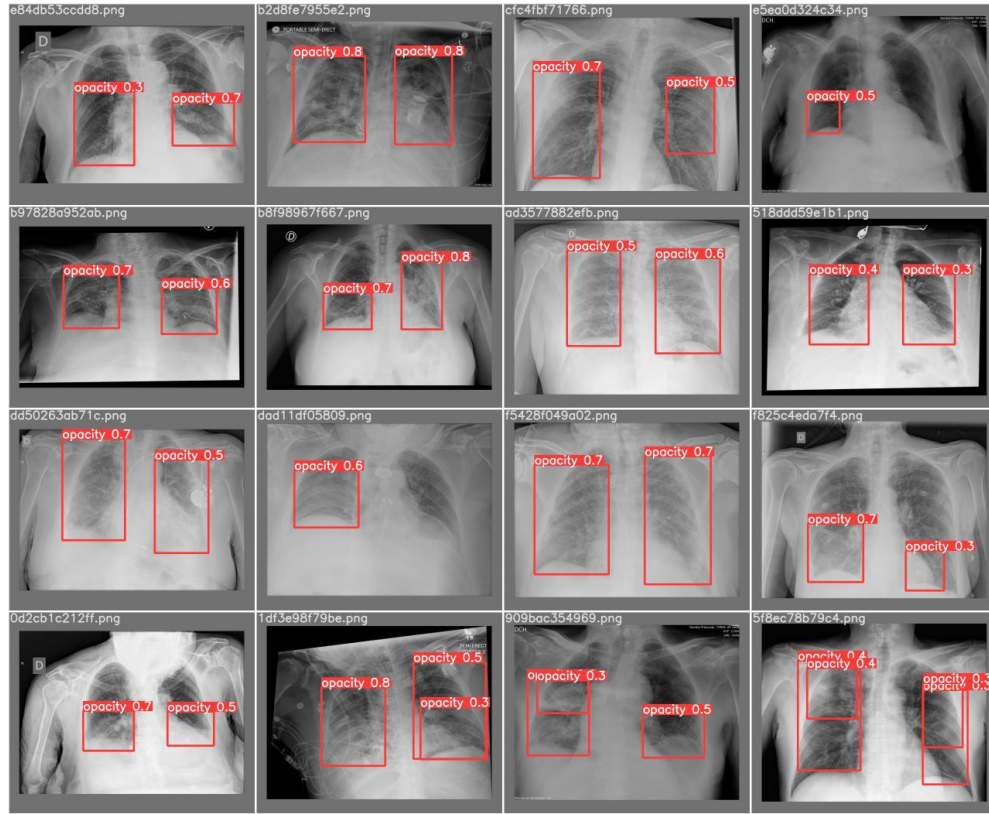


Figure 5.20 Visualization of predicted opacities by detection models on validation dataset

5.6 Summary

EfficientNet trained models are used for predicting the classification class on the test dataset. EfficientNet-v2m classification model along with YOLO detection trained models achieved an overall mAP score of 61.7%. EfficientNet-v1 B5 classification model along with YOLO detection trained models achieved an overall mAP score of 61.6%.

EfficientDet and YOLO trained models are used for predicting the bounding boxes of the opacity class on the test dataset. EfficientDet-D3 and EfficientDet-D5 along with classification trained models achieved an overall mAP score of 62%. YOLO-v5x and YOLO-v5l along with classification trained models achieved an overall mAP score of 61.9%.

Weighted ensemble of classification and detection models attained an overall final mAP score of 62.3%. Final mAP score on the private test dataset by both classification and detection models stands among top 15 results in the leader board ranking. Error analysis of the classification and detection models gives directions for further research and improvements in future.

Chapter 6 CONCLUSIONS AND RECOMMENDATIONS

6.1 Introduction

EfficientNet Classification models and YOLO and EfficientDet Detection models have been successfully trained, evaluated, and validated on the COVID CXRs dataset used in this research. Findings and results of the final trained models are discussed and concluded in section 6.2 to map with the objectives of this research. Key contributions and findings have been summarized in section 6.3. Limitations of the achieved work and important future recommendations are highlighted in section 6.4.

6.2 Discussion and Conclusion

Image Classification architectures of EfficientNets can understand and learn the overall class (Negative, Typical, Indeterminate and Atypical) of the chest X-ray images and performed better than DenseNets and ResNets. Usage of class weights along with usage of segmentation head during training further improved the performance and generalization of classification models. A strong data augmentation pipeline along with data resizing and normalization and proper selection of hyperparameters that have been used in this research are critical for neural network architectures to learn and generalize well on such complex medical imaging tasks. EfficientNet trained networks can distinguish between the 4 classification classes with approx. 68% accuracy, 57.5% mAP and 81.7% AUC-ROC scores on out of fold validation datasets. Ensembling and combining predictions from multiple architectures helped in further improving the overall results and accuracy of the trained models on unseen test dataset. EfficientNet-v2m with a weightage of 60% and EfficientNet-B5 with a weightage of 40% are used to predict the classification class on the private test dataset.

Latest state of art object detection architectures such as EfficientDets and YOLOs can learn the localization of opacities on chest X-ray images. EfficientDet and YOLO trained networks can localize the opacities with approx. 56% mAP@IOU=0.5 score on out of fold validation datasets. EfficientDet and YOLO trained models are given equal weightage during ensembling to predict the opacity bounding boxes on the private test dataset. The final trained classification and detection models achieved top 1% ranking (12th in the leader board ranking) with an overall mAP score of 62.3% on the private test dataset.

6.3 Contribution to Knowledge

The proposed image classification and object detection models in this research achieved top 1% results on the test dataset. These models can play a significant role in understanding and categorizing the COVID chest X-ray images and localizing the opacities in the form of bounding boxes on these images. Some of the major findings with respect to analysis, training and evaluation of the developed AI models are as follows:

- EfficientNet architectures performed better than ResNets and DenseNets on classification task.
- Adding an auxiliary or segmentation head during training of classification models and backpropagating the segmentation loss along with classification loss during training helped the classification models learn and generalize better on the training dataset.
- Adding more weightage to the loss function for the less represented classes (Indeterminate and Atypical) aided in balancing the overall class distribution and improved precision, recall and F1 scores of these minor classes.
- EfficientNet-v2m training using MISH activations achieved comparable results as with training using default RELU activations.
- Appending attention layers to image classification architectures enhanced the models to learn and perform better.
- State of art EfficientDet object detection architectures performed well in detecting and localizing the opacities on chest X-ray images.
- State of art YOLO object detection architectures performed well in detecting and localizing the opacities on chest X-ray images.
- Ensembling and combining predictions from multiple architectures improved the overall performance on the private test dataset which has greater number of images than the training dataset.

6.4 Future Recommendations

The proposed models in this research achieved top 1% state of art results. At the same time, there is definite scope to further improve the results and performance of these models to be more feasible and reliable for deployment in real world.

Few of the important recommendations categorized with priorities are as follows:

P1:

- Currently, 5-fold models are ensembled for final predictions, but the training and deployment of such models is quite costly. Can single trained models of say EfficientNet-v2m and EfficientNet-v1 B5 achieve same results on classification task as 5-fold trained models? Similarly, can single trained models of say YOLO and EfficientDet achieve same results on detection task as 5-fold trained models? If single trained models can achieve similar results, then they can really ease the deployment and usage of inference resources as well as improve the overall inference or prediction latency on real world data. An attempt to prune a couple of detection models has already been made on the private test dataset.
- In this research, class weights have been used in the loss function to handle class imbalance which helped to improve the results of under-represented classification classes. Very few to almost none of the top solutions seems to have attempted usage of appropriate class weights to handle the imbalance in the dataset. Along with applying class weights, there is scope to further improve the results by techniques such as oversampling of the misrepresented classes in the dataset.
- Approx. 20% images of the private test dataset are available in the dataset repository but without any golden truth or labels. Those images can be leveraged to generate soft labels from the trained models and the models can be further refined on the actual training dataset plus the test images with the generated soft labels. Such a technique has been used by most of the top solutions. As an example, if a trained classification model predicts a class with greater than 70% confidence on an image then those test images can also be sampled as part of training dataset with a soft golden label corresponding to that class. Such a technique can be particularly useful for under sampled classification classes that are indeterminate and atypical classes. Similarly, if a trained detection model predicts a bounding box with a confidence score of greater than 70% confidence

on an image then those test images can also be sampled as part of training dataset with a soft golden label corresponding to the predicted bounding box.

- Training of 2-stage detectors such as Mask R-CNN or Faster R-CNN for bounding boxes predictions can be studied in contrast to the single stage detectors - YOLO and EfficientDet that have been used in this research. 2-stage detectors are slower as compared to the single stage detectors but can have higher accuracy. Many of the winning solutions also used a 2-stage detector for final test dataset predictions.
- Emerging transformer-based detection networks can be trained and evaluated for localization of opacities.
- There is scope to further understand, visualize and improve the predictions of detection models for opacities with three or four bounding boxes and for opacities with smaller bounding boxes.

P2:

- Segmentation head is used along with classification architecture during training to predict the segmentation masks and classification classes. A simple convolution architecture is designed for the segmentation head. There is scope to use state of art segmentation architectures such as U-Net that has also been adopted in some of the top solutions. Adding a complex segmentation network will also attribute to increase of training parameters and training time. During inference, segmentation head is not used so it is not a cause of concern during deployment phase.
- Training models on higher resized resolution images such as 768x768 or 1024x1024 since the original images are of higher resolutions. Many of the winning solutions have trained models on larger image sizes. Image sizes that have been mainly used in this research are 512x512 for EfficientNet classification models, 640x640 for YOLO detection models, and 384x384 for EfficientDet-D5 detection models because of GPU memory limitations on Colab Pro. EfficientDet-D5 has been trained on a significantly lower resolution as compared to other models.
- In this research, pretrained classification models on ImageNet and pretrained detection models on COCO dataset have been used. Pretraining the models on external datasets related to chest X-rays can improve the initial weights of these models before fitting and

transferring them on the actual training dataset. Such an approach of transfer learning from related tasks tends to improve the results of the final prediction models and has been adopted by few of the top-20 solutions. On other hand, this methodology amounts to further increase of training time as pretrained models on related datasets may not be available for direct usage.

- There is scope to experiment with the data augmentation pipeline. As an example, Mosaic augmentation is only applied for training YOLO models but not for training EfficientDet models.
- Evaluation of models across the meta data information such as patient gender, modality, and photometric interpretation can provide useful insights.
- RELU activations have been replaced with MISH activations in the EfficientNet-v2m classification network. It didn't seem to have a significant impact on the tracked metrics such as mAP or AUC-ROC but impact on other metrics such as top-1 accuracy can be further researched.

REFERENCES

- Alshazly, H., Linse, C., Barth, E. and Martinetz, T., (2021) Explainable COVID-19 detection using chest CT scans and deep learning. *Sensors (Switzerland)*, 212, pp.1–22.
- Anon (2021) *COVID-19 AI Detection Challenge (2021)*. [online] Available at: <https://www.rsna.org/education/ai-resources-and-training/ai-image-challenge/covid-19-ai-detection-challenge-2021> [Accessed 6 Dec. 2021].
- Anon (2021) *Transforming healthcare with AI: The impact on the workforce and organizations* | McKinsey. [online] Available at: <https://www.mckinsey.com/industries/healthcare-systems-and-services/our-insights/transforming-healthcare-with-ai> [Accessed 3 Oct. 2021].
- Badža, M.M. and Barjaktarović, M.C., (2020) Classification of brain tumors from mri images using a convolutional neural network. *Applied Sciences (Switzerland)*, 106.
- Cai, W., Yang, J., Fan, G., Xu, L., Zhang, B. and Liu, R., (2020) Chest CT findings of coronavirus disease 2019 (COVID-19). *Journal of the College of Physicians and Surgeons Pakistan*, 301, pp.S53–S55.
- Camilleri, N. and Gatt, T., (2020) Detecting road potholes using computer vision techniques. *Proceedings - 2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing, ICCP 2020*, pp.343–350.
- Chowdhury, M.E.H., Rahman, T., Khandakar, A., Mazhar, R., Kadir, M.A., Mahbub, Z. Bin, Islam, K.R., Khan, M.S., Iqbal, A., Emadi, N. Al, Reaz, M.B.I. and Islam, M.T., (2020) Can AI Help in Screening Viral and COVID-19 Pneumonia? *IEEE Access*, 8, pp.132665–132676.
- Chung, M., Bernheim, A., Mei, X., Zhang, N., Huang, M., Zeng, X., Cui, J., Xu, W., Yang, Y., Fayad, Z.A., Jacobi, A., Li, K., Li, S. and Shan, H., (2020) CT imaging features of 2019 novel coronavirus (2019-NCoV). *Radiology*, 2951, pp.202–207.
- Cohen, J.P., Morrison, P. and Dao, L., (2020) COVID-19 Image Data Collection. [online] Available at: <http://arxiv.org/abs/2003.11597>.
- Deshpande, S., Muron, W. and Cai, Y., (2017) Vehicle Classification. *Computer Vision and Imaging in Intelligent Transportation Systems*, pp.47–79.

Gulum, M.A., Trombley, C.M. and Kantardzic, M., (2021) A review of explainable deep learning cancer detection models in medical imaging. *Applied Sciences (Switzerland)*, 1110.

Gunraj, H., Wang, L. and Wong, A., (2020) COVIDNet-CT: A Tailored Deep Convolutional Neural Network Design for Detection of COVID-19 Cases From Chest CT Images. *Frontiers in Medicine*, 7, pp.1–12.

He, K., Zhang, X., Ren, S. and Sun, J., (2016) Deep residual learning for image recognition. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016-Decem, pp.770–778.

He, X., Yang, X., Zhang, S., Zhao, J., Zhang, Y., Xing, E. and Xie, P., (2020) Sample-efficient deep learning for COVID-19 diagnosis based on CT scans. *IEEE Transactions on Medical Imaging*, XXXx.

Hirata, D. and Takahashi, N., (2020) Ensemble learning in CNN augmented with fully connected subnetworks. [online] Available at: <http://arxiv.org/abs/2003.08562>.

Huang, G., Liu, Z., Van Der Maaten, L. and Weinberger, K.Q., (2017) Densely connected convolutional networks. *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017*, 2017-Janua, pp.2261–2269.

Huang, H.K., Chiu, C.F., Kuo, C.H., Wu, Y.C., Chu, N.N.Y. and Chang, P.C., (2016) Mixture of deep CNN-based ensemble model for image retrieval. *2016 IEEE 5th Global Conference on Consumer Electronics, GCCE 2016*, 1, pp.5–6.

Irmak, E., (2021) Multi-Classification of Brain Tumor MRI Images Using Deep Convolutional Neural Network with Fully Optimized Framework. *Iranian Journal of Science and Technology - Transactions of Electrical Engineering*, [online] 453, pp.1015–1036. Available at: <https://doi.org/10.1007/s40998-021-00426-9>.

Jia Deng, Wei Dong, Socher, R., Li-Jia Li, Kai Li and Li Fei-Fei, (2009) ImageNet: A large-scale hierarchical image database. pp.248–255.

Jocher, G., Stoken, A., Borovec, J., NanoCode012, Chaurasia, A., TaoXie, Changyu, L., V, A., Laughing, tkianai, yxNONG, Hogan, A., lorenzomamma, AlexWang1900, Hajek, J., Diaconu, L., Marc, Kwon, Y., oleg, wanghaoyang0106, Defretin, Y., Lohia, A., ml5ah,

Milanko, B., Fineran, B., Khromov, D., Yiwei, D., Doug, Durgesh and Ingham, F., (2021) ultralytics/yolov5: v5.0 - YOLOv5-P6 1280 models, AWS, Supervise.ly and YouTube integrations. [online] Available at: <https://zenodo.org/record/4679653> [Accessed 1 Aug. 2021].

Kanne, J.P., Bai, H., Bernheim, A., Chung, M., Haramati, L.B., Kallmes, D.F., Little, B.P., Rubin, G. and Sverzellati, N., (2021) COVID-19 imaging: What we know now and what remains unknown. *Radiology*, 2993, pp.E262–E279.

Khan, A.I., Shah, J.L. and Bhat, M.M., (2020) CoroNet: A deep neural network for detection and diagnosis of COVID-19 from chest x-ray images. *Computer Methods and Programs in Biomedicine*, [online] 196, p.105581. Available at: <https://doi.org/10.1016/j.cmpb.2020.105581>.

Ko, H., Ha, H., Cho, H., Seo, K. and Lee, J., (2019) Pneumonia Detection with Weighted Voting Ensemble of CNN Models. *2019 2nd International Conference on Artificial Intelligence and Big Data, ICAIBD 2019*, pp.306–310.

Kornblith, S., Shlens, J. and Le, Q. V., (n.d.) Kornblith_Do_Better_ImageNet_Models_Transfer_Better_CVPR_2019_paper. pp.2661–2671.

Krizhevsky, B.A., Sutskever, I. and Hinton, G.E., (2012) Cnn实际训练的. *Communications of the ACM*, 606, pp.84–90.

Li, Z., Yang, W., Peng, S. and Liu, F., (n.d.) CNN survey.

Lin, T.Y., Goyal, P., Girshick, R., He, K. and Dollar, P., (2020) Focal Loss for Dense Object Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 422, pp.318–327.

Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C.L., (2014) Microsoft COCO: Common objects in context. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 8693 LNCS PART 5, pp.740–755.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S. and Guo, B., (2021) Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. [online] Available at: <http://arxiv.org/abs/2103.14030>.

Maghdid, H., Asaad, A.T., Ghafoor, K.Z.G., Sadiq, A.S., Mirjalili, S. and Khan, M.K.K., (2021) Diagnosing COVID-19 pneumonia from x-ray and CT images using deep learning and transfer learning algorithms. p.26.

Misra, D., (2019) Mish: A Self Regularized Non-Monotonic Activation Function. [online] Available at: <http://arxiv.org/abs/1908.08681>.

Pan, F., Ye, T., Sun, P., Gui, S., Liang, B. and Li, L., (2020) Time Course of Lung Changes at Chest CT during Recovery. *Radiology*, [online] 2953, pp.715–721. Available at: doi: 10.1148/radiol.2020200370.

Perumal, S. and Velmurugan, T., (2018) Preprocessing by Contrast Enhancement Techniques for Medical Images. *International Journal of Pure and Applied Mathematics*, 11818, pp.3681–3688.

Qiao, S., Chen, L.-C. and Yuille, A., (n.d.) *DetectoRS: Detecting Objects with Recursive Feature Pyramid and Switchable Atrous Convolution*. [online] Available at: <https://github.com/joe-siyuan-qiao/DetectoRS>.

Rahman, T., Khandakar, A., Qiblawey, Y., Tahir, A., Kiranyaz, S., Abul Kashem, S. Bin, Islam, M.T., Al Maadeed, S., Zughaier, S.M., Khan, M.S. and Chowdhury, M.E.H., (2021) Exploring the effect of image enhancement techniques on COVID-19 detection using chest X-ray images. *Computers in Biology and Medicine*, 132, p.104319.

Rajaraman, S. and Antani, S.K., (n.d.) Modality-Specific Deep Learning Model Ensembles Toward Improving TB Detection in Chest Radiographs.

Rajaraman, S., Siegelman, J., Alderson, P.O., Folio, L.S., Folio, L.R. and Antani, S.K., (2020) Iteratively Pruned Deep Learning Ensembles for COVID-19 Detection in Chest X-Rays. *IEEE Access*, 8, pp.115041–115050.

Rajpurkar, P., Irvin, J., Zhu, K., Yang, B., Mehta, H., Duan, T., Ding, D., Bagul, A., Langlotz, C., Shpanskaya, K., Lungren, M.P. and Ng, A.Y., (2017) CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. [online] pp.3–9. Available at: <http://arxiv.org/abs/1711.05225>.

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A., (2016) You only look once: Unified, real-time object detection. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016-Decem, pp.779–788.

Redmon, J. and Farhadi, A., (2018) YOLOv3: An Incremental Improvement. [online] Available at: <http://arxiv.org/abs/1804.02767>.

Release, R. and Covid, C., (n.d.) Medical Imaging Data Resource Center (MIDRC) - RSNA International COVID-19 Open Radiology Database. pp.1–6.

Sadeghi-Tehran, P., Sabermanesh, K., Virlet, N. and Hawkesford, M.J., (2017) Automated method to determine two critical growth stages of wheat: Heading and flowering. *Frontiers in Plant Science*, 8.

Safdar, M.F., Alkobaisi, S.S. and Zahra, F.T., (2020) A comparative analysis of data augmentation approaches for magnetic resonance imaging (MRI) scan images of brain tumor. *Acta Informatica Medica*, 281, pp.29–36.

Santosh, K.C., Dhar, M.K., Rajbhandari, R. and Neupane, A., (2020) Deep neural network for foreign object detection in chest X-rays. *Proceedings - IEEE Symposium on Computer-Based Medical Systems*, 2020-July, pp.538–541.

Simonyan, K. and Zisserman, A., (2015) Very deep convolutional networks for large-scale image recognition. *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, pp.1–14.

Soares, E., Angelov, P., Biaso, S., Froes, M.H. and Abe, D.K., (2020) SARS-CoV-2 CT-scan dataset: A large dataset of real patients CT scans for SARS-CoV-2 identification. *medRxiv*, pp.1–8.

Solovyev, R., Wang, W. and Gabruseva, T., (2021) Weighted boxes fusion: Ensembling boxes from different object detection models. *Image and Vision Computing*, 107.

Suchá, D., van Hamersvelt, R.W., van den Hoven, A.F., de Jong, P.A. and Verkooijen, H.M., (2020) Suboptimal Quality and High Risk of Bias in Diagnostic Test Accuracy Studies at Chest Radiography and CT in the Acute Setting of the COVID-19 Pandemic: A Systematic Review. *Radiology: Cardiothoracic Imaging*, 24, p.e200342.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V. and Rabinovich, A., (2015) Going deeper with convolutions. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 07-12-June, pp.1–9.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. and Wojna, Z., (2016) Rethinking the Inception Architecture for Computer Vision. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016-Decem, pp.2818–2826.

Tan, M. and Le, Q. V., (2019) EfficientNet: Rethinking model scaling for convolutional neural networks. *36th International Conference on Machine Learning, ICML 2019*, 2019-June, pp.10691–10700.

Tan, M. and Le, Q. V., (2021) EfficientNetV2: Smaller Models and Faster Training. [online] Available at: <http://arxiv.org/abs/2104.00298>.

Tan, M., Pang, R. and Le, Q. V., (2020) EfficientDet: Scalable and efficient object detection. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp.10778–10787.

The DeepRadiology Team, (2018) Pneumonia Detection in Chest Radiographs. [online] Available at: <http://arxiv.org/abs/1811.08939>.

Tsai, E.B., Simpson, S., Lungren, M.P., Hershman, M., Roshkovan, L., Colak, E., Erickson, B.J., Shih, G., Stein, A., Kalpathy-Cramer, J., Shen, J., Hafez, M., John, S., Rajiah, P., Pogatchnik, B.P., Mongan, J., Altinmakas, E., Ranschaert, E.R., Kitamura, F.C., Topff, L., Moy, L., Kanne, J.P. and Wu, C.C., (2021) The RSNA International COVID-19 Open Radiology Database (RICORD). *Radiology*, 2991, pp.E204–E213.

Usama, S., Safwan, S., Bukhari, K., Syed, A. and Sajid, S., (2020) The evaluation of convolutional neural network (CNN) for the assessment of chest X-ray of COVID-19 patients. *Annals of Clinical and Analytical Medicine*, 116.

Vayá, M. de la I., Saborit, J.M., Montell, J.A., Pertusa, A., Bustos, A., Cazorla, M., Galant, J., Barber, X., Orozco-Beltrán, D., García-García, F., Caparrós, M., González, G. and Salinas, J.M., (2020) BIMCV COVID-19+: a large annotated dataset of RX and CT images from COVID-19 patients. [online] pp.1–22. Available at: <http://arxiv.org/abs/2006.01174>.

Waheed, A., Goyal, M., Gupta, D., Khanna, A., Al-Turjman, F. and Pinheiro, P.R., (2020) CovidGAN: Data Augmentation Using Auxiliary Classifier GAN for Improved Covid-19 Detection. *IEEE Access*, 8, pp.91916–91923.

Wang, H. and Xia, Y., (2018) ChestNet: A Deep Neural Network for Classification of Thoracic Diseases on Chest Radiography. [online] pp.1–8. Available at: <http://arxiv.org/abs/1807.03058>.

Wynants, L., Van Calster, B., Collins, G.S., Riley, R.D., Heinze, G., Schuit, E., Bonten, M.M.J., Damen, J.A.A., Debray, T.P.A., De Vos, M., Dhiman, P., Haller, M.C., Harhay, M.O., Henckaerts, L., Kreuzberger, N., Lohmann, A., Luijken, K., Ma, J., Andaur Navarro, C.L., Reitsma, J.B., Sergeant, J.C., Shi, C., Skoetz, N., Smits, L.J.M., Snell, K.I.E., Sperrin, M., Spijker, R., Steyerberg, E.W., Takada, T., Van Kuijk, S.M.J., Van Royen, F.S., Wallisch, C., Hooft, L., Moons, K.G.M. and Van Smeden, M., (2020) Prediction models for diagnosis and prognosis of covid-19: Systematic review and critical appraisal. *The BMJ*, 369.

Zagoruyko, B.S. and Komodakis, N., (2016) Wide Residual. *Proceedings of the British Machine Vision Conference (BMVC)*. [online] Available at: <https://dx.doi.org/10.5244/C.30.87>.

Zaidi, S.S.A., Ansari, M.S., Aslam, A., Kanwal, N., Asghar, M. and Lee, B., (2021) A Survey of Modern Deep Learning based Object Detection Models. [online] pp.1–18. Available at: <http://arxiv.org/abs/2104.11892>.

Zhu, Y., Zhao, C., Wang, J., Zhao, X., Wu, Y. and Lu, H., (2017) CoupleNet: Coupling Global Structure with Local Parts for Object Detection. *Proceedings of the IEEE International Conference on Computer Vision*, 2017-Octob, pp.4146–4154.

